Lecture notes. Lecture 9

Reservoir computing with echo state networks, spiking neurons and cellular automata

Marcin Płodzień
Deep Learning in Classical and Quantum Physics
Institute of Theoretical Physics, Jagiellonian University in Kraków

1 The cost of trained memory

1.1 Five records and the forecasting task

A far-infrared laser filled with ammonia gas emits light whose intensity pulsates. The amplitude of the pulsations grows slowly over a few hundred oscillations and then collapses within one or two of them, after which the growth starts again. The collapses come at irregular times. A detector records the intensity at equal time intervals, and the result is a list of numbers, a record. This record was distributed as data set A of the Santa Fe time-series competition [1], and it poses the question of this chapter. Given the record up to the present moment, we want its next values.

We write the record as $s_0, s_1, s_2, \ldots$, where k counts the time steps and s_k is the value at step k. Every record of this chapter has 4000 values, rescaled linearly so that the minimum and the maximum of its first 3000 values become 0 and 1. At the present step k, the origin of the forecast, we want the values $s_{k+\tau}$ for the horizons $\tau = 1, 2, \ldots, 50$, and a forecast $\hat{s}_{k+\tau}$ is a number computed from $s_k, s_{k-1}, \ldots$ alone. All 50 forecasts of one origin are collected in the forecast vector $\hat{\mathbf{y}}_k = (\hat{s}_{k+1}, \ldots, \hat{s}_{k+50})$, and the true values in the target vector $\mathbf{y}_k = (s_{k+1}, \ldots, s_{k+50})$.

Besides the laser we use four records generated by equations, whose behaviour is known in advance. The first is the coordinate x of the Lorenz-63 model of thermal convection [2],

$$\dot{x} = 10\,(y - x), \qquad \dot{y} = x\,(28 - z) - y, \qquad \dot{z} = xy - \tfrac{8}{3}\,z, \tag{1.1}$$

sampled every $\Delta t = 0.02$ time units. Its trajectory circles around one of two points, jumps to the other at irregular times and never repeats itself. The other three come from the Mackey–Glass equation, a model of the production of blood cells in which the production rate depends on the number of cells at a time τ_{MG} in the past [3],

$$\dot{s}(t) = \frac{0.2\,s(t - \tau_{\mathrm{MG}})}{1 + s(t - \tau_{\mathrm{MG}})^{10}} - 0.1\,s(t), \tag{1.2}$$

sampled once per time unit at the delays $\tau_{\mathrm{MG}} = 15$, 25 and 35. At $\tau_{\mathrm{MG}} = 15$ the solution is periodic, and at 25 and 35 it is irregular. We call them Lorenz, MG 15, MG 25, MG 35 and Santa Fe.

Each record is divided in time into three blocks, the first 2500 values for training, the next 500 for validation and the last 1000 for testing, with the origins placed so that no target crosses the border of a block. The first training origin is $k = 63$, so that AR, NVAR and the trained networks have the 64 values $s_{63}, s_{62}, \ldots, s_0$ available, the longest window of past values that any of them uses. Every model is fitted on the training block. Every choice between variants of a model, such as its size or a penalty strength, is made by its score on the validation block, and the test block is used once, for the final scores. This order keeps the test scores free of the choices made while building the model. Figure 1 shows the five records and the three blocks.

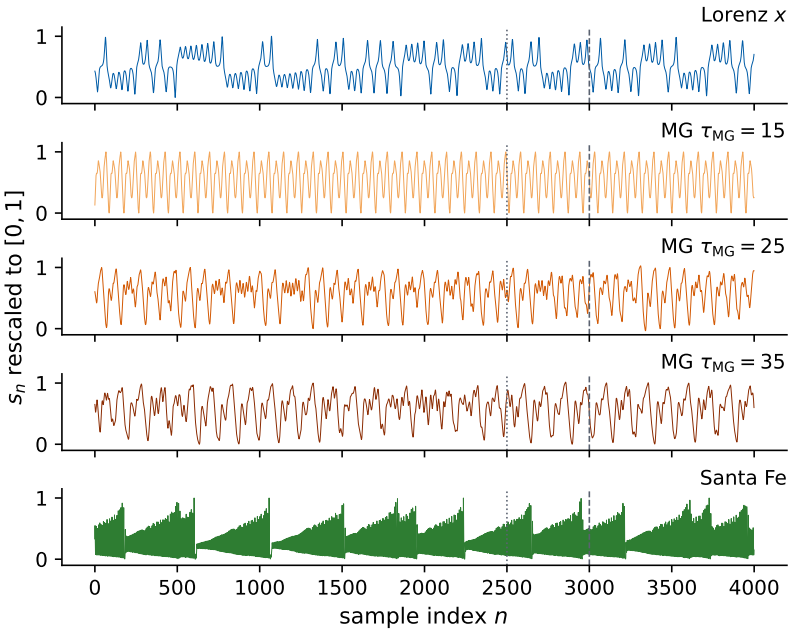


Figure 1: The five records, each rescaled to $[0, 1]$, against the step (labelled n on the axis, the step k of the text). From the top, the coordinate x of the Lorenz-63 model, the Mackey–Glass equation at the delays $\tau_{\mathrm{MG}} = 15$, 25 and 35, and the intensity of the Santa Fe laser. The dotted line separates the training block from the validation block, the dashed line the validation block from the test block.

1.2 Scoring a forecast

A forecast at the horizon τ is compared with the truth over all origins of the test block. Its score is built from the Pearson correlation coefficient of two lists of numbers $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$,

$$r(a, b) \coloneqq \frac{\sum_j (a_j - \bar{a})(b_j - \bar{b})}{\sqrt{\sum_j (a_j - \bar{a})^2 \, \sum_j (b_j - \bar{b})^2}}, \tag{1.3}$$

where $\bar{a}$ and $\bar{b}$ are the means. It lies between -1 and 1 and equals ± 1 when the points (a_j, b_j) lie on a straight line. The capacity at the horizon τ is its square,

$$C(\tau) \coloneqq r\big(\hat{s}_{k+\tau}, s_{k+\tau}\big)^2, \qquad C_{\mathrm{tot}} \coloneqq \sum_{\tau=1}^{50} C(\tau), \tag{1.4}$$

with the correlation taken over the test origins k. The capacity is the fraction of the variance of the true values that the best straight line through the forecasts accounts for, and the total capacity C_{tot}, between 0 and 50, summarises a whole forecast.

Figure 2 shows what the capacity rewards. A forecast scattered around the truth reaches $C = 0.81$. A forecast that is a straight-line function of the truth reaches $C = 1$, even when its scale is wrong or its sign is reversed. The capacity therefore measures how much information about the future the forecast carries, and it ignores errors that one straight-line correction would remove. This is its limit, since a forecast with $C = 1$ may still need such a correction before it is used.

The simplest forecast repeats the present value, $\hat{s}_{k+\tau} = s_k$, and is called persistence. It reaches $C_{\mathrm{tot}} = 8.4$ on Lorenz, 19.5 on MG 15 and 5.5 on Santa Fe. A model is useful only where it does clearly better than these numbers.

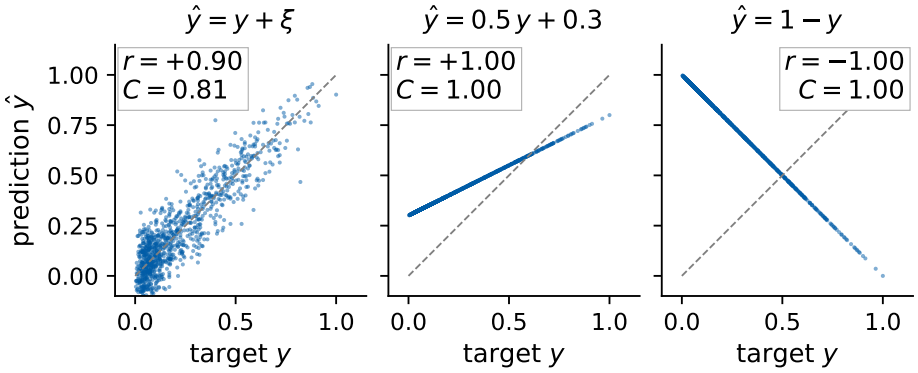


Figure 2: Three predictions $\hat{y}$ against the same targets y, with the correlation r and the capacity $C = r^2$. Left, the target plus noise ξ. Middle, a straight-line function of the target with the wrong scale and offset. Right, the target reversed. The dashed line is the diagonal $\hat{y} = y$.

1.3 A recurrent network

A forecast of the laser needs the recent history of the record, since one value of the intensity does not tell whether a pulsation is rising or falling, or how far the slow growth has come. A recurrent neural network (RNN) keeps this history in a state that it carries from step to step [4],

$$\mathbf{h}_k = \tanh\left(W_h \mathbf{h}_{k-1} + \mathbf{w}_x s_k + \mathbf{b}_h\right), \qquad \hat{\mathbf{y}}_k = W_y \mathbf{h}_k + \mathbf{b}_y. \tag{1.5}$$

The hidden state $\mathbf{h}_k \in \mathbb{R}^{n_h}$ holds n_h numbers, the matrix W_h feeds the old state back, the vector $\mathbf{w}_x$ brings in the new value, $\mathbf{b}_h$ is a constant offset, and the hyperbolic tangent acts on every component. The matrix W_y and the vector $\mathbf{b}_y$ form a linear readout that turns the state into the 50 forecasts. Since $\mathbf{h}_k$ depends on $\mathbf{h}_{k-1}$, which depends on $\mathbf{h}_{k-2}$, the state depends on the whole past of the record. The network is itself a dynamical system driven by the record, and its state is its memory.

1.4 Training through time

All weights of the network, W_h, $\mathbf{w}_x$, $\mathbf{b}_h$, W_y and $\mathbf{b}_y$, are fitted by gradient descent on the squared error $\sum_k \|\hat{\mathbf{y}}_k - \mathbf{y}_k\|^2$. A change of the state m steps back reaches the present state through the chain rule, one step at a time. Writing $\mathbf{a}_k = W_h \mathbf{h}_{k-1} + \mathbf{w}_x s_k + \mathbf{b}_h$, so that $\mathbf{h}_k = \tanh \mathbf{a}_k$, the derivative of one step is

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \Gamma_k W_h, \qquad \Gamma_k \coloneqq \operatorname{diag}\left(1 - \tanh^2 \mathbf{a}_k\right), \tag{1.6}$$

since $\tanh'(a) = 1 - \tanh^2 a$. The diagonal matrix Γ_k has entries between 0 and 1, close to 1 for a unit near zero and close to 0 for a saturated unit. Over m steps the derivatives multiply,

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-m}} = \Gamma_k W_h \, \Gamma_{k-1} W_h \cdots \Gamma_{k-m+1} W_h. \tag{1.7}$$

Every contribution of an input m steps old to the gradient carries this product of m matrices. The norm of a product is at most the product of the norms, and the norm of Γ_i is at most 1, so the norm of the product is at most $\|W_h\|^m$, where $\|W_h\|$ is the largest singular value of W_h, the largest factor by which the matrix can stretch a vector. Products of many matrices typically shrink or grow exponentially with their number.

Figure 3 shows this for untrained networks with $n_h = 32$ units whose matrix W_h has all singular values equal to ρ, driven by windows of the laser record. After a few steps every curve is a straight line on the logarithmic scale. For $\rho = 0.5$ the logarithm of the norm changes by -0.755 per step, a little faster than the bound $\ln 0.5 = -0.693$, and over 63 steps this rate gives a factor of about 10^{-21}. For $\rho = 1$ the bound allows no decay, and still the norm falls at the rate -0.146, because the factors Γ_i shrink every vector a little. For $\rho = 1.5$ the norm grows at the rate $+0.030$, far below the bound $\ln 1.5 = 0.405$, because large states saturate the units and make Γ_i small. For $\rho \le 1$ the oldest inputs

contribute to the gradient only with factors of about 10^{-4} at $\rho = 1$ and 10^{-21} at $\rho = 0.5$, the problem of vanishing gradients. For $\rho > 1$ the bound allows exponential growth with the age of the input, the problem of exploding gradients [5], and here the growth is mild, a factor of about $e^{0.030 \cdot 63} \approx 6.6$ over 63 steps.

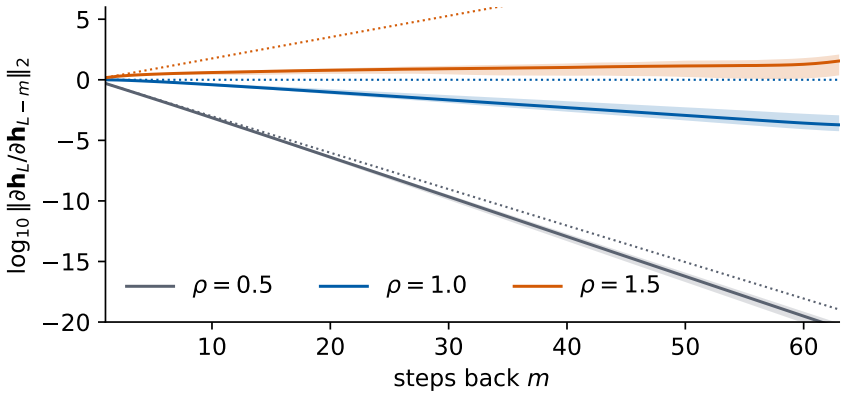


Figure 3: The product of step derivatives of equation (1.7) in untrained recurrent networks with 32 units. The decimal logarithm of its largest singular value against the number of steps back m, for networks whose matrix W_h has all singular values equal to ρ. Lines, the mean over 10 random networks, bands, their range, dotted, the bound $m \log_{10} \rho$.

Training a recurrent network is therefore slow and needs care, such as a careful initialisation of W_h and rescaling of the gradient whenever its norm exceeds a threshold, called gradient clipping [6]. Each gradient step passes through the whole history, and many such steps are needed. The memory of the network, the recurrence that makes it useful for a record, is the part that makes its training hard. This leaves a plain possibility. The recurrence could be kept and its training dropped.

2 A fixed random system as a feature map

The recurrent network of equation (1.5) owes its memory to the recurrence and its slow training to the same recurrence, through the product of equation (1.7). If the features of a forecast were computed by a recurrence that is never trained, the only fitted part would be a linear map from features to forecasts, and fitting it would be one linear solve. This idea is worth something only if a linear map of fixed features can forecast a nonlinear record such as the laser, and only if the fit can be done reliably when the features are many and nearly dependent. The simplest feature with memory, one leaky unit, then shows what such a fixed system can and cannot provide.

2.1 Features and a linear readout

Every forecasting model of this chapter has two parts. A feature map turns the past of the record into a vector $\mathbf{x}_k \in \mathbb{R}^M$ of M numbers, the features, and a linear readout turns the features into forecasts,

$$\hat{\mathbf{y}}_k = W_{\text{out}}^\top \mathbf{x}_k + \mathbf{b}, \tag{2.1}$$

with an $M \times 50$ matrix of readout weights W_{out} and an offset $\mathbf{b} \in \mathbb{R}^{50}$. The linear autoregressive model (AR) of order p uses the last p values as features, $\mathbf{x}_k = (s_k, s_{k-1}, \ldots, s_{k-p+1})$. Nonlinear vector autoregression (NVAR) adds all products $s_{k-i} s_{k-j}$ of two of these values [8]. In both the feature map is a fixed formula chosen in advance. The recurrent network of equation (1.5) computes its features by a recursion, $\mathbf{h}_k = \mathcal{F}(\mathbf{h}_{k-1}, s_k)$, and learns the law $\mathcal{F}$ together with the readout.

A reservoir combines the two. Its features are the state $\mathbf{r}_k \in \mathbb{R}^N$ of a driven dynamical system,

$$\mathbf{r}_k = \mathcal{F}(\mathbf{r}_{k-1}, s_k), \qquad \mathbf{x}_k = \mathbf{r}_k, \tag{2.2}$$

whose law $\mathcal{F}$ is drawn at random once and never changed. The state carries memory like the state of a recurrent network, and only the readout of equation (2.1) is fitted. Figure 4 draws the idea. The record enters the system one value at a time, the state moves through a space of N dimensions, and the whole record becomes a path in that space. A forecast is a fixed linear function of the present point of the path.

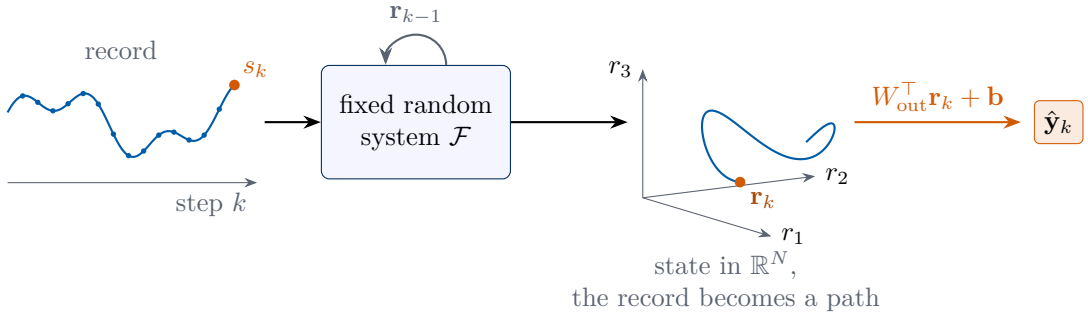


Figure 4: A reservoir as a feature map. The record drives a fixed random dynamical system, whose state $\mathbf{r}_k$ depends on the present value s_k and on the previous state $\mathbf{r}_{k-1}$. The record becomes a path in the space of states, drawn here for three of the N components. Only the linear readout (orange) to the forecast vector $\hat{\mathbf{y}}_k$ is fitted.

2.2 Lifting into more dimensions

A linear readout can forecast a nonlinear system when the features are suitable nonlinear functions of the state. A small map shows the mechanism. Let two numbers evolve by

$$A_{k+1} = \gamma_1 A_k, \qquad Z_{k+1} = \gamma_2 Z_k + A_k^2, \tag{2.3}$$

with constants γ_1 and γ_2. The map is nonlinear because of the square. The three functions (A, Z, A^2) of the state, however, evolve by a matrix,

$$\begin{pmatrix} A \\ Z \\ A^2 \end{pmatrix}_{k+1} = \begin{pmatrix} \gamma_1 & 0 & 0 \\ 0 & \gamma_2 & 1 \\ 0 & 0 & \gamma_1^2 \end{pmatrix} \begin{pmatrix} A \\ Z \\ A^2 \end{pmatrix}_k , \tag{2.4}$$

since $A_{k+1}^2 = \gamma_1^2 A_k^2$. Applying the matrix τ times gives $Z_{k+\tau}$ as a fixed linear combination of the three numbers at step k, for every horizon. The extra function A^2 lifts the system into three dimensions, where its evolution is linear. A set of functions closes under the evolution when the value of each of them at the next step is a linear combination of the values of all of them at the present step, as for the three functions here. Koopman showed for Hamiltonian systems that the evolution acts linearly on functions of the state [9]. The same holds for any deterministic map $\mathbf{v} \mapsto \boldsymbol{\varphi}(\mathbf{v})$, since a function h of the state becomes the function $h \circ \boldsymbol{\varphi}$ one step later, and this replacement is linear in h. The difficulty is that the space of all functions has infinitely many dimensions.

For irregular motions such as the Lorenz record a finite set of functions that closes in this way is in general not available, and a finite set on which the evolution acts approximately linearly turns forecasting into an approximate linear regression. AR and NVAR choose a few such functions in advance. A reservoir driven by the record produces N nonlinear functions of the input history at random, and the readout selects a linear combination of them. If the functions that a forecast needs lie close to the span of the reservoir states, a linear readout forecasts well. This picture is qualitative. Nothing guarantees that a random set of functions contains the right ones [10], and the quality of a reservoir has to be measured.

2.3 Fitting the readout by ridge regression

The readout is fitted by least squares with a penalty on the size of the weights, a method called ridge regression [11]. The matrix X has one row for every training origin, the feature vector $\mathbf{x}_k^\top$, and the

matrix Y has the target vectors $\mathbf{y}_k^\top$ as its rows. The weights minimise

$$E(W_{\text{out}}, \mathbf{b}) = \sum_k \left\| W_{\text{out}}^\top \mathbf{x}_k + \mathbf{b} - \mathbf{y}_k \right\|^2 + \beta \left\| W_{\text{out}} \right\|_F^2, \tag{2.5}$$

where $\|W_{\text{out}}\|_F^2$, the squared Frobenius norm, is the sum of the squares of all weights and $\beta > 0$ is the ridge strength. Setting the derivative with respect to the offset to zero gives $\sum_k (W_{\text{out}}^\top \mathbf{x}_k + \mathbf{b} - \mathbf{y}_k) = 0$, so

$$\mathbf{b} = \bar{\mathbf{y}} - W_{\text{out}}^\top \bar{\mathbf{x}}, \tag{2.6}$$

with the means $\bar{\mathbf{x}}$ and $\bar{\mathbf{y}}$ over the training origins. Substituting this offset back replaces every feature vector by its deviation from the mean and every target by its deviation from the mean. With the centred matrices X_c and Y_c the error becomes $\|X_c W_{\text{out}} - Y_c\|_F^2 + \beta \|W_{\text{out}}\|_F^2$, and its derivative with respect to W_{out}, $2X_c^\top (X_c W_{\text{out}} - Y_c) + 2\beta W_{\text{out}}$, vanishes for

$$\left(X_c^\top X_c + \beta \, \mathbb{1} \right) W_{\text{out}} = X_c^\top Y_c, \tag{2.7}$$

where $\mathbb{1}$ is the $M \times M$ identity matrix. Equation (2.7) is a set of linear equations, the normal equations, and the fit is one linear solve. All 50 horizons share the matrix on the left and differ only in the column of Y_c, so they are solved together.

The centring keeps the offset out of the penalty, so a record with a large mean is not punished for it. The penalty itself has a reason. Features of a reservoir are often nearly linear combinations of each other, the matrix $X_c^\top X_c$ then has eigenvalues close to zero, and without the penalty the solution would have huge weights of opposite signs that cancel on the training data and amplify every small change of the features. Adding $\beta\mathbb{1}$ raises every eigenvalue by β. For a single feature the solution reads $w = \sum_k x_{c,k} y_{c,k} / (\sum_k x_{c,k}^2 + \beta)$, the least-squares slope shrunk towards zero by the penalty. The ridge strength is chosen on the validation block.

2.4 One leaky unit

The simplest system with memory is one unit that keeps a fraction of its old value and adds a fraction of a new input,

$$x_k = (1 - \alpha)\, x_{k-1} + \alpha\, g_k, \qquad 0 < \alpha \le 1, \tag{2.8}$$

where x_k is the state, g_k the input at step k, and α is called the leak rate. Inserting the equation into itself repeatedly gives

$$\begin{aligned} x_k &= (1-\alpha)^2 x_{k-2} + \alpha(1-\alpha)\, g_{k-1} + \alpha\, g_k = \dots \\ &= (1-\alpha)^k x_0 + \alpha \sum_{j=0}^{k-1} (1-\alpha)^j g_{k-j}. \end{aligned} \tag{2.9}$$

The state is a weighted sum of all past inputs. An input of age j enters with the weight $\alpha(1-\alpha)^j$, and the starting value with the weight $(1-\alpha)^k$. The weights of the inputs add up to $1 - (1-\alpha)^k$, which approaches 1, so a constant input g drives the state to g. For $\alpha = 0.1$ the weights are 0.1 for the present input, 0.09 for the previous one, 0.035 at the age $j = 10$ and 0.012 at $j = 20$. Their mean age is $\sum_j j\, \alpha(1-\alpha)^j = (1-\alpha)/\alpha = 9$ steps. A weight falls by the factor e over $-1/\ln(1-\alpha) = 9.5$ steps, which is close to $1/\alpha$ for a small leak rate. The leak rate therefore sets a memory time of about $1/\alpha$ steps.

Two limits bracket this behaviour. Without the factor $(1-\alpha)$ in front of x_{k-1} the unit becomes an integrator, $x_k = x_{k-1} + \alpha g_k$, which adds every input it has ever received and never forgets. For $\alpha = 1$ the unit copies the present input, $x_k = g_k$, and has no memory at all. Figure 5 drives the three units with the same four unit pulses at the steps 10, 40, 45 and 80. The integrator steps up by 0.1 at every pulse and stays at 0.40 after the last one. The memoryless unit returns to zero after each pulse. The leaky unit decays between the pulses, and after the last pulse it falls to the fraction 0.016 of its value in 39 steps, which is $(1-\alpha)^{39}$.

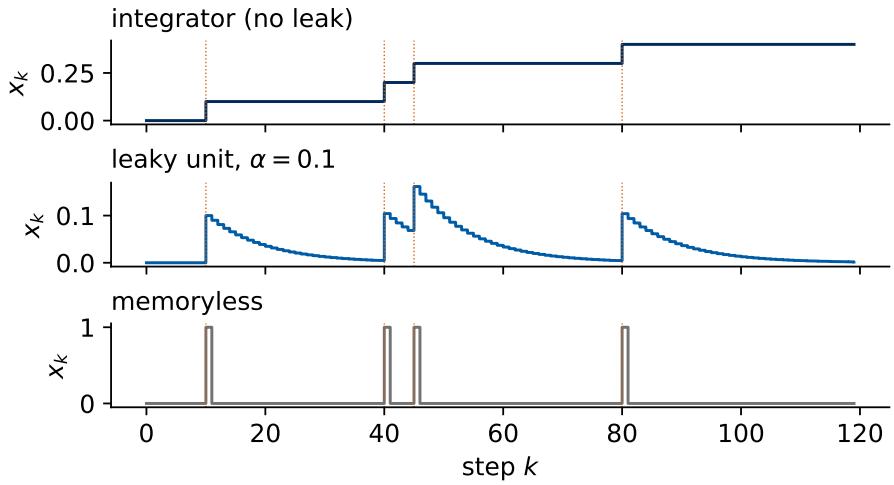


Figure 5: Three units driven by the same four unit pulses (dotted lines). Top, an integrator without leak. Middle, a leaky unit with $\alpha = 0.1$, equation (2.8). Bottom, a unit without memory. The state x_k against the step k.

3 The echo state network

One leaky unit, equation (2.8), weights past inputs by a single exponential and responds linearly. A forecast of the laser needs inputs of several ages weighted differently and products of past values, and no single linear filter supplies them. Many leaky units with a nonlinear response, coupled to each other at random, could supply both, provided the coupled system stays well behaved. The echo state network is this construction, and its three parameters decide how long it remembers and how nonlinearly it responds.

3.1 Definition of the echo state network

The echo state network (ESN) of Jaeger [12, 13] couples N leaky units through a fixed random matrix and a hyperbolic tangent. The size of the matrix is set by its spectral radius, the largest modulus of its eigenvalues, which governs how fast powers of the matrix grow or shrink.

Echo state network

The state $\mathbf{r}_k \in \mathbb{R}^N$ is driven by the record through

$$\mathbf{r}_k = (1-\alpha)\,\mathbf{r}_{k-1} + \alpha \tanh\left(W\mathbf{r}_{k-1} + \mathbf{w}_{\text{in}} s_k\right), \qquad W \coloneqq \rho\,\widetilde{W}. \tag{3.1}$$

$\widetilde{W}$ is a sparse random $N \times N$ matrix with 10% nonzero entries drawn from a Gaussian distribution, divided by the largest modulus of its eigenvalues. Its eigenvalues are written μ_i, so that $\max_i |\mu_i| = 1$, and W has the eigenvalues $\rho\mu_i$ and the spectral radius ρ. The input weights $\mathbf{w}_{\text{in}} \in \mathbb{R}^N$ are drawn uniformly from $[-\sigma_{\text{in}}, \sigma_{\text{in}}]$, the input scaling. The leak rate $\alpha \in (0, 1]$ is the one of equation (2.8). W and $\mathbf{w}_{\text{in}}$ are drawn once and never trained, and only the readout of equation (2.1) with $\mathbf{x}_k = \mathbf{r}_k$ is fitted by ridge regression.

Component i of equation (3.1) is the leaky unit of equation (2.8) with the input $g_{i,k} = \tanh\left(\sum_j W_{ij} r_{j,k-1} + w_{\text{in},i} s_k\right)$. The record flows into every unit with its own weight, every unit flows into the others through the matrix W, whose entry W_{ij} says how strongly and with which sign unit j feeds unit i, and the hyperbolic tangent limits this inflow however strongly it is pushed. For $\alpha = 1$ the update is the recurrent network of equation (1.5) without the offset, with random weights that are never changed. Figure 6 draws the network.

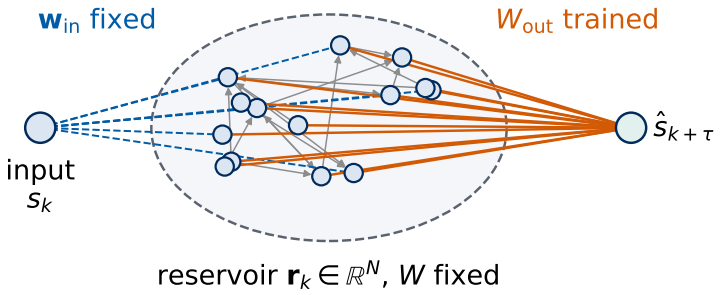


Figure 6: An echo state network. The input s_k reaches the units through the fixed input weights $\mathbf{w}_{\text{in}}$ (dashed blue), the units are coupled by the fixed sparse matrix W (grey arrows), and only the readout W_{out} (orange) to the forecast $\hat{s}_{k+\tau}$ is trained.

3.2 The three parameters

The input scaling σ_{in} sets how far the units are pushed into the curved part of the hyperbolic tangent. For a small σ_{in} the units stay near zero, where $\tanh a \approx a$, and respond almost linearly. For a large one they saturate, and products of inputs appear in their response.

The spectral radius and the leak rate set how slowly the reservoir forgets. Near the state $\mathbf{r} = 0$ and without input the hyperbolic tangent is close to the identity, and equation (3.1) becomes linear,

$$\mathbf{r}_k \approx \left[(1-\alpha)\,\mathbb{1} + \alpha\rho\,\widetilde{W}\right]\mathbf{r}_{k-1}. \tag{3.2}$$

An eigenvector of $\widetilde{W}$ with eigenvalue μ_i is also an eigenvector of the matrix in brackets, with the eigenvalue

$$(1-\alpha) + \alpha\rho\,\mu_i. \tag{3.3}$$

A small perturbation along this eigenvector is multiplied by this number at every step, so it shrinks when its modulus is below 1 and grows when it is above. When the eigenvalue of largest modulus is real and positive, $\mu_i = 1$, the slowest perturbation shrinks by the factor $(1-\alpha)+\alpha\rho$ per step, the effective decay per step. For $\alpha = 1$ it is ρ. For $\alpha < 1$ and $\rho < 1$ it lies between ρ and 1, so a small leak rate lengthens the memory. The number of steps over which a perturbation falls by the factor e, the memory time $-1/\ln[(1-\alpha)+\alpha\rho]$, is 4.5 steps for $\rho = 0.8$ and $\alpha = 1$, and 16.2 steps for $\rho = 0.8$ and $\alpha = 0.3$.

A large random matrix has its eigenvalues spread over a disc in the complex plane (Figure 7), and the eigenvalue of largest modulus is in general complex. For one reservoir of 300 units drawn in this way it is $-0.121 + 0.993\,i$. For a complex μ_i and $\alpha < 1$ the modulus of the number (3.3) is smaller than $(1-\alpha)+\alpha\rho$, and the formula for the memory time becomes an upper bound. At $\rho = 0.8$ and $\alpha = 0.3$ the largest eigenvalue modulus of the linearised update of this reservoir, computed from its eigenvalues, is 0.924 against 0.940 from the formula, which gives a memory time of 12.6 steps against 16.2, still about three times the value at $\alpha = 1$.

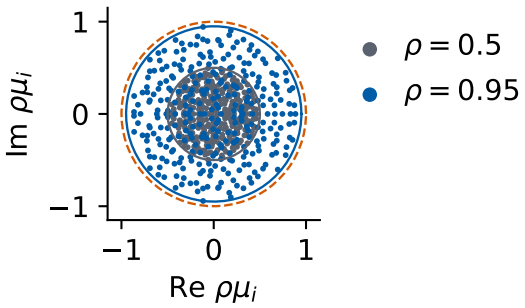


Figure 7: Eigenvalues $\rho\mu_i$ of the recurrent matrix W of a reservoir with $N = 300$ units in the complex plane, for $\rho = 0.5$ (grey) and $\rho = 0.95$ (blue), with circles of radius ρ and the unit circle (dashed orange).

3.3 Two units

A reservoir of two units shows these numbers on paper. The values are chosen for illustration. Take

$$W = \begin{pmatrix} 0.3 & 1.8 \\ 0.3 & 0 \end{pmatrix}, \qquad \mathbf{w}_{\text{in}} = \begin{pmatrix} 1 \\ -0.5 \end{pmatrix}. \tag{3.4}$$

The eigenvalues $\rho\mu_i$ of W solve $(\rho\mu)^2 - 0.3\,\rho\mu - 0.54 = 0$, which gives $\rho\mu_1 = 0.9$ and $\rho\mu_2 = -0.6$, so the spectral radius is $\rho = 0.9$, with $\mu_1 = 1$ and $\mu_2 = -2/3$. For $\alpha = 1$ the two modes of the linearised update decay by the factors 0.9 and -0.6 per step, the second one with a change of sign at every step, and their memory times are 9.5 and 2.0 steps. For $\alpha = 0.5$ the factors (3.3) become $0.5 + 0.5 \cdot 0.9 = 0.95$ and $0.5 + 0.5 \cdot (-0.6) = 0.2$, with the memory times 19.5 and 0.6 steps. The leak doubles the memory of the slow mode and removes the sign changes of the fast one.

A direct simulation confirms the linear picture. Two copies of this reservoir started from $\mathbf{r}_0 = (1, -1)$ and $\mathbf{r}'_0 = (-1, 1)$ without input approach each other, and between the steps 20 and 40 their distance shrinks by the factor 0.900 per step for $\alpha = 1$ and 0.949 for $\alpha = 0.5$, the factors of the slow mode. The matrix of equation (3.4) also has a property that matters when the starting state has to be forgotten. Its largest singular value is 1.83, twice its spectral radius, because the matrix is far from symmetric. Over a single step it can stretch some differences of states, although every difference shrinks over many steps.

3.4 Response to a record and the washout

Figure 8 shows a reservoir of $N = 300$ units with $\rho = 0.95$ and $\sigma_{\text{in}} = 0.5$, started from $\mathbf{r}_0 = 0$ and driven by the laser record. Every unit follows the pulsations with its own amplitude, sign and phase, so the readout receives many different nonlinear functions of the record. The smaller leak rate smooths the response and slows it down. The first tens of steps depend visibly on the arbitrary start $\mathbf{r}_0 = 0$. The states of these first steps, the washout, are left out of the fit. With the first training origin at $k = 63$ the washout consists of the states after the 63 values $s_0, \ldots, s_{62}$, and the first state used in the fit has also received s_{63}.

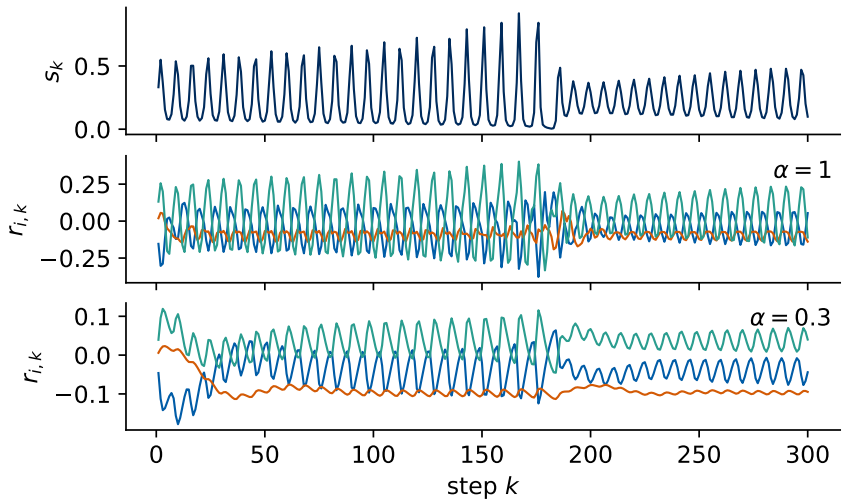


Figure 8: Response of an echo state network with 300 units, $\rho = 0.95$ and $\sigma_{\text{in}} = 0.5$, to the laser record. Top, the rescaled input s_k against the step k. Middle and bottom, three components $r_{i,k}$ of the state for the leak rates $\alpha = 1$ and $\alpha = 0.3$.

4 The echo state property

The washout of Figure 8 rests on an assumption, that the influence of the arbitrary start $\mathbf{r}_0 = 0$ dies out after some steps. Nothing in equation (3.1) guarantees it, and the two-unit reservoir of

equation (3.4) already showed that a single step can stretch the difference between two states. If the start were never forgotten, a readout fitted on one run would describe that run only. The question is when a driven reservoir forgets where it started, and how this can be checked for a given reservoir and record.

4.1 Two copies of one reservoir

The question whether a reservoir forgets its start can be put to a direct test. Two copies of the same reservoir, with the same W and $\mathbf{w}_{\text{in}}$, are started from two different states $\mathbf{r}_0$ and $\mathbf{r}'_0$ and driven by the same record. If the distance between their states shrinks to zero, the state at late times is fixed by the record alone, and the start has been forgotten (Figure 9). The state is then an echo of the input history, which gave the network its name.

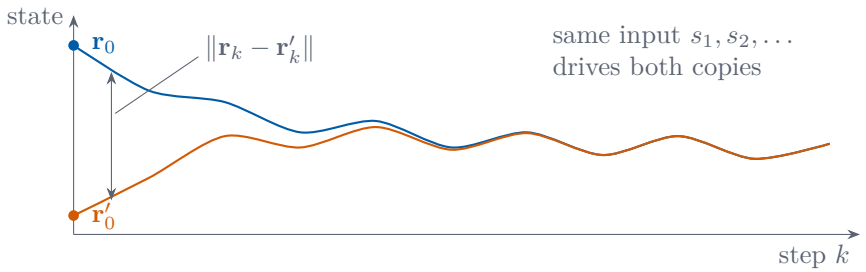


Figure 9: The echo state property. Two copies of one reservoir start from different states $\mathbf{r}_0$ and $\mathbf{r}'_0$ and receive the same input. When their distance shrinks to zero, the late states no longer depend on the start. The curves are a sketch, one component of the state against the step.

Echo state property

A reservoir has the echo state property for a set of admissible input sequences, here sequences with values in a bounded interval, when for every such sequence and every pair of starting states the two copies approach each other,

$$\|\mathbf{r}_k - \mathbf{r}'_k\| \to 0 \qquad (k \to \infty). \tag{4.1}$$

The property is needed for two reasons. A readout fitted on one run of the reservoir has to work on another run, which starts from another state or from another point of the record, and it can do so only if the state does not depend on the start. The washout relies on the same property, since leaving out the first states is useful only if the states after it have forgotten $\mathbf{r}_0$.

4.2 A sufficient condition

For $\alpha = 1$ the two copies obey $\mathbf{r}_k = \tanh(W\mathbf{r}_{k-1} + \mathbf{w}_{\text{in}}s_k)$ and $\mathbf{r}'_k = \tanh(W\mathbf{r}'_{k-1} + \mathbf{w}_{\text{in}}s_k)$. By the mean value theorem $\tanh a - \tanh b = \tanh'(c)\,(a - b)$ for some c between a and b, and since $0 < \tanh' \le 1$,

$$|\tanh a - \tanh b| \le |a - b|. \tag{4.2}$$

Applied to every component, with $\mathbf{a} = W\mathbf{r}_{k-1} + \mathbf{w}_{\text{in}}s_k$ and $\mathbf{a}' = W\mathbf{r}'_{k-1} + \mathbf{w}_{\text{in}}s_k$, this gives $\|\mathbf{r}_k - \mathbf{r}'_k\| \le \|\mathbf{a} - \mathbf{a}'\|$. The input is the same in both copies and drops out of the difference, $\mathbf{a} - \mathbf{a}' = W(\mathbf{r}_{k-1} - \mathbf{r}'_{k-1})$, and the largest singular value $\sigma_{\max}(W)$ is the largest factor by which W stretches a vector. Hence

$$\|\mathbf{r}_k - \mathbf{r}'_k\| \le \sigma_{\max}(W)\,\|\mathbf{r}_{k-1} - \mathbf{r}'_{k-1}\| \le \cdots \le \sigma_{\max}(W)^k\,\|\mathbf{r}_0 - \mathbf{r}'_0\|, \tag{4.3}$$

which goes to zero for every input when $\sigma_{\max}(W) < 1$. For the leaky update the difference of the two copies is $(1 - \alpha)(\mathbf{r}_{k-1} - \mathbf{r}'_{k-1})$ plus α times the difference of the hyperbolic tangents, and the same steps give the factor $(1 - \alpha) + \alpha\,\sigma_{\max}(W)$ per step. It is below 1 under the same condition, $\sigma_{\max}(W) < 1$, and it approaches 1 as $\alpha \to 0$, so a small leak rate slows the contraction.

The condition is sufficient and often far from necessary. The largest singular value is at least the spectral radius, and for a matrix far from symmetric it can be much larger. The two-unit reservoir of equation (3.4) has $\rho = 0.9$ and $\sigma_{\max} = 1.83$. The bound (4.3) says nothing for it, and indeed a single step can stretch a difference. The states $(0, 0.1)$ and $(0, -0.1)$, a distance 0.2 apart, are mapped without input to states a distance 0.36 apart. Over many steps the copies still converge, as the simulation of the two copies at zero input showed, because the product of many steps shrinks every difference by about ρ^k.

4.3 A necessary condition

At zero input, $s_k = 0$ for all k, the state $\mathbf{r} = 0$ is a fixed point of equation (3.1), since $\tanh 0 = 0$. Near it the update is the linear map (3.2), whose eigenvalues are the numbers (3.3). When one of them has a modulus above 1, for instance when $\rho > 1$ and the eigenvalue of largest modulus is real and positive, a small perturbation along the corresponding eigenvector grows. A copy started at $\mathbf{r}_0 = 0$ then stays at zero forever, while a copy started next to it moves away, and the two never meet. The two-unit reservoir rescaled to $\rho = 1.2$ shows this. At zero input and $\alpha = 1$, a copy started at $(10^{-3}, 0)$ ends at the fixed point $(0.774, 0.300)$, a copy started at $(-10^{-3}, 0)$ at $(-0.774, -0.300)$, and a copy started at zero stays there. Whenever the zero sequence is an admissible input, $\rho \leq 1$ is therefore necessary when the leading eigenvalue is real and positive, $\mu_i = 1$.

Between the sufficient condition $\sigma_{\max}(W) < 1$ and the necessary condition at zero input there is a gap, and no condition is known that is both necessary and sufficient [15]. In practice ρ is used as a control parameter, and the property is tested directly with the input at hand.

4.4 Testing the property

Figure 10 shows the test for one reservoir of $N = 100$ units with $\alpha = 1$ and $\sigma_{\text{in}} = 0.5$. The two copies start from random states with entries uniform in $[-1, 1]$ and are driven by the first 500 values of the laser record. For $\rho = 0.8$ the distance decays exponentially and reaches 3.2×10^{-16} after 500 steps, the level of round-off errors of double-precision arithmetic, and it falls below 10^{-14} at step 138. For $\rho = 1.0$ the decay is much slower, and the distance after 500 steps is 6.8×10^{-12}. For $\rho = 1.2$ and 2.0 the distance stays of order one over all 500 steps, 4.7×10^{0} and 1.0×10^{1} at the end, and so does the run at $\rho = 1.2$ without input. In these cases the state keeps a memory of its start, and a readout trained on one run does not transfer to another.

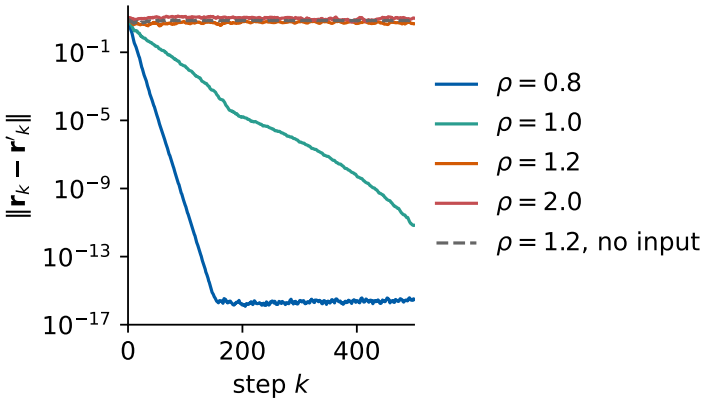


Figure 10: Test of the echo state property. The distance $\|\mathbf{r}_k - \mathbf{r}'_k\|$ between two copies of a reservoir with $N = 100$ units, $\alpha = 1$ and $\sigma_{\text{in}} = 0.5$, started from different random states and driven by the same laser record, against the step k on a logarithmic scale, for four spectral radii. Dashed, $\rho = 1.2$ without input.

The sufficient condition explains little of this figure. The largest singular value of the matrix $\widetilde{W}$ of this reservoir is 2.21, so $\sigma_{\max}(\rho\widetilde{W}) < 1$ holds only for $\rho < 0.45$. The convergence at $\rho = 0.8$ and $\rho = 1.0$ is a property of this reservoir and this input that the bound does not guarantee.

4.5 The input helps the contraction

The test is repeated on a grid of spectral radii for four input scalings, with 1000 input steps and three pairs of starts, and Figure 11 shows the median distance over the last 100 steps. The distance drops to round-off below a threshold value of ρ and jumps to order one above it. The first value of ρ on the grid with a distance above 10^{-6} is 1.00 for $\sigma_{\text{in}} = 0.1$, 1.10 for 0.5, 1.25 for 1 and 1.45 for 2, on a grid of step 0.05.

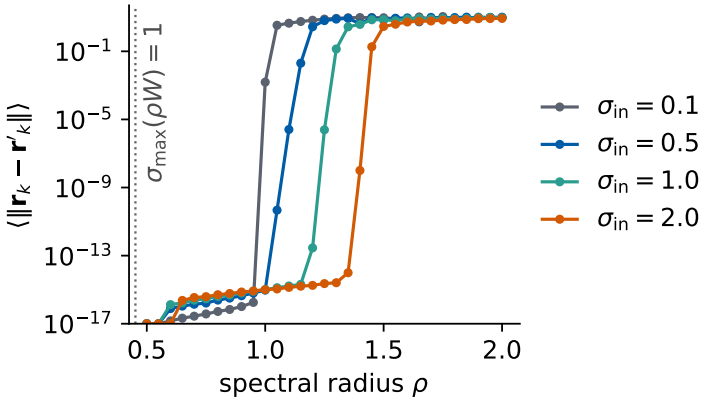


Figure 11: The echo state property against the spectral radius, for the reservoir of Figure 10. The median, over three pairs of starts, of the mean distance between the two copies over the last 100 of 1000 input steps, on a logarithmic scale, against ρ for four input scalings σ_{in}. Dotted, the sufficient bound $\sigma_{\max}(\rho\widetilde{W}) = 1$.

A strong input pushes the units towards saturation, where the derivative $\tanh'$ is small. Two nearby states then differ after one step by about $\Gamma_k W$ times their difference, with the diagonal matrix Γ_k of the derivatives $1 - \tanh^2$ that appeared in equation (1.7), and a small Γ_k lowers the effective gain of the recurrence. The two-unit reservoir shows the effect in numbers. Driven by the constant input $s_k = 0.5$, it settles at a state where the two derivatives are 0.921 and 0.973, and two copies then approach each other by the factor 0.848 per step, against 0.900 at zero input. The common rule $\rho < 1$ is therefore a working rule for weak inputs. With a strong input a reservoir above $\rho = 1$ can still have the property for that input. A reservoir of 300 units at $\rho = 1.1$ with $\sigma_{\text{in}} = 1$, driven by the Lorenz record, is an example. Two copies started from random states differ by 4.5×10^{-3} after 63 steps, by 2.7×10^{-9} after 263 steps, and by less than 4×10^{-15} on all origins of the test block. Such a statement holds for one reservoir and one input, and it has to be checked again whenever either changes.

5 Fading memory

A reservoir that forgets its start can still remember too much of its input. Its state at the present step could depend on values of the laser record from a thousand steps back, which carry no information about the next pulsations and would make the readout respond to an irrelevant past. The leaky unit forgets old inputs with the weights $\alpha(1-\alpha)^j$ of equation (2.9). The question is how to state this second kind of forgetting for any reservoir, why it matters for what a linear readout can compute, and how it is tested.

5.1 Definition of fading memory

Two records that agree over the recent past and differ only long ago should lead to nearly the same present state (Figure 12).

Fading memory

Let $\mathbf{r}_k(\mathbf{s})$ be the state at step k after the input history $\mathbf{s} = (\dots, s_{k-1}, s_k)$. The reservoir has a fading memory when histories that agree over the last K steps lead to states whose distance goes to zero as K grows,

$$s_{k-j} = s'_{k-j} \text{ for } 0 \le j < K \quad \Longrightarrow \quad \|\mathbf{r}_k(\mathbf{s}) - \mathbf{r}_k(\mathbf{s}')\| \le \varepsilon_K, \qquad \varepsilon_K \to 0 \;\; (K \to \infty), \tag{5.1}$$

with a bound ε_K that holds for all admissible histories.

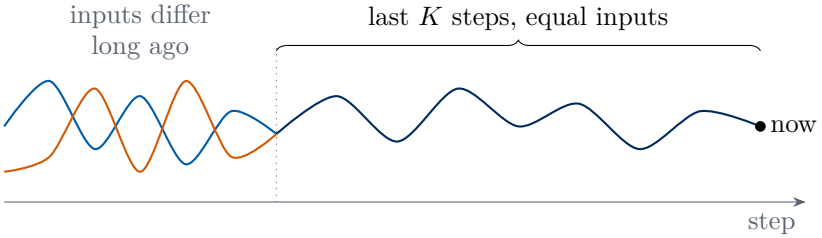


Figure 12: Fading memory. Two input histories (blue and orange) differ long ago and agree over the last K steps (dark line). A reservoir with fading memory ends in nearly the same state after both, and the more nearly the larger K is. The curves are a sketch.

The echo state property compares two starting states under one input, and fading memory compares two input histories that share their recent part. For the leaky unit of equation (2.8) both follow from the same sum (2.9). Let the inputs be bounded, $|g_k| \le g_{\max}$, and let two histories agree for the last K steps. Their states differ only through the inputs of age $j \ge K$, and

$$|x_k(\mathbf{g}) - x_k(\mathbf{g}')| \le \alpha \sum_{j \ge K} (1-\alpha)^j \, |g_{k-j} - g'_{k-j}| \le 2 g_{\max} \, \alpha \sum_{j \ge K} (1-\alpha)^j = 2 g_{\max} \, (1-\alpha)^K. \tag{5.2}$$

The bound falls exponentially with K. For $\alpha = 0.1$ and $K = 50$ the factor $(1-\alpha)^K$ is 0.0052. The same argument holds for any reservoir whose update shrinks every difference of states by at least a factor $c < 1$ per step, whatever the common input. Its states lie in a bounded set, since every component of the hyperbolic tangent lies in $[-1, 1]$, so at the moment from which two histories agree their states are at most $2\sqrt{N}$ apart, and K steps of common input shrink this distance to at most $2\sqrt{N}\, c^K$. Such a reservoir has both properties. A reservoir without a uniform contraction, like the reservoirs of Figure 10, has to be tested for each.

5.2 Fading memory and polynomial readouts

Fading memory matters because of a result of Boyd and Chua [16]. Consider an operator that maps a bounded input history to an output at the present step, does not change its rule in time, and has fading memory. Every such operator can be approximated, uniformly over all bounded inputs and to any required accuracy, by a finite Volterra series,

$$y_k \approx h_0 + \sum_{j=0}^{K-1} h_1(j) \, g_{k-j} + \sum_{j_1, j_2 = 0}^{K-1} h_2(j_1, j_2) \, g_{k-j_1} g_{k-j_2} + \dots, \tag{5.3}$$

a polynomial of finite degree in the inputs of a finite window of K past steps, with coefficients $h_0, h_1, h_2, \dots$ called the kernels. Boyd and Chua stated the result for signals in continuous time, with integrals over the past, and equation (5.3) is its discrete-time analogue, with sums over past steps. An equivalent form is a bank of linear filters of the input followed by a polynomial readout. A reservoir provides the filters through its memory and the polynomial through its nonlinearity, and a linear readout combines them. Fading memory is therefore the natural requirement for a reservoir that is meant to approximate such operators.

The leaky unit with a quadratic readout, $y_k = x_k^2$, shows the structure. Inserting the sum (2.9) for a unit started long ago gives

$$y_k = \alpha^2 \sum_{j_1, j_2 \geq 0} (1-\alpha)^{j_1+j_2}\, g_{k-j_1}\, g_{k-j_2}, \tag{5.4}$$

a Volterra series with the single kernel $h_2(j_1, j_2) = \alpha^2(1-\alpha)^{j_1+j_2}$. Cutting the sums at K changes x_k by at most $g_{\max}(1-\alpha)^K$, the tail of the sum, and since $|x_k| \leq g_{\max}$ it changes $y_k = x_k^2$ by at most $2g_{\max}^2(1-\alpha)^K$. The finite series of equation (5.3) approximates this operator with an error that falls exponentially with the window, and the fading memory of the unit is what makes the window finite.

5.3 Testing fading memory

The test drives two copies from the same state with two random input sequences, uniform in $[-1, 1]$, that differ during the first 50 steps and coincide afterwards, and follows the distance of the two states (Figure 13, left). For the leaky unit with $\alpha = 0.1$ the logarithm of the distance falls at the rate -0.105 per step, the value $\ln(1-\alpha)$ of equation (5.2). An echo state network of 100 units with $\alpha = 1$, $\sigma_{\text{in}} = 0.5$ and $\rho = 0.8$ forgets faster, at the rate -0.333 per step, and its distance reaches 5.8×10^{-15}, the round-off level, after 100 steps. At $\rho = 1.2$ the distance is still 1.6×10^{0} after 10 steps and 2.8×10^{-5} after 200, so with this random input the reservoir forgets old inputs, but slowly.

The same test with the laser record, where 50 random values replace the first 50 values of one of the two inputs, gives a different picture for another reservoir of 100 units with $\sigma_{\text{in}} = 0.5$ (Figure 13, right). At $\rho = 0.8$ the distance reaches 3.7×10^{-16} at step 500. At $\rho = 1.2$ it is still 1.1×10^{0} at step 500, so the 50 differing inputs are not forgotten within the run. With the laser input fading memory is observed at $\rho = 0.8$ and is not observed within the run at $\rho = 1.2$, as in the echo state test. The two tests use different reservoirs, so the difference between them cannot be attributed to the input alone. With the random input the reservoir at $\rho = 1.2$ still forgets, slowly, with a distance of 2.8×10^{-5} after 200 steps. A spectral radius near or above 1 gives a long memory, and its price is a slow or missing loss of the start and of old inputs.

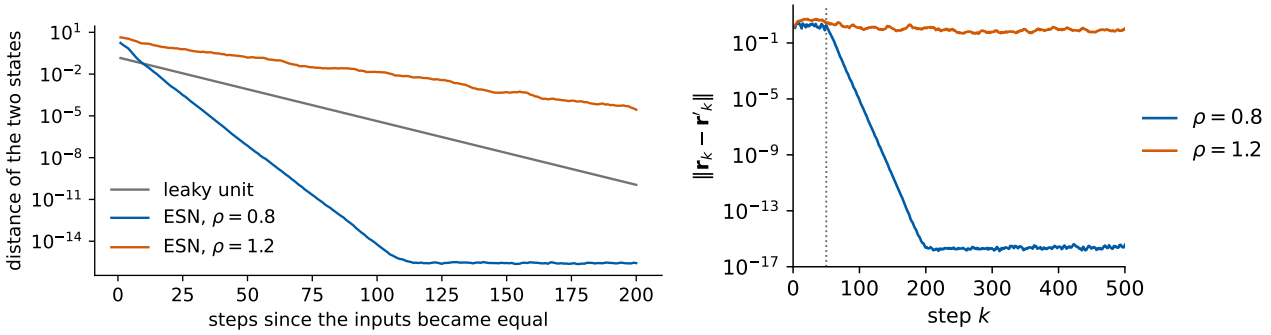


Figure 13: Tests of fading memory. Left, two random inputs uniform in $[-1, 1]$ differ during 50 steps and coincide afterwards. The median distance of the two states over 10 pairs of inputs, on a logarithmic scale, against the number of steps since the inputs became equal, for a leaky unit with $\alpha = 0.1$ and for an echo state network with 100 units at $\rho = 0.8$ and 1.2. Right, the laser record and the same record with its first 50 values replaced by random ones (end of the replacement dotted). The distance $\|\mathbf{r}_k - \mathbf{r}'_k\|$ of the two states of a second reservoir with 100 units against the step k.

6 The memory of a reservoir

The echo state property and fading memory say what a reservoir forgets. A forecast lives on what it keeps, the recent history of the record in a form that a linear readout can use. A measure of this memory is needed that does not depend on the record, since the laser and Lorenz differ in how much of their own past predicts their future, and a bound on it shows what a reservoir of N units can hold at most.

6.1 Memory capacity

The memory of a reservoir is measured with an input that cannot be predicted from its own past, so that every recovered past value must come from the state [17]. The input u_k is drawn independently at every step and uniformly from $[-1, 1]$. For every delay d a separate linear readout of the state is fitted to return the past input u_{k-d}, and its quality on test data is the recall capacity

$$C(d) \coloneqq r(\hat{u}_{k-d}, u_{k-d})^2, \qquad \mathrm{MC} \coloneqq \sum_{d \geq 1} C(d), \tag{6.1}$$

with the Pearson correlation of equation (1.3) over the test steps. The memory capacity MC counts how many past inputs the state holds, where a partly recovered input counts as a fraction. It measures the recall of a random input and differs from C_{tot}, which measures forecasts of a record.

6.2 Three linear units

A linear reservoir of three units, $\mathbf{r}_k = W\mathbf{r}_{k-1} + \mathbf{w}_{\mathrm{in}} u_k$, shows how the state stores the past. With the shift matrix and input vector

$$W = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad \mathbf{w}_{\mathrm{in}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \tag{6.2}$$

the first unit receives the present input and every unit passes its value to the next, so the state is the delay line $\mathbf{r}_k = (u_k, u_{k-1}, u_{k-2})$. A readout recovers u_{k-d} without error for $d = 0, 1, 2$ and not at all for $d \geq 3$, since u_{k-3} is independent of the three stored numbers. For a general matrix the recursion unrolls into

$$\mathbf{r}_k = \sum_{m \geq 0} W^m \mathbf{w}_{\mathrm{in}}\, u_{k-m}, \tag{6.3}$$

so the three numbers mix all delays with weights that decay like the powers of W, and each past input can be recovered only in part. Figure 14 compares the two cases, with one least-squares readout per delay fitted on 2000 steps and tested on 1000 further steps. The shift reservoir gives $C(d) = 1$ for $d \leq 2$ and at most 0.003 beyond, the level that the squared correlation of two independent sequences reaches on 1000 steps by chance. A random 3×3 matrix rescaled to $\rho = 0.8$ spreads its memory over more delays, from 0.752 at $d = 0$ to 0.271 at $d = 3$ and 0.002 at $d = 12$, and recovers less in total, 2.64 for the sum of $C(d)$ over the delays 0 to 12 against 3 for the shift. These sums include the present input, $d = 0$, which the memory capacity defined above leaves out.

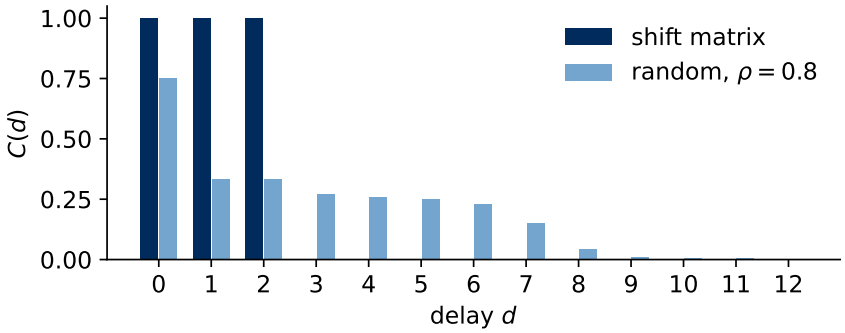


Figure 14: Recall capacity $C(d)$ against the delay d for two linear reservoirs of three units driven by independent uniform inputs, the shift matrix (dark) and a random matrix with $\rho = 0.8$ (light).

6.3 The bound by the number of units

Three units did not store more than three inputs, and this is a general bound [12, 17]. Consider random variables with zero mean as vectors with the inner product $\langle a, b \rangle = \mathbb{E}[ab]$, the covariance. The centred

components of the state span a space $\mathcal{S}$ of dimension at most N. A linear readout produces an element of $\mathcal{S}$. Let $\omega_d = u_{k-d}/\sigma_u$ be the past input divided by its standard deviation σ_u, a vector of unit length. The mean squared error $\|\omega_d - v\|^2$ of a readout $v \in \mathcal{S}$ is smallest when the error $\omega_d - v$ is orthogonal to $\mathcal{S}$, by the theorem of Pythagoras, that is for the orthogonal projection $v = P_{\mathcal{S}}\,\omega_d$. For this readout $\langle P_{\mathcal{S}}\omega_d, \omega_d\rangle = \|P_{\mathcal{S}}\omega_d\|^2$, so its correlation with the target is $\|P_{\mathcal{S}}\omega_d\|^2/(\|P_{\mathcal{S}}\omega_d\|\,\|\omega_d\|) = \|P_{\mathcal{S}}\omega_d\|$, and the recall capacity of the best readout is $C(d) = \|P_{\mathcal{S}}\,\omega_d\|^2$. Inputs at different delays are independent, so the vectors ω_d are orthogonal to each other. Let $\phi_1, \ldots, \phi_{n_{\mathcal{S}}}$, with $n_{\mathcal{S}} \le N$, be an orthonormal basis of $\mathcal{S}$. Then

$$\sum_d C(d) = \sum_d \sum_{m=1}^{n_{\mathcal{S}}} \langle \omega_d, \phi_m\rangle^2 = \sum_{m=1}^{n_{\mathcal{S}}} \sum_d \langle \phi_m, \omega_d\rangle^2 \le \sum_{m=1}^{n_{\mathcal{S}}} \|\phi_m\|^2 = n_{\mathcal{S}} \le N, \tag{6.4}$$

where the inequality is Bessel's inequality, the statement that the squared components of a vector along orthonormal directions add up to at most its squared length. The memory capacity of a reservoir with N units is therefore at most N, also when the delay $d = 0$ is included in the sum. The argument needs uncorrelated inputs. For a correlated input the vectors ω_d overlap, the same information is counted several times, and the sum can exceed N. The bound holds for the best readout on unlimited data, and capacities estimated on a finite test block scatter around their exact values by about the chance level seen above, so a sum over many delays can exceed N slightly.

6.4 Memory of the echo state network

Figure 15 applies the measurement to an echo state network of $N = 50$ units with $\alpha = 1$ and a small input scaling, $\sigma_{\text{in}} = 0.1$, so that the units work close to their linear range. The readouts are fitted on 4000 steps after 200 discarded steps, one ridge strength for all delays is chosen on the next 1000 steps, and $C(d)$ is measured on the last 2000. The memory curves stay close to 1 for short delays and then drop to zero, and the drop moves to longer delays as ρ grows up to $\rho = 0.95$. For one reservoir the memory capacity grows from 12.3 at $\rho = 0.5$ to 19.9 at $\rho = 0.95$, 40% of the bound, and falls to 16.8 at $\rho = 1.1$. Averaged over three random reservoirs, on a grid of ρ in steps of 0.1 that does not contain 0.95, it peaks at 17.0 at $\rho = 0.9$. The single reservoir with 19.9 is the best of the three, and the average is lower because the other two store less. Above $\rho = 1$ the three reservoirs behave differently, from 4.4 to 14.2 at $\rho = 1.5$, so the position and the height of the maximum are statements about these reservoirs. A longer memory needs a slower relaxation, which a larger spectral radius provides until the state stops being a function of the input alone.

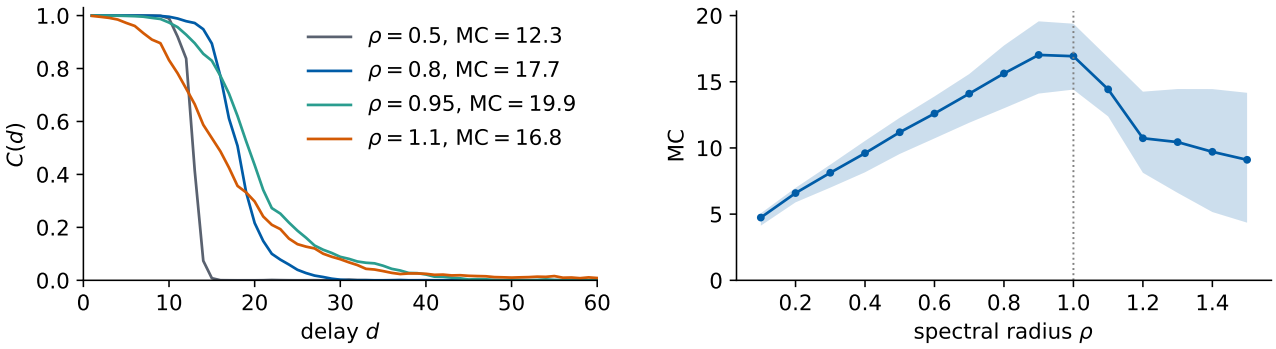


Figure 15: Memory of an echo state network with 50 units, $\alpha = 1$ and $\sigma_{\text{in}} = 0.1$. Left, the recall capacity $C(d)$ against the delay d for four spectral radii, with the memory capacity in the legend. Right, the memory capacity against ρ, mean (line) and range (band) over three random reservoirs, with a dotted line at $\rho = 1$, above which the linearised update at zero input stops contracting.

6.5 Memory and products together

Recalling past inputs is a linear task, while a forecast needs nonlinear functions of past values as well. The nonlinear autoregressive moving-average task of order n (NARMA) asks for both [7]. Its input is

a new random sequence, again written u_k and now uniform in $[0, 0.5]$, and drives the system

$$y_k = 0.3\, y_{k-1} + 0.05\, y_{k-1} \sum_{i=1}^{n} y_{k-i} + 1.5\, u_{k-n} u_{k-1} + 0.1, \tag{6.5}$$

and the model, which sees only the input u, must return y_{k+1} at step k. The target y_{k+1} contains the product $u_{k+1-n}u_k$, which needs a memory of $n-1$ steps and a multiplication, and the feedback through y adds a dependence on older inputs. The error is measured by the normalised mean-square error,

$$\mathrm{NMSE} \coloneqq \frac{\langle (\hat{y}_k - y_k)^2 \rangle}{\mathrm{Var}(y)}, \tag{6.6}$$

with the average over the test steps. It is 0 for a perfect model and 1 for a model that always returns the mean.

A linear readout of the last 20 inputs leaves an NMSE between 0.105 for $n = 2$ and 0.148 for $n = 10$, since the product of two inputs is not a linear function of the inputs. NVAR on the last 12 inputs contains this product among its features and reaches 3.9×10^{-5} for $n = 2$ and 3.1×10^{-4} for $n = 5$, but 0.100 for $n = 10$, where the feedback brings in older inputs and higher powers. An echo state network of 100 units, whose ρ, σ_{in} and β are chosen on validation data, sees only its state and reaches 2.4×10^{-4}, 5.6×10^{-3} and 0.094 for $n = 2$, 5 and 10 (Figure 16). The reservoir supplies memory and products of the inputs at once, and its error grows with the order of the task.

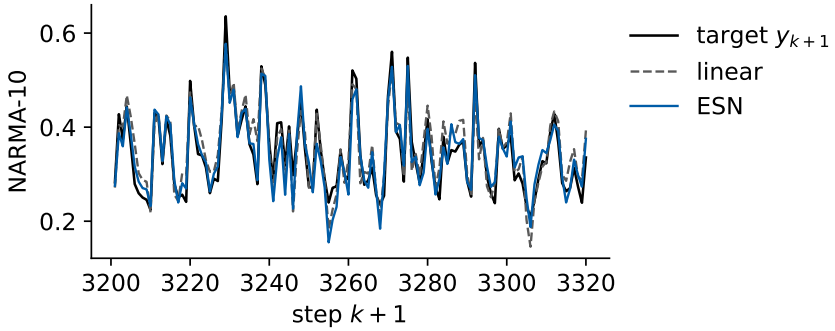


Figure 16: The NARMA task of order 10. The target y_{k+1} of equation (6.5) (black), the forecast of a linear readout of the last 20 inputs (dashed) and of an echo state network with 100 units (blue), against the step, for 120 test steps.

7 Forecasting the five records

Memory, forgetting and nonlinear response have been measured with random inputs and with the NARMA task. The test that matters is the forecast of the five records of Figure 1 scored by the capacity of equation (1.4) on the test block, and a fair account of what one random draw of a reservoir is worth against fixed dictionaries of features and against networks with all weights trained.

7.1 Forecasting protocol

The forecasting reservoir has $N = 300$ units and is driven by the rescaled record from its first value on, so its state at the first training origin $k = 63$ has received the 63 earlier values and s_{63} itself, and the earlier states form the washout. The readout maps the state to all 50 horizons at once. The spectral radius $\rho \in \{0.5, 0.8, 0.95, 1.1\}$, the input scaling $\sigma_{\mathrm{in}} \in \{0.1, 0.5, 1\}$, the leak rate $\alpha \in \{0.3, 1\}$ and the ridge strength $\beta \in \{10^{-8}, 10^{-6}, 10^{-4}, 10^{-2}\}$ are chosen by the largest C_{tot} on the validation block, separately for each record. One random reservoir, drawn once, serves all five records. Each setting costs one run of the reservoir and one linear solve per ridge strength, and the whole search over 24 reservoir settings and four ridge strengths for the five records takes 23 seconds. The AR and

NVAR models, readouts (2.1) of the last p values and of their products, serve as baselines, with their order p and ridge strength chosen on the validation block in the same way.

7.2 Test scores

The first question is whether a fixed random reservoir with a fitted readout can match or beat the fixed dictionaries of features of AR and NVAR. Table 1 and Figure 17 answer it. In the figure, look at how far to the right each curve stays close to 1, since a forecast that keeps a high capacity at long horizons carries information far into the future. On Lorenz the echo state network reaches $C_{\mathrm{tot}} = 39.2$, against 15.4 for AR and NVAR, and its capacity falls below 0.5 only at the horizon 50, while that of NVAR does so at the horizon 16. On MG 25 and MG 35 it is ahead of NVAR by 2.8 and 1.2. On the periodic record MG 15 NVAR and the echo state network reach the maximum and AR comes within 0.2 of it, since a periodic signal is forecast by repeating its period. On the laser the network and NVAR differ by 0.04. The gain is largest where the forecast needs a long memory of a nonlinear signal. On Lorenz the delay window of AR and NVAR covers only a fraction of a loop around one of the two centres, while the reservoir state carries information over the whole history.

test C_{tot}	Lorenz	MG 15	MG 25	MG 35	Santa Fe
persistence	8.4	19.5	13.3	11.8	5.5
AR	15.4	49.8	33.1	38.0	16.5
NVAR	15.4	50.0	45.5	48.6	34.7
ESN, $N = 300$	39.2	50.0	48.3	49.8	34.7
selected ρ	1.1	1.1	0.95	0.8	0.95
selected σ_{in}	1.0	0.5	1.0	1.0	0.1
selected α	1.0	1.0	0.3	0.3	1.0

Table 1: Total capacity on the test block of persistence, AR, NVAR and the echo state network (maximum 50), and the reservoir settings chosen on the validation block.

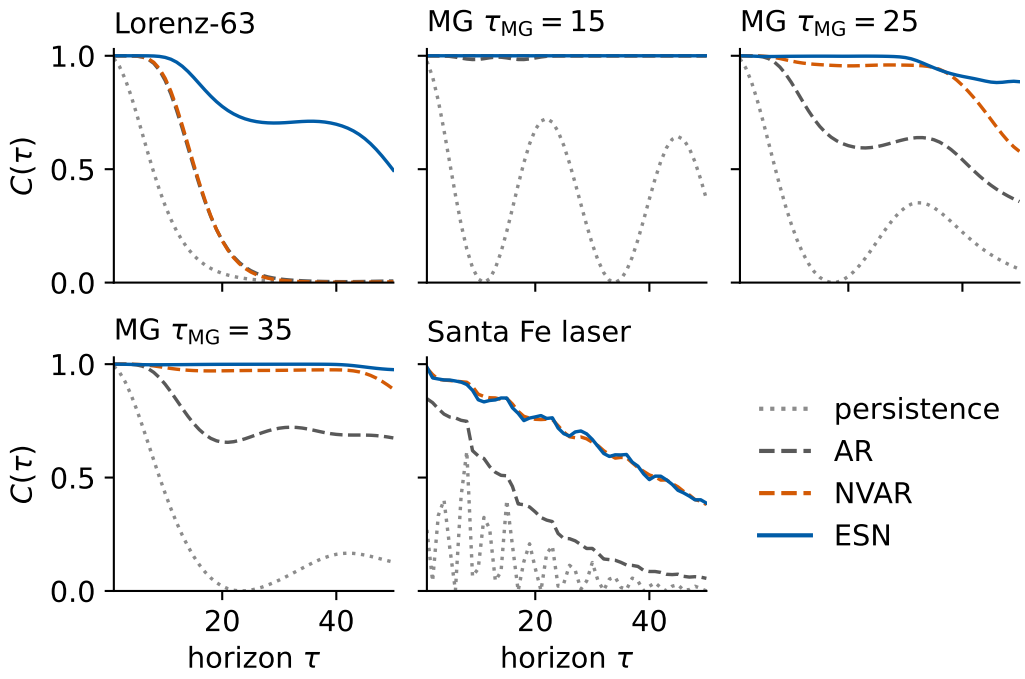


Figure 17: Capacity $C(\tau)$ on the test block against the horizon τ for persistence, AR, NVAR and the echo state network with 300 units, one panel per record.

The selected settings follow the time scales of the records. The slowly varying records MG 25 and MG 35 receive $\alpha = 0.3$, a long memory per unit, and the other three $\alpha = 1$. The laser receives the weak input $\sigma_{\text{in}} = 0.1$, which keeps the units close to their linear range and resolves small changes of the amplitude. Lorenz and MG 15 receive $\rho = 1.1$, above the rule $\rho < 1$. For Lorenz this is the reservoir of the echo state test with $\sigma_{\text{in}} = 1$ described next to Figure 11, whose two copies agree to better than 4×10^{-15} on the test origins, and on MG 15 the copies agree to 4.2×10^{-15}. At the first training origin the copies still differ by up to 0.10, on MG 15, so the first rows of the fit carry a trace of the start. A longer washout of 263 steps changes the test C_{tot} by at most 0.66, on the laser.

One step ahead the echo state network forecasts Lorenz with $C = 1.000$, and the forecasts of the three Mackey–Glass records lie on the diagonal of Figure 18, with C of at least 0.9999. The laser reaches $C = 0.987$, and the visible scatter comes from the largest pulses and from the collapses.

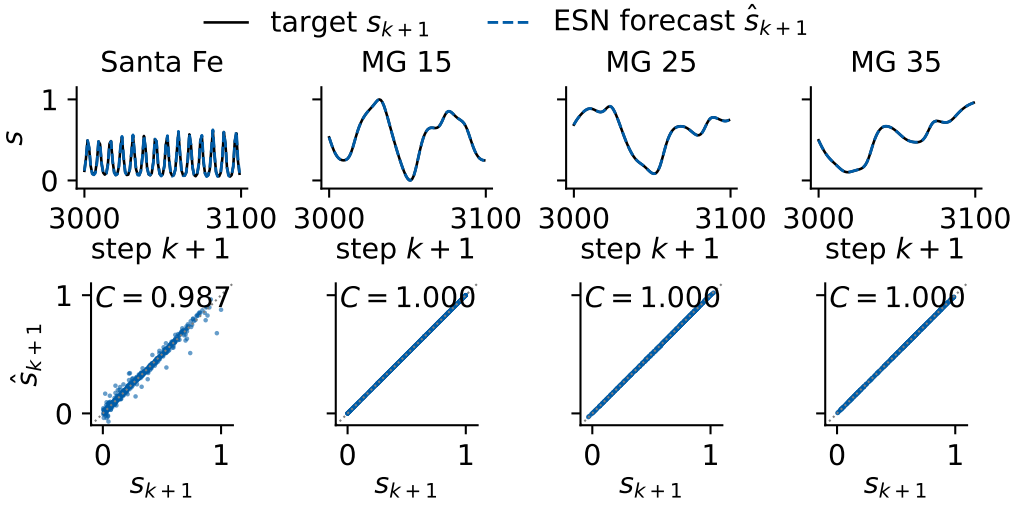


Figure 18: One-step forecasts of the echo state network on the test block. Top, the target s_{k+1} (black) and the forecast $\hat{s}_{k+1}$ (dashed) against the step for the first 100 test targets. Bottom, the forecast against the target for all test targets, with the capacity C at $\tau = 1$.

7.3 Differences between random reservoirs

A reservoir is drawn at random, so a score obtained with one draw could be luck. Five reservoirs drawn independently at the same settings measure how much the score depends on the draw. On Lorenz they spread by 1.0 in C_{tot} and all stay far above NVAR, so the advantage of the reservoir on this record belongs to the method. On the laser they spread by 3.1, from 31.5 to 34.7, and the first draw, the one that tied NVAR, is the best of the five. The tie is therefore a property of one draw, and the five reservoirs of this size lie about two units below NVAR on average on the laser. A difference between two models smaller than the spread between random reservoirs says nothing about the models, and a comparison has to state this spread next to the scores.

7.4 Size of the reservoir

The next question is whether more units help, and where. Figure 19 shows the validation C_{tot} on a grid of N and ρ, with σ_{in} and α fixed at the values selected for each record. Reading a column upwards shows the role of N. On the laser the best value grows from 23.4 with 50 units to 40.1 with 500 and shows no saturation, while on Lorenz it grows only from 37.5 to 43.0. The laser forecast needs many different nonlinear functions of the last few pulsations, and every added unit contributes such functions, while on Lorenz a few hundred units already provide most of what a linear readout can use. Reading a row from left to right shows the echo state property at work. At the weak input $\sigma_{\text{in}} = 0.1$ of the laser the scores of the reservoirs with 100 or more units collapse for $\rho \geq 1.1$, where

the test of Figure 11 found that the state stops forgetting its start. The reservoir of 50 units instead reaches its best laser score, 23.4, at $\rho = 1.25$, a setting that was not tested for the echo state property, while on Lorenz the strong input keeps good scores up to $\rho = 1.1$.

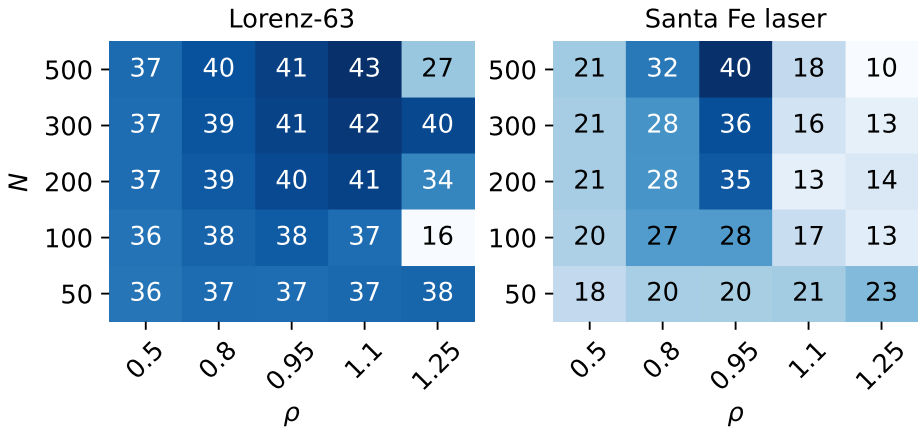


Figure 19: Validation C_{tot} of the echo state network for the numbers of units N and the spectral radii ρ of the grid, for Lorenz (left) and the laser (right), at the input scaling and leak rate selected for each record.

7.5 Trained networks and the cost of training

The echo state network can be set against networks whose weights are all trained by gradient descent on the same records, origins and scores. The multilayer perceptron (MLP) is a feedforward network that reads a window of past values. The recurrent network is the one of equation (1.5). The gated recurrent unit (GRU) and the long short-term memory network (LSTM) are recurrent networks whose units contain gates, factors between 0 and 1 computed from the input and the state that multiply what is stored and what is passed on, which eases the training over long histories. Table 2 lists their scores.

test C_{tot}	Lorenz	MG 15	MG 25	MG 35	Santa Fe	fit time
ESN, $N = 300$	39.2	50.0	48.3	49.8	34.7	92 ms
MLP	29.4	50.0	48.4	49.5	35.3	2 s
RNN	43.8	50.0	49.4	49.5	42.5	12 s
GRU	45.1	50.0	49.8	49.9	43.0	51 s
LSTM	43.8	50.0	49.9	49.9	44.3	52 s

Table 2: Total capacity on the test block of the echo state network and of four networks with all weights trained, with the median fit time of one model at its selected setting, rounded to whole seconds for the trained networks. The times are wall-clock times on one machine shared with other jobs, and they change with hardware, load and implementation.

The trained recurrent networks keep a clear lead on the laser, 44.3 for the LSTM against 34.7, and on Lorenz, 45.1 for the GRU against 39.2. On the Mackey–Glass records the echo state network stays within 1.6 of the best trained network. One step ahead the ranking on the laser reverses, with $C = 0.987$ for the echo state network against 0.959 for the best trained network, so a ranking of models depends on the horizon at which they are compared.

The cost differs by orders of magnitude (Figure 20). One fit of the echo state network, including the run of the reservoir, takes 92 milliseconds, and the trained networks need between 26 and 561 times longer. The number of fitted parameters does not explain this. The readout of the echo state network has 15050 parameters, 301 for each horizon, and the LSTM 6002. The readout is the solution of one linear system, while the weights of the LSTM come out of many passes of gradient descent through the history, the procedure whose difficulties the product of equation (1.7) expresses.

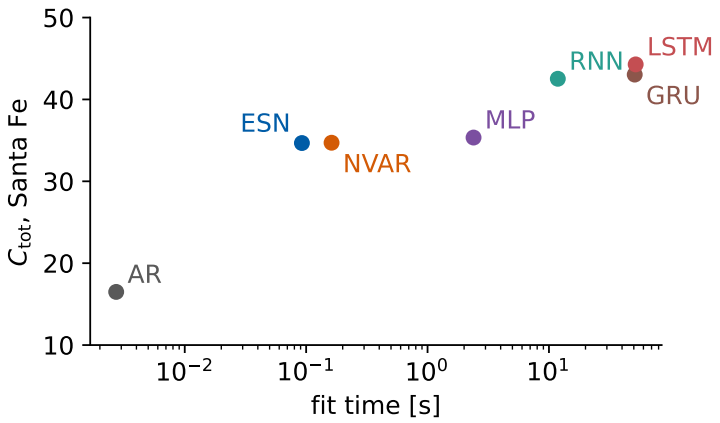


Figure 20: Cost and quality on the laser record. Test C_{tot} against the median fit time, on a logarithmic scale, for AR, NVAR, the echo state network and four trained networks. The times change with hardware and load and compare orders of magnitude only.

8 The reservoir running on its own

Every forecast of Table 1 was computed from the measured past. At each origin the reservoir received the true record up to the present, and the readout returned the 50 horizons. A model of a dynamical system can be asked for more, to generate its own future without any further data and to keep doing so for as long as the true system runs. Two questions then separate, how long such a free run follows the true trajectory, and whether its long-time behaviour resembles the true system once the trajectory is lost. Answering them needs a measure of how fast errors grow in a chaotic system, which the Lorenz model provides in a form that can be computed.

8.1 The full Lorenz state

The test system of this section is the Lorenz-63 model of equation (1.1) with all three coordinates measured. Lorenz derived it from a truncated model of a fluid layer heated from below [2]. The variable x is proportional to the strength of the convective motion, y to the temperature difference between rising and sinking fluid, and z to the deviation of the vertical temperature profile from a straight line. Besides the state of rest the equations have two fixed points, written $C_{\pm}$ as in the figures, a label unrelated to the capacity C,

$$C_{\pm} = \left(\pm\sqrt{\tfrac{8}{3}\cdot 27},\ \pm\sqrt{\tfrac{8}{3}\cdot 27},\ 27 \right) = (\pm 8.485,\ \pm 8.485,\ 27), \tag{8.1}$$

which describe steady convection in either sense of rotation. At the parameter values of equation (1.1) all three are unstable. The trajectory spirals around one of $C_{\pm}$, jumps irregularly to the other and never settles (Figure 21). The divergence of the vector field, $\partial\dot{x}/\partial x + \partial\dot{y}/\partial y + \partial\dot{z}/\partial z = -10 - 1 - \frac{8}{3} = -\frac{41}{3}$, is the rate at which a small volume of states shrinks, the same everywhere. The motion therefore settles on a set of zero volume, the strange attractor.

The equations are integrated with the fourth-order Runge–Kutta method and sampled every $\Delta t = 0.02$ time units. The three coordinates are normalised by their means and standard deviations over the training block, and $\mathbf{u}_k \in \mathbb{R}^3$ denotes the normalised state at step k, a vector unrelated to the scalar random inputs u_k of the memory and NARMA tasks. The training block has 10000 samples, the test block 20000, and a long block of 110000 samples serves for the statistics.

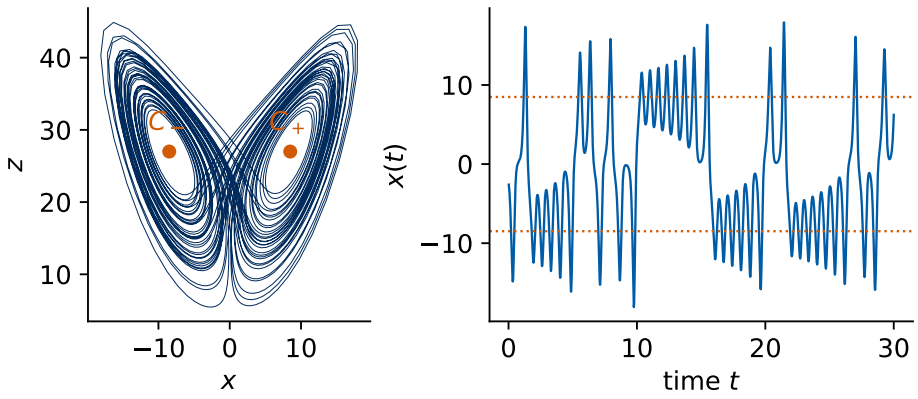


Figure 21: The Lorenz-63 attractor. Left, projection of 50 time units of a trajectory on the (x, z) plane, with the fixed points $C_\pm$ (orange). Right, the coordinate x against the time t over 30 time units, with dotted lines at $x = \pm 8.49$.

8.2 Lyapunov exponents

Chaos is measured by the rate at which nearby states separate. Two trajectories that start a small distance δ_0 apart separate on average as

$$\delta(t) \approx \delta_0 \, e^{\lambda_1 t}, \tag{8.2}$$

where λ_1 is the largest Lyapunov exponent, the average exponential rate of separation. A system with $\lambda_1 > 0$ is chaotic. A three-dimensional system has three exponents $\lambda_1 \geq \lambda_2 \geq \lambda_3$. The first gives the growth of a small line element, the sum of the first two the growth of a small area element and the sum of all three the growth of a small volume. The sum of all three is therefore the average divergence of the vector field, $-41/3$ for Lorenz-63.

Benettin and coworkers computed all exponents by following a set of small perturbations, the tangent vectors, along a trajectory [21]. Let φ be the map that advances the state by one sampling interval, $\mathbf{u}_{k+1} = \varphi(\mathbf{u}_k)$, and $D\varphi(\mathbf{u}_k)$ its Jacobian matrix, which advances small perturbations. A matrix O_k with three orthonormal columns holds the tangent vectors. At every step they are advanced and orthonormalised again by a QR decomposition, a product of a matrix with orthonormal columns and an upper triangular matrix U,

$$D\varphi(\mathbf{u}_k)\, O_k = O_{k+1} U_{k+1}, \qquad \lambda_i = \frac{1}{n_\lambda \Delta t} \sum_{k=1}^{n_\lambda} \ln \left| (U_k)_{ii} \right|. \tag{8.3}$$

Here n_λ is the number of steps of the run. The first column follows the most expanding direction, and $(U_k)_{11}$ is its growth factor over one step. Without the orthonormalisation all columns would turn into this direction. The QR step keeps the second column perpendicular to the first, so $(U_k)_{11}(U_k)_{22}$ is the growth factor of the area that the first two span, and so on. Over 3980 time units the method gives

$$\lambda_1 = 0.904 \pm 0.002, \qquad \lambda_2 = -0.0001, \qquad \lambda_3 = -14.57, \tag{8.4}$$

with the error estimated from ten blocks of the run (Figure 22). Their sum, -13.667, agrees with $-41/3 = -13.667$ to the fourth digit, which checks the computation. The second exponent belongs to the direction along the trajectory, in which a perturbation is a shift in time and neither grows nor shrinks on average, and it vanishes for every bounded trajectory that does not end on a fixed point.

The natural unit of time for forecasts is the Lyapunov time

$$T_\lambda \coloneqq \frac{1}{\lambda_1} = 1.11 \text{ time units}, \tag{8.5}$$

or 55 samples, over which a small error grows on average by the factor e. The training block covers 181 Lyapunov times.

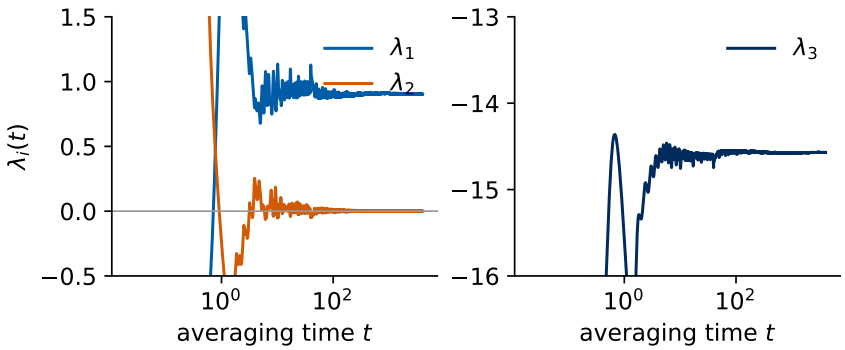


Figure 22: Running averages $\lambda_i(t)$ of the three Lyapunov exponents of Lorenz-63 against the averaging time t on a logarithmic scale. Left, λ_1 and λ_2. Right, λ_3.

8.3 Open and closed loop

The reservoir is driven by the three-component state, so its input weights form an $N \times 3$ matrix W_{in} with entries uniform in $[-\sigma_{\text{in}}, \sigma_{\text{in}}]$, and every unit receives a fixed random bias, the vector $\boldsymbol{\theta}$ with entries uniform in $[-0.1, 0.1]$. In the open loop the measured state drives the reservoir and the readout predicts the next state,

$$\mathbf{r}_k = (1-\alpha)\,\mathbf{r}_{k-1} + \alpha \tanh\left(W\mathbf{r}_{k-1} + W_{\text{in}}\mathbf{u}_k + \boldsymbol{\theta}\right), \qquad \hat{\mathbf{u}}_{k+1} = W_{\text{out}}^{\top}\mathbf{r}_k + \mathbf{b}. \tag{8.6}$$

The readout is fitted by ridge regression on the one-step targets $\mathbf{u}_{k+1}$ of the training block, after a washout of 200 steps. In the closed loop the prediction replaces the measurement,

$$\mathbf{r}_{k+1} = (1-\alpha)\,\mathbf{r}_k + \alpha \tanh\left(W\mathbf{r}_k + W_{\text{in}}(W_{\text{out}}^{\top}\mathbf{r}_k + \mathbf{b}) + \boldsymbol{\theta}\right) =: \mathcal{M}(\mathbf{r}_k). \tag{8.7}$$

The map $\mathcal{M}$ receives no data. It is an autonomous dynamical system on $\mathbb{R}^N$, and a free run is its iteration (Figure 23). Pathak and coworkers showed that such a closed-loop reservoir can reproduce a chaotic attractor and its Lyapunov exponents [19], following the forecasts of chaotic systems with echo state networks by Jaeger and Haas [18], and Lu, Hunt and Ott studied when the reconstruction succeeds [20].

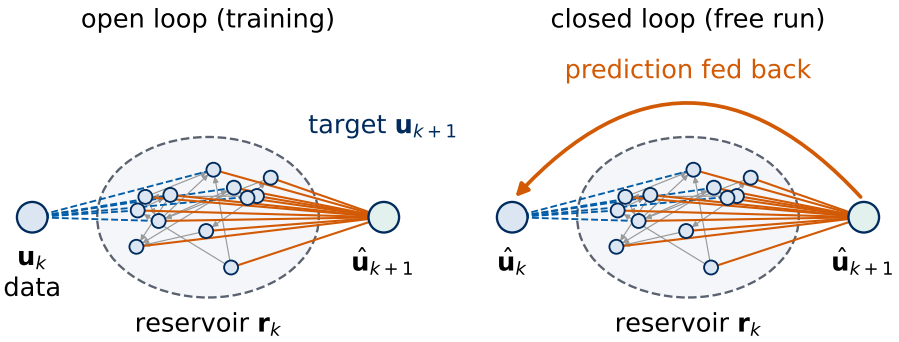


Figure 23: Open and closed loop. Left, during training the measured state $\mathbf{u}_k$ drives the reservoir through the fixed input weights (dashed blue) and the readout (orange) predicts $\hat{\mathbf{u}}_{k+1}$. Right, in the closed loop the prediction is fed back as the next input.

A closed-loop model is judged on two time scales. Its weather is the short-term forecast of one trajectory, the time over which the free run started from a given state follows the truth. Its climate is the long-time statistics of the free run, whether it moves on the same attractor with the same distribution of states as the true system. The two words are borrowed from meteorology.

The reservoir studied here has $N = 300$ units, a matrix W with 5% nonzero entries at $\rho = 0.9$, $\sigma_{\text{in}} = 0.1$, $\alpha = 1$ and $\beta = 10^{-8}$. Its one-step NMSE, the squared error divided by the variance and averaged over the three coordinates, is 5.7×10^{-11} on the test block. One step ahead the fit is close to perfect, and the free run shows what this fit is worth when the model has to generate its own inputs.

8.4 Weather and the valid time

The weather is measured from 190 origins in the test block, 100 samples apart. At each origin the reservoir has been driven by the true record, so the forecast starts from a state synchronised with the data, and then the loop is closed. The error after the horizon τ is the normalised distance

$$e(\tau) := \frac{\|\hat{\mathbf{u}}(\tau) - \mathbf{u}(\tau)\|}{\sqrt{\langle \|\mathbf{u} - \langle \mathbf{u} \rangle\|^2 \rangle}}, \tag{8.8}$$

with the averages over the training block. For two independent points $\mathbf{u}$ and $\mathbf{u}'$ of the attractor the mean of $\|\mathbf{u} - \mathbf{u}'\|^2$ is twice the mean of $\|\mathbf{u} - \langle \mathbf{u} \rangle\|^2$, since the cross term averages to zero, so $e \approx \sqrt{2}$ for a forecast that has lost all information. The valid time t_v is the first horizon at which $e(\tau)$ exceeds 0.4.

Figure 24 shows the forecast from the origin with the median valid time. It follows all three coordinates through several turns around both fixed points and then departs from the truth, after which it continues as a plausible Lorenz trajectory out of phase with the true one. Over the 190 origins the median valid time is $6.9\,T_\lambda$, and 80% of the origins lie between 4.9 and 8.7 Lyapunov times.

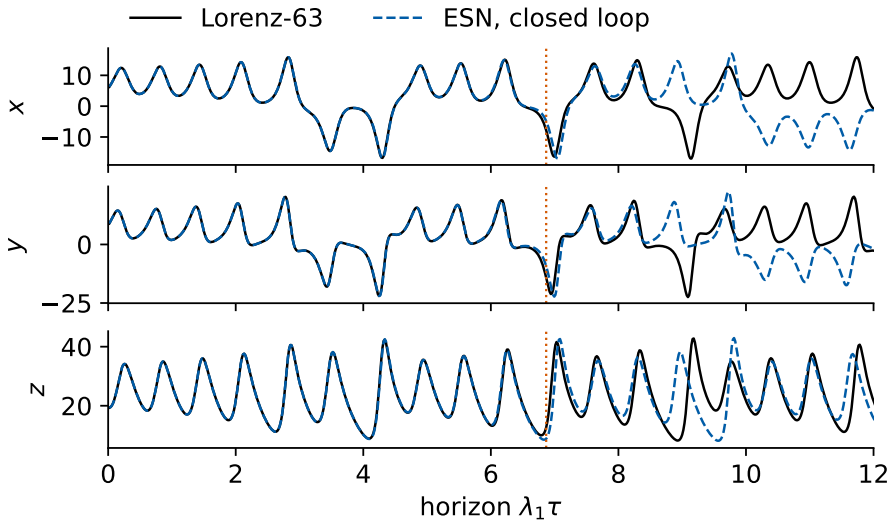


Figure 24: A closed-loop forecast of Lorenz-63 from the origin with the median valid time. The three coordinates of the truth (black) and of the free run (dashed blue) against the horizon in Lyapunov times, $\lambda_1 \tau$. The dotted orange line marks the valid time.

8.5 The length of a forecast

The valid time can be estimated from equation (8.2). If the error starts at a size δ and grows as $\delta\, e^{\lambda_1 t}$, it reaches the threshold when $\delta\, e^{\lambda_1 t} = 0.4$, that is after

$$t \approx \frac{1}{\lambda_1} \ln \frac{0.4}{\delta}. \tag{8.9}$$

With the error after the first step, $\delta_0 = 1.9 \times 10^{-6}$, the formula gives $12.3\,T_\lambda$, almost twice the measured median. The left panel of Figure 25 shows why. During the first fraction of a Lyapunov time the mean error rises much faster than the slope λ_1 allows. Afterwards the logarithm of the error grows at the rate $0.93\,\lambda_1$, fitted between $\lambda_1 \tau = 1.7$ and 6.6, close to the rate of the Lorenz flow. Extrapolated back to $\tau = 0$, this exponential stage starts from an effective initial error $\delta_{\text{eff}} = 2.1 \times 10^{-4}$, and with it equation (8.9) gives $\ln(0.4/\delta_{\text{eff}}) = 7.5$ Lyapunov times, close to the measured 6.9. A plausible reading of the fast first stage, which was not tested separately, is that the state of the reservoir relaxes away from the state it had while the data drove it, a growth of the model error that the Lorenz flow itself does not have. Once this initial error has formed, the forecast is lost because the system is chaotic.

The logarithm in equation (8.9) has a consequence for every forecast of a chaotic system. A model whose effective initial error were a hundred times smaller would gain $\ln 100 \approx 4.6$ Lyapunov times. For a chaotic system the length of a forecast grows only logarithmically with the precision of the model and of the initial state.

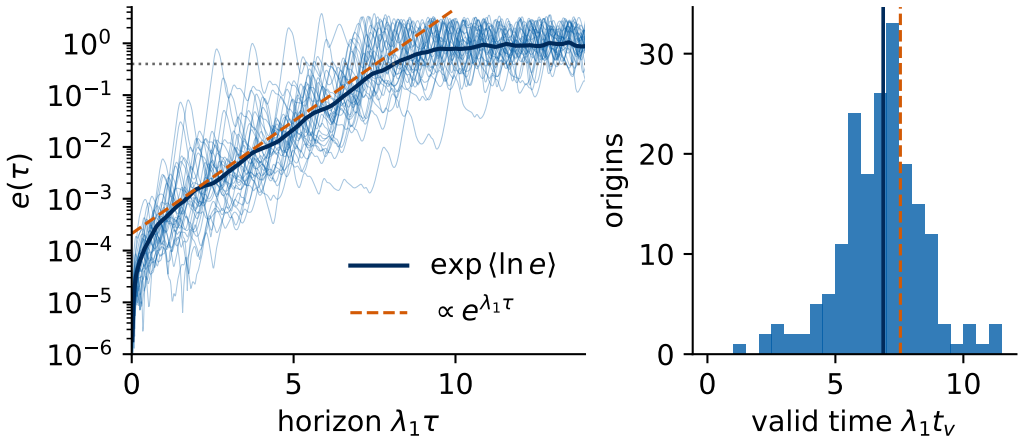


Figure 25: Weather of the closed-loop reservoir. Left, the error $e(\tau)$ of equation (8.8) against the horizon in Lyapunov times on a logarithmic scale, thin lines for every sixth origin, the dark line for the geometric mean over all origins, dashed a line of slope λ_1. Right, histogram of the valid times t_v of the 190 origins, with the median (dark line) and the estimate of equation (8.9) with the effective initial error (dashed).

8.6 Climate

The climate is tested on a free run of about 1988 Lyapunov times, started from the state of the reservoir at the end of the test block. The projection of the free run on the (x, z) plane shows the two wings of the true attractor with the same outer boundary and the same holes around the fixed points (Figure 27, left).

The distribution of each coordinate is compared with that of a long true run by the Wasserstein distance. For two distributions p and q of one variable with the cumulative distribution functions P_p and P_q,

$$W_1(p, q) \coloneqq \int_{-\infty}^{\infty} |P_p(\chi) - P_q(\chi)| \, d\chi. \tag{8.10}$$

For a distribution shifted by χ_0 against another, $P_p(\chi) = P_q(\chi - \chi_0)$, the integral equals $|\chi_0|$, so W_1 measures how far probability has to be moved to turn one distribution into the other, in the units of the variable. In the normalised coordinates it is measured in standard deviations. The free run gives $W_1 = 0.016$, 0.015 and 0.015 for x, y and z (Figure 26). Two independent true runs of the same length differ by 0.015, 0.013 and 0.0027. For x and y the free run is as close to the truth as a second true run, and for z the difference is about 5.6 times larger and still 0.015 standard deviations.

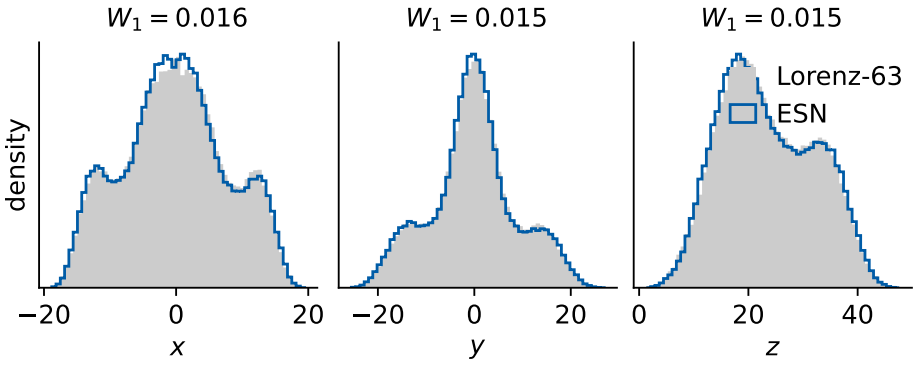


Figure 26: Invariant densities of the three Lorenz coordinates from a long true run (grey) and from the free run of the reservoir (blue line), with the Wasserstein distance (8.10) in standard deviations above each panel.

8.7 Lyapunov exponents of the reservoir

The closed loop (8.7) is a map on $\mathbb{R}^N$, and its Lyapunov exponents follow from equation (8.3) with the Jacobian of $\mathcal{M}$ in place of $D\varphi$. With $\mathbf{a}(\mathbf{r}) = W\mathbf{r} + W_{\text{in}}(W_{\text{out}}^\top \mathbf{r} + \mathbf{b}) + \boldsymbol{\theta}$ the chain rule gives $\partial \mathbf{a}/\partial \mathbf{r} = W + W_{\text{in}} W_{\text{out}}^\top$, and the derivative of the hyperbolic tangent contributes the diagonal matrix of $1 - \tanh^2$,

$$D\mathcal{M}(\mathbf{r}) = (1-\alpha)\,\mathbb{1} + \alpha\ \text{diag}\left(1 - \tanh^2 \mathbf{a}(\mathbf{r})\right)\left(W + W_{\text{in}} W_{\text{out}}^\top\right). \tag{8.11}$$

The term $W_{\text{in}} W_{\text{out}}^\top$ is the feedback through the readout, and it is the only place where the trained weights enter. Dividing the logarithms by Δt turns rates per step into rates per time unit. The reservoir has N exponents, and the first four, computed over 2000 time units, are compared with those of the flow in Figure 27, right. The largest is 0.909 ± 0.004, within 1% of $\lambda_1 = 0.904$, and the second, -0.0002 ± 0.0013, is zero within its error, as it must be if the map imitates a sampled flow. The reservoir has learned the expansion rate of the attractor from one-step data alone. Its third exponent, -2.40, is far from $\lambda_3 = -14.57$, and the fourth, -7.04, has no counterpart in a three-dimensional flow. A plausible reason is that the training data lie on the attractor and carry no information on how fast states off the attractor are pulled back to it, so the reservoir supplies its own contraction rates in these directions.

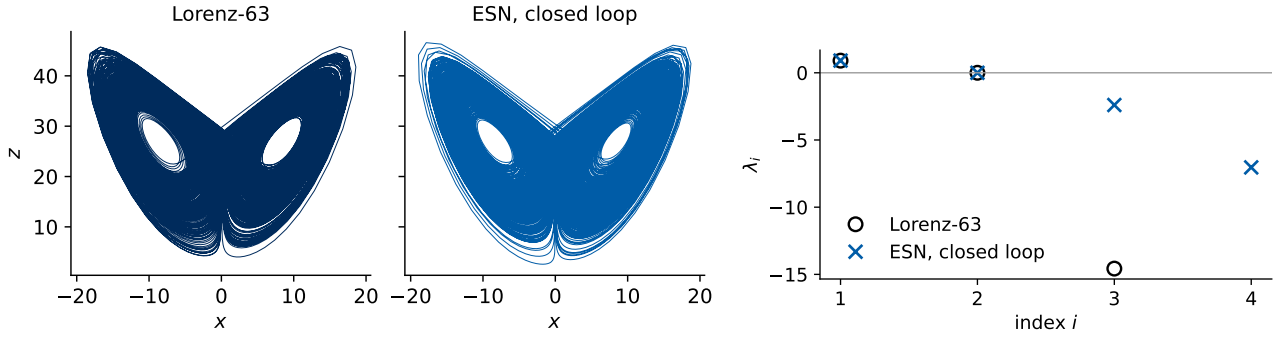


Figure 27: Climate of the closed-loop reservoir. Left, the (x, z) projection of a long true run and of the free run. Right, the first Lyapunov exponents of the reservoir map, from equations (8.3) and (8.11), against those of the Lorenz flow.

8.8 Robustness over random reservoirs

The weather and climate of Figures 25 and 27 belong to one random draw at one setting, and the question is whether they are a property of the method or of a lucky reservoir. We train 240 reservoirs, ten random draws for each of 24 settings with $N \in \{100, 300\}$, $\rho \in \{0.5, 0.9, 1.2\}$, $\sigma_{\text{in}} \in \{0.1, 0.5\}$ and $\beta \in \{10^{-8}, 10^{-4}\}$, run each one freely for 50000 steps and sort the free runs into classes. A run has diverged when a normalised coordinate exceeds 10 in modulus, and it sits at a fixed point when every coordinate varies by less than 10^{-3} over the last 10000 steps. It is periodic when the maxima of z

over these steps take few distinct values, fewer than 20% of their number after rounding to 0.01. The remaining, aperiodic runs are Lorenz-like when the largest of the three Wasserstein distances to the true densities is below 0.1, a threshold about three times the largest distance, 0.031, between two true runs of this length, and aperiodic with a wrong climate otherwise.

Fewer than half of the runs reproduce the climate, 96 of 240, and the others end in every other class, 62 periodic, 44 aperiodic with a wrong climate, 22 at a fixed point and 16 diverging. In Figure 28, look first at the columns with $\rho = 1.2$, which are nearly empty, only one Lorenz-like run of 80. This is a spectral radius above 1, at which the open-loop echo state tests of the earlier sections lost the property, and a plausible reading, not tested here, is that its own dynamics then competes with the dynamics that the readout tries to impose. Even the setting studied above succeeds for only 7 of its ten draws, so the success of Figure 27 is typical of this setting without being guaranteed. The runs that pass the density test also have the right largest exponent, with the median 0.904, so the density criterion and the Lyapunov exponent agree on what a good free run is. The criterion based on maxima is coarse, and every count carries an uncertainty of a few runs.

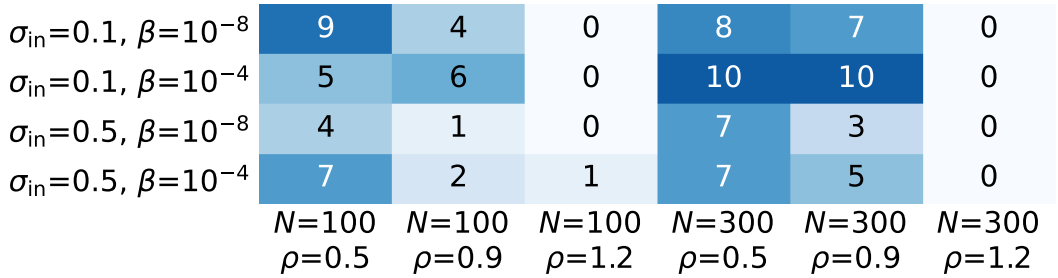


Figure 28: Robustness of the climate. Number of the ten random reservoirs whose free run is Lorenz-like, for each setting of the number of units N, the spectral radius ρ, the input scaling σ_{in} and the ridge strength β.

The setting of the reservoir studied above, $\rho = 0.9$, was fixed by a short trial before the analysis. A second search over 18 settings on the training block alone, with the readout fitted on the first 8000 training samples and closed-loop forecasts started in the last 2000, does not rank it first by valid time. The longest median valid time on these validation data, 7.65 Lyapunov times, belongs to $\rho = 0.5$ at the same σ_{in} and β, against 6.63 for $\rho = 0.9$, and both give a Lorenz-like free run. The setting $\rho = 0.9$ was kept, since the weather and climate results above refer to it. Over ten random draws the two settings give 8 and 7 Lorenz-like runs and the median valid times 7.63 and 6.46 Lyapunov times, so on these data $\rho = 0.5$ gives the longer forecasts.

8.9 One-step error and climate

The open-loop fit produces one number for free, the one-step error, while the climate test needs a long free run. The question is whether the cheap number can tell the good reservoirs from the bad ones. In Figure 29, look at whether the colours separate along the horizontal axis. In the right panel they do in part. A long forecast needs a small one-step error, and the median valid time falls from 6.6 Lyapunov times for the runs with a one-step NMSE below 10^{-10} to 0.4 for those above 10^{-5}. In the left panel they do not. Periodic orbits, fixed points and wrong attractors appear over the whole range of one-step errors, and the smallest one-step error of the whole scan belongs to a periodic run. In this scan a small one-step error accompanied every long valid time, and it did not separate good climates from bad ones.

Figure 30 shows two failures with small one-step errors. One reservoir first moves chaotically and then comes to rest at $(x, y, z) = (-8.62, -8.65, 26.83)$, at a distance 0.27 from the fixed point C_- of the Lorenz equations. The reservoir has created a stable fixed point close to the unstable fixed point C_- of the flow. Another, with a one-step NMSE of 2.9×10^{-12}, leaves the attractor within a few Lyapunov times and settles on a large periodic orbit that reaches $z = 56$, outside the true attractor. The readout is fitted only on states that the true record produces, while the free run also depends on what $\mathcal{M}$ does near the attractor, where the training data say nothing. A closed-loop model has to be

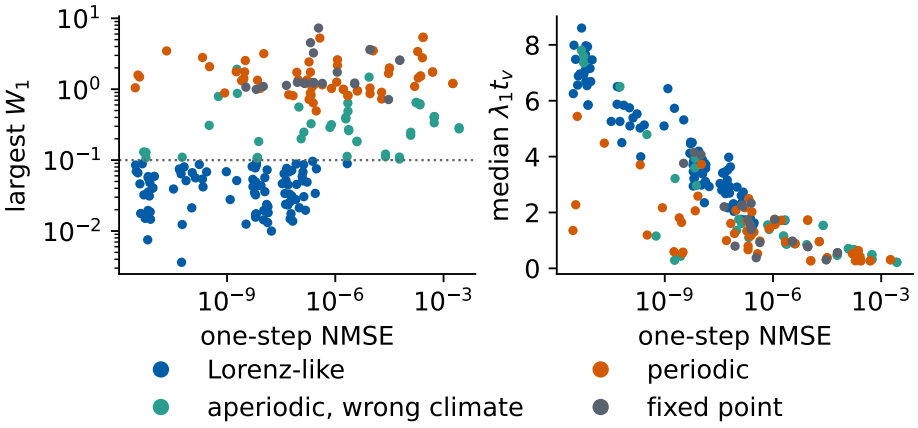


Figure 29: One-step error and free run over the scan of 240 reservoirs, without the diverged runs. Left, the largest Wasserstein distance of the three coordinates against the one-step NMSE, with the threshold 0.1 of the Lorenz-like class dotted. Right, the median valid time against the one-step NMSE. Colours mark the class of the free run.

judged by its free run.

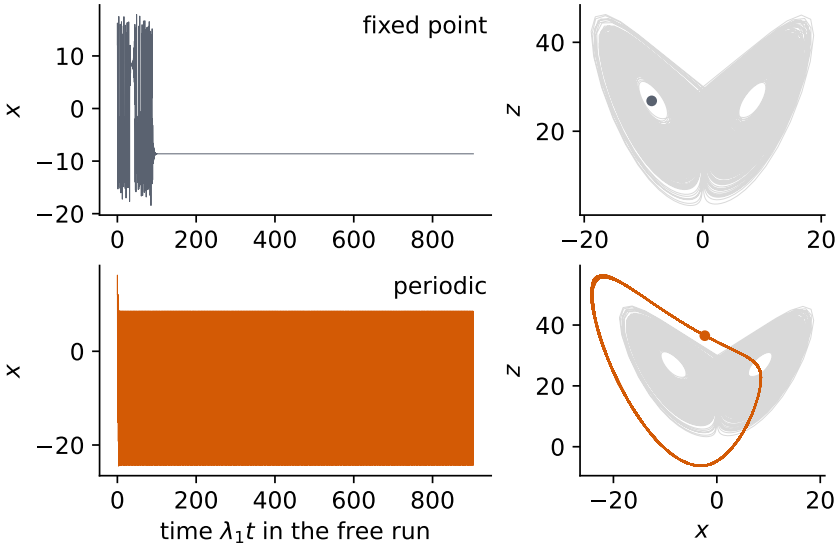


Figure 30: Two failed free runs. Left, the coordinate x over the whole free run against the time in Lyapunov times. Right, the last 5000 steps in the (x, z) plane on top of the true attractor (grey), with the last point marked. Top, a run that ends on a fixed point. Bottom, a run that ends on a periodic orbit.

Two further observations limit what the reservoir is doing. A linear model closed in a loop, an AR model that predicts the next state from the last two, relaxes to a fixed point for all three ridge strengths tried. A closed-loop linear model is a linear map, whose largest eigenvalue modulus is here 0.9912, and a linear map can only decay, grow or oscillate, with no bounded chaotic motion at all. A nonlinear model is therefore necessary for the climate, though not necessarily a reservoir. NVAR on the last two states, closed in the same way, gives a Lorenz-like free run at the ridge strengths 10^{-6} and 10^{-4} and diverges at 10^{-8}, since its quadratic features contain the quadratic terms of the Lorenz equations. The bias $\boldsymbol{\theta}$ matters as well. Without it the ten draws of the base setting give 0 Lorenz-like runs, and 8 of them diverge. A hypothesis that was not tested here is a symmetry argument. Without the bias the open-loop update (8.6) is unchanged under the joint sign change $(\mathbf{r}, \mathbf{u}) \to (-\mathbf{r}, -\mathbf{u})$, while the Lorenz equations change under $\mathbf{u} \to -\mathbf{u}$, since $z \to -z$ changes them, and the bias removes the symmetry from the reservoir.

9 Requirements and physical reservoirs

The echo state network is one realisation of a general recipe, and the echo state test, the fading-memory test, the memory capacity and the forecasts can be read as requirements on any driven system that is to serve as a reservoir. The state has to forget its start, the echo state property, or a readout fitted on one run fails on another. It has to forget old inputs, fading memory, so that the readout responds to the recent history that matters for the future. Different recent histories have to lead to different states, since a readout cannot tell apart histories that produce the same state. The response has to be nonlinear, so that products of inputs at different delays appear among the components of the state, as the NARMA task demanded. The state has to offer many quantities that are not linear combinations of each other, since the readout can only combine what the state provides, and the gain of the laser forecasts from 50 to 500 units showed the value of many such quantities. Finally the memory time of the system has to match the time scales of the task, the role that the leak rate played on the Mackey–Glass records.

Nothing in these requirements asks for a neural network. Grigoryeva and Ortega proved that echo state networks with linear readouts are universal approximants of fading-memory operators [23], and the same recipe works with physical systems in place of the random network [22]. Fernando and Sojakka drove the surface of water in a tank with motors that encoded the input and used camera images of the waves as features [24]. Hauser and coworkers studied networks of masses and nonlinear springs driven by the input, whose positions are read out [25]. Appeltant and coworkers used a single nonlinear electronic node with a delayed feedback loop, in which the samples along the delay line act as virtual units [26]. Du and coworkers used memristors, resistors whose resistance depends on the current that has passed through them, so that their state remembers the recent input [27]. In each case the physical system is fixed and only a linear readout of measured quantities is fitted.

10 A spiking reservoir

The requirements for a reservoir say nothing about the form of the units, and a network whose units communicate by pulses can be tested as a reservoir in the same way, for forgetting its start, holding a record of its input and forecasting the five records.

10.1 Neurons and spikes

The units of an echo state network send smooth signals, values of a hyperbolic tangent. Nerve cells communicate differently. They send short voltage pulses, the spikes or action potentials, and the information lies in when and how often they are sent. Networks of model neurons that emit spikes are called spiking neural networks. In neuroscience they are models of how circuits of real neurons process information. In engineering they run on neuromorphic chips, on which a neuron uses energy only when it fires [28], and they process the output of event-based sensors, such as event cameras, which report changes of brightness as streams of spikes instead of frames [29].

A spiking network driven by a signal is also a reservoir. Maass, Natschläger and Markram proposed this at the same time as Jaeger proposed the echo state network and called it a liquid state machine (LSM) [14], with the picture of a liquid whose surface keeps the ripples of everything thrown into it for a while. The picture is an analogy, and the model is a set of coupled differential equations with a threshold.

10.2 One leaky equation

A single equation lies behind the leaky units of the echo state network and the membrane of a nerve cell,

$$\tau_r \frac{dx}{dt} = -x + g(t), \tag{10.1}$$

in which a state x relaxes towards a target $g(t)$ with the relaxation time τ_r. Without a target the state decays as e^{-t/τ_r}, and with a target it follows it with a delay of about τ_r, keeping a fading trace of earlier targets.

A bucket of water gives an intuition, and it is an analogy. Water flows in at a rate set by the input and drains through a long thin pipe at the bottom. The slow viscous flow through such a pipe is proportional to the pressure at its entrance and so to the water level, while through a plain hole the outflow would grow as the square root of the level and the equation would no longer be linear. The level is the state, the inflow sets the target, and the level remembers recent inflow and forgets old inflow. A wide bucket with a narrow pipe has a long relaxation time and a long memory.

One Euler step of equation (10.1) with the time step Δt, $x(t+\Delta t) \approx x(t) + \frac{\Delta t}{\tau_r}\big(-x(t)+g(t)\big)$, reads

$$x_k = (1-\alpha)\, x_{k-1} + \alpha\, g_k, \qquad \alpha = \frac{\Delta t}{\tau_r}, \tag{10.2}$$

which is the leaky unit of equation (2.8). The leak rate is the ratio of the time step to the relaxation time, and $\alpha = 1$ means that a unit forgets its previous state within one step. With the target $g_{i,k} = \tanh\big(\sum_j W_{ij} r_{j,k-1} + w_{\mathrm{in},i} s_k\big)$ the equation becomes the echo state network of equation (3.1), which is a set of leaky units wired together at random. A spiking neuron keeps the leaky equation and changes only the nonlinearity. Instead of a smooth limit on its inflow, it empties at once when its level reaches a mark, the threshold, and records the event as a spike.

10.3 The membrane as a resistor–capacitor circuit

The membrane of a nerve cell is a thin lipid layer between two conducting salt solutions. It stores charge like a capacitor of capacitance C_m, and ion channels let a small current leak through it like a resistor R. Ion pumps keep the inside at a resting voltage V_{rest} when no current is injected, which in the circuit is a battery in series with the resistor (Figure 31). Lapicque modelled the nerve membrane by this resistor–capacitor (RC) circuit in 1907 [30]. A current $I(t)$ injected into the cell splits, by Kirchhoff's law of currents, into the charging current of the capacitor and the leak current through the resistor,

$$I(t) = C_m \frac{dV}{dt} + \frac{V - V_{\mathrm{rest}}}{R}, \tag{10.3}$$

where V is the membrane voltage. Multiplying by R and defining the membrane time constant $\tau_m \coloneqq RC_m$ gives

$$\tau_m \frac{dV}{dt} = -(V - V_{\mathrm{rest}}) + R\, I(t), \tag{10.4}$$

equation (10.1) with the state $V - V_{\mathrm{rest}}$, the target $RI(t)$ and the relaxation time τ_m. In the bucket analogy the capacitor is the bucket, the resistor the pipe and the current the inflow. The illustrative values $\tau_m = 20$ ms and $R = 100$ MΩ give $C_m = \tau_m/R = 0.2$ nF, and with currents in nA the product RI is in mV.

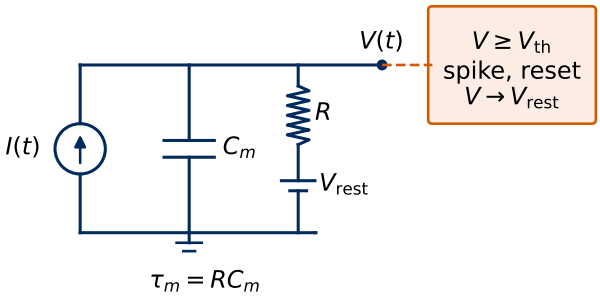


Figure 31: The membrane of a neuron as an RC circuit. The input current $I(t)$ charges the capacitor C_m or leaks through the resistor R in series with the resting voltage V_{rest}. The box on the right stands for the threshold and reset rule of the integrate-and-fire neuron.

10.4 The membrane remembers and forgets

With the voltage above rest $v := V - V_{\text{rest}}$, equation (10.4) reads $\tau_m \dot{v} = -v + RI(t)$. Without input the voltage relaxes as $v(t) = v(0)\, e^{-t/\tau_m}$, and after one time constant 37% of it is left. For a general input we multiply the equation by e^{t/τ_m}. The left-hand side becomes a total derivative, $\tau_m \frac{d}{dt}(v\, e^{t/\tau_m}) = \tau_m \dot{v}\, e^{t/\tau_m} + v\, e^{t/\tau_m} = RI(t)\, e^{t/\tau_m}$, and integration from 0 to t gives

$$v(t) = v(0)\, e^{-t/\tau_m} + \int_0^t \kappa(t-t')\, I(t')\, dt', \qquad \kappa(t-t') := \frac{R}{\tau_m}\, e^{-(t-t')/\tau_m}. \tag{10.5}$$

The voltage is a convolution of the input current with the exponential kernel κ. A current that flowed a time $t - t'$ ago enters with the weight $e^{-(t-t')/\tau_m}$, and the initial voltage is forgotten on the same time scale. The membrane has the echo state property (4.1) and fading memory (5.1), the loss of the start and the loss of old inputs, both at the rate $1/\tau_m$. A simulation with Euler steps of 0.05 ms reproduces the response to current steps and the convolution integral for a random current, with largest deviations of 1.2×10^{-2} and 2.0×10^{-2} mV (Figure 32).

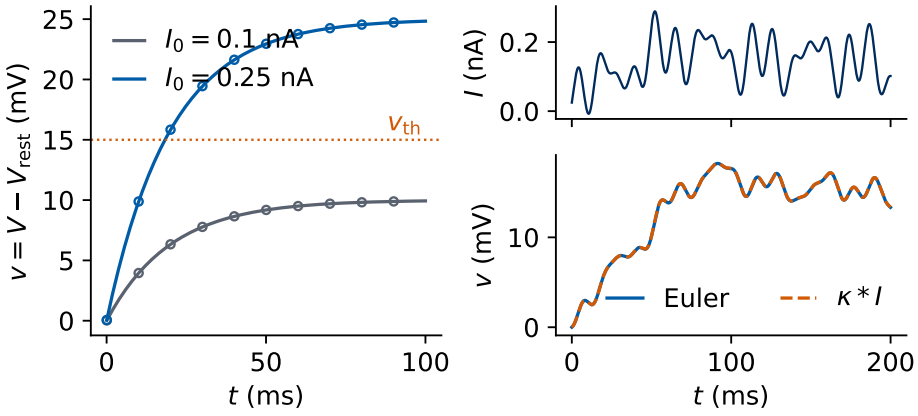


Figure 32: Relaxation of the membrane. Left, the voltage above rest v after two current steps, simulated (circles) and exact (lines), relaxing to RI_0 with the time constant τ_m, with a dotted line at 15 mV. Right, the response to a random current, simulated (blue) and from the convolution of equation (10.5) (dashed orange), labelled $\kappa * I$ in the legend, the convolution of the kernel κ with the current I.

Integrating equation (10.5) over one step Δt with a constant current I_k gives the exact update $v_k = e^{-\Delta t/\tau_m} v_{k-1} + (1 - e^{-\Delta t/\tau_m})\, RI_k$, which is equation (10.2) with $\alpha = 1 - e^{-\Delta t/\tau_m}$, equal to $\Delta t/\tau_m$ to first order. The network of this section uses Euler steps of 0.5 ms and holds each value of the record for 10 ms, 20 Euler steps. Without input the voltage then keeps the fraction $(1-\alpha)^{20}$ over one sample of the record, and the leak per sample, $1 - (1-\alpha)^{20}$, is 0.88 for $\tau_m = 5$ ms, 0.64 for 10 ms and 0.40 for 20 ms. These are the numbers to compare with the leak rates $\alpha \in \{0.3, 1\}$ of the echo state network, which updates once per sample.

10.5 Threshold and reset

The passive circuit relaxes to RI_0 under a constant current I_0 and stays there. The leaky integrate-and-fire (LIF) neuron adds one rule. When v reaches the threshold $v_{\text{th}} := V_{\text{th}} - V_{\text{rest}}$, here 15 mV, the neuron emits a spike, and v is reset to zero and held there for a refractory time t_{ref}, here 2 ms. Between spikes equation (10.4) holds, and at a spike the state jumps, so the LIF neuron combines a continuous evolution with discrete jumps. The spike carries no waveform in the model, and only its time matters (Figure 33).

For a constant current the neuron fires only if the equilibrium RI_0 lies above the threshold, that is, above the rheobase current $I_{\text{rh}} := v_{\text{th}}/R = 0.15$ nA. From the reset value $v = 0$ the voltage follows $v(t) = RI_0(1 - e^{-t/\tau_m})$, which equals v_{th} when $e^{-t/\tau_m} = 1 - v_{\text{th}}/(RI_0)$, after

$$T(I_0) = \tau_m \ln \frac{RI_0}{RI_0 - v_{\text{th}}}. \tag{10.6}$$

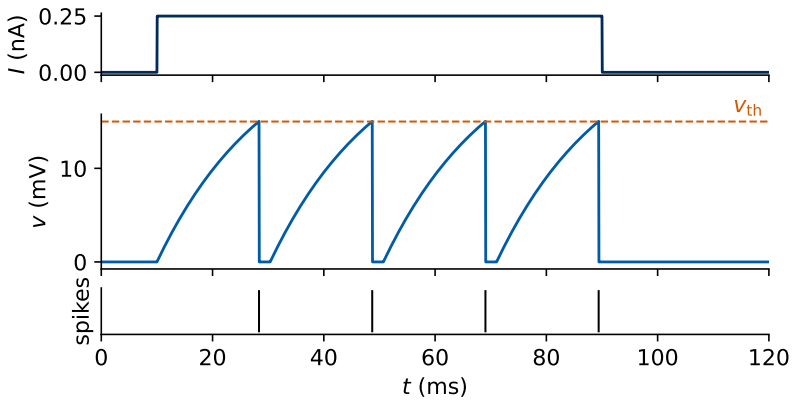


Figure 33: A leaky integrate-and-fire neuron. Top, a current step of 0.25 nA. Middle, the voltage above rest, charging towards RI_0, reaching the threshold (dashed) and being reset. Bottom, the spike times.

After the refractory time the cycle repeats, so the neuron fires periodically with the rate

$$\nu(I_0) = \frac{1}{t_{\text{ref}} + T(I_0)} \quad \text{for } I_0 > I_{\text{rh}}, \qquad \nu(I_0) = 0 \quad \text{for } I_0 \le I_{\text{rh}}. \tag{10.7}$$

For $I_0 = 0.25$ nA the equilibrium is $RI_0 = 25$ mV, and $T = \tau_m \ln(25/10) = 18.33$ ms, while the simulated first spike appears after 18.35 ms, the difference being set by the Euler step. With the refractory time the rate is $1/(20.33\text{ ms}) = 49.2$ Hz.

The rate curve $\nu(I_0)$ is the response function of the neuron, the spiking counterpart of the hyperbolic tangent (Figure 34). Below the rheobase the neuron is silent. Close above it the rate rises steeply, since T grows without bound as $RI_0 \to v_{\text{th}}$, logarithmically in the distance to the rheobase. At $I_0 = 0.16$ nA, just above the rheobase, $T = \tau_m \ln 16 = 55.5$ ms. For large currents $\ln[RI_0/(RI_0 - v_{\text{th}})] = -\ln(1 - v_{\text{th}}/RI_0) \approx v_{\text{th}}/(RI_0)$, so without refractory time the rate grows linearly, $\nu \approx RI_0/(v_{\text{th}}\tau_m)$, and with it the rate saturates at $1/t_{\text{ref}} = 500$ Hz. Simulation and formula agree within 0.6% for rates above 10 Hz.

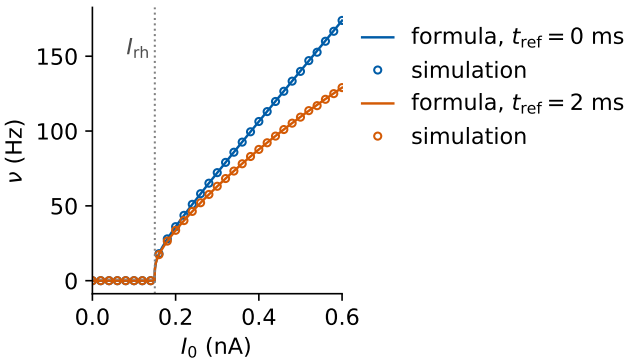


Figure 34: Firing rate ν of the LIF neuron against a constant current I_0, simulated (points) and from equation (10.7) (lines), without and with a refractory time of 2 ms. The rheobase I_{rh} is marked.

10.6 A network of spiking neurons

The liquid state machine is a fixed random recurrent network of LIF neurons with a trained linear readout. We measure the voltage of neuron i above rest in units of the threshold, so that every neuron fires at $v_i = 1$ and is reset to $v_i = 0$, and currents in units of the rheobase. The voltage of neuron i then obeys

$$\tau_m \frac{dv_i}{dt} = -v_i + I_i(t) + \tau_m \sum_j J\, W_{ij}^0 \sum_f \delta(t - t_j^f), \tag{10.8}$$

where t_j^f, $f = 1, 2, \ldots$, are the spike times of neuron j and δ is the Dirac delta, a meaning unrelated to the error δ of the closed-loop forecasts, so the last term raises v_i instantly by JW_{ij}^0 at every spike

of neuron j. A synapse is the connection through which a spike of one neuron changes the voltage of another. An excitatory neuron raises the voltage of the neurons it connects to, and an inhibitory neuron lowers it. The network has $N = 300$ neurons. The first 80% are excitatory and the rest inhibitory, and every ordered pair is connected with probability 0.1, which gives 30 incoming synapses per neuron on average. The sign pattern W^0 has the entries 1 for a synapse from an excitatory neuron, -5 for one from an inhibitory neuron and 0 for no synapse, and the coupling strength J sets the size of all jumps in units of the threshold, so $J = 0.01$ means that an excitatory spike raises the voltage of its targets by 1% of the threshold. Inhibition outweighs excitation on average, since $0.8 \cdot 1 < 0.2 \cdot 5$, as in a classic model of cortical networks by Brunel [31].

The record enters as a current. The value $z_k = 2s_k - 1 \in [-1, 1]$ of sample k is held for 10 ms, and neuron i receives the current $I_i = \theta_i + G\, w_i\, z_k$ during this time, with a bias θ_i uniform in $[0, 1]$, below the threshold, an input weight w_i uniform in $[-1, 1]$ and an input gain G. A neuron fires only when the signal pushes its current above 1, and since the signs of w_i differ, different neurons respond to different ranges of the signal.

A linear readout cannot use a train of spikes directly, since at almost every moment it is zero. Each spike train is therefore smoothed into a trace x_i that jumps by 1 at every spike of neuron i and decays as $e^{-t/\tau_{s,i}}$ between spikes, and the traces at the ends of the samples form the feature vector $\mathbf{r}_k = (x_1, \ldots, x_N)$, the counterpart of the state of the echo state network. The time constants $\tau_{s,i}$ are spread log-uniformly from 5 ms to a largest value τ_s, which means that their logarithms are uniform, so every factor of two between 5 ms and τ_s contains about the same number of neurons, so that different traces remember over different times. Figure 35 draws the network, and Table 3 lists what is fixed and what is chosen.

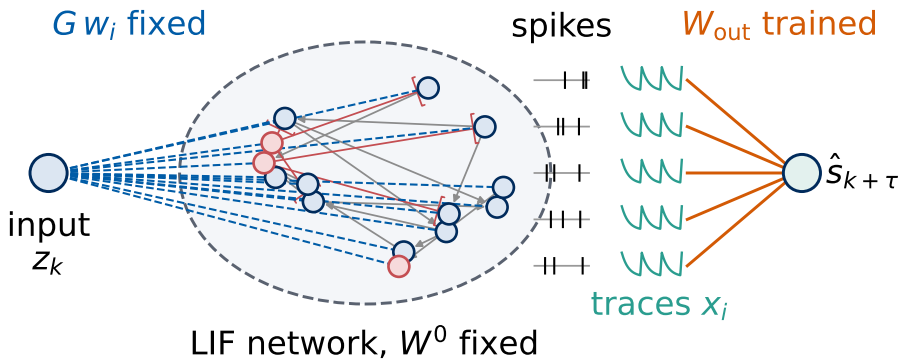


Figure 35: A liquid state machine. The input reaches a random network of excitatory (blue) and inhibitory (red) LIF neurons through fixed input weights (dashed), the neurons are coupled by fixed random synapses, each spike train is smoothed into a trace, and only the linear readout of the traces (orange) is trained.

trained	readout W_{out}, $\mathbf{b}$
fixed, random	sign pattern W^0, input weights w_i, biases θ_i, trace time constants $\tau_{s,i}$
chosen on validation data	coupling J, input gain G, membrane time constant τ_m, largest trace time τ_s, ridge strength β

Table 3: The parts of the liquid state machine.

Driven by the laser record at $J = 0.01$, $G = 2$, $\tau_m = 10$ ms and $\tau_s = 300$ ms, a setting chosen for illustration, the network fires in bursts whose timing follows the signal (Figure 36). The rescaled intensity is small most of the time, so z_k is mostly negative, and the neurons with $w_i < 0$ fire between the pulses, while each pulse silences them. Over the whole record the mean rate is 19.0 Hz, and 104 of the 300 neurons never fire. Figure 37 shows three traces. One with a short time constant follows the individual pulses, and two with longer ones integrate the firing over many pulses. Each trace is a

count of recent spikes weighted by their age, a nonlinear function of the input history with its own memory time.

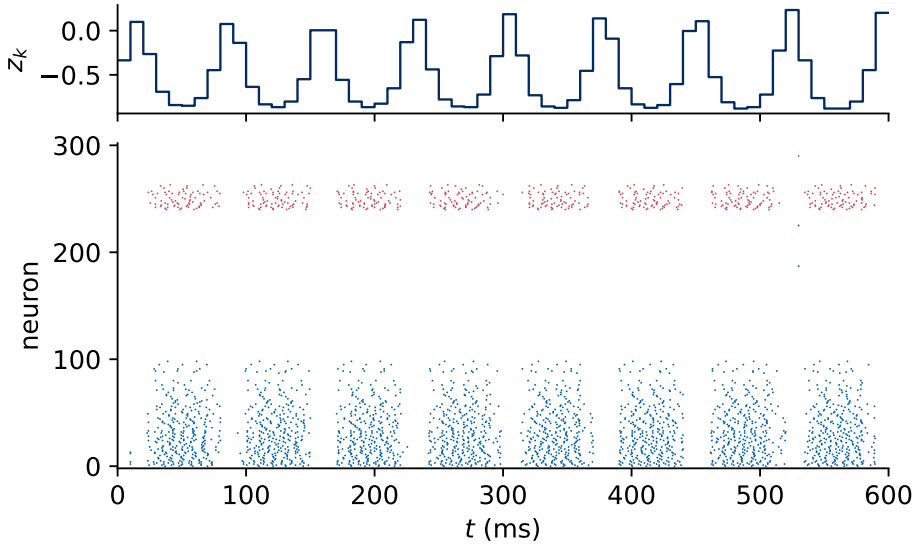


Figure 36: A spiking network driven by the laser record. Top, the input $z_k = 2s_k - 1$ over 600 ms of neuron time. Bottom, one dot per spike and one row per neuron, excitatory neurons (blue) sorted by their input weight w_i from negative (bottom) to positive, and inhibitory neurons above them.

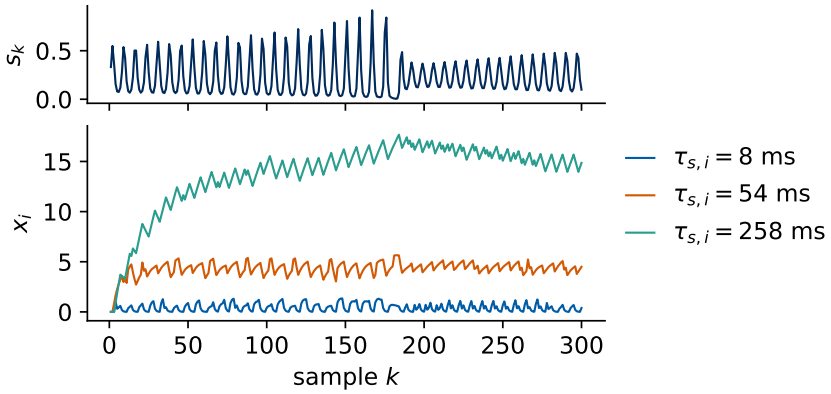


Figure 37: Features of the spiking reservoir. Top, the rescaled laser record s_k over 300 samples. Bottom, the traces x_i of three neurons at the ends of the samples, for three trace time constants $\tau_{s,i}$.

10.7 The echo state property for spikes

The echo state property is tested as for the echo state network, with two copies of the network started from voltages drawn uniformly from $[0, 1]$ and driven by the same 1000 samples of the laser record. Two quantities measure their difference. The relative trace distance D_k is the distance $\|\mathbf{r}_k - \mathbf{r}'_k\|$ of the two feature vectors divided by the mean length of the feature vector over the run. The differing fraction is the fraction of neurons whose spikes during sample k differ between the two copies.

Without coupling the network locks to the input, and after 59 samples both copies emit identical spike trains (Figure 38). The trace distance then decays exponentially, since the remaining difference is the memory of the first spikes, which fades with the trace time constants. With $J = 0.01$ the copies lock after 532 samples. With $J = 0.03$ and $J = 0.1$ a fraction 0.15 and 0.26 of the neurons still fire differently in the last 100 samples. A spike in one copy and none in the other changes the voltages of about 30 neighbours, and with strong coupling these differences multiply, which plays the role that a spectral radius above 1 plays in the echo state network.

A network whose biases θ_i are uniform in $[1, 2]$, above the threshold, fails even without coupling.

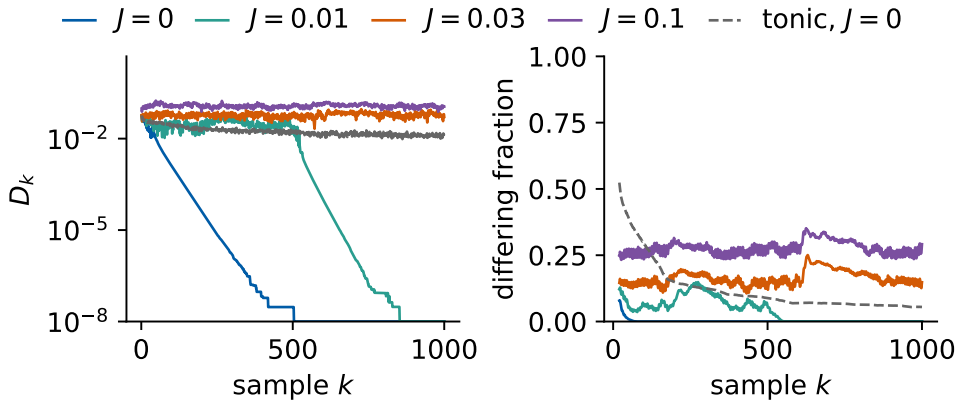


Figure 38: The echo state property for spikes. Two copies of the network with different initial voltages, driven by the same laser record at $G = 2$, $\tau_m = 10$ ms and $\tau_s = 300$ ms. Left, the relative trace distance D_k against the sample k. Right, the fraction of neurons whose spikes differ, averaged over windows of 20 samples, for four couplings J and for an uncoupled network with biases above threshold (dashed, labelled tonic, a neuron that fires on its own under a constant current).

A neuron with a current above threshold fires periodically on its own, and two copies that start at different phases of this cycle keep their phase difference under a constant input. A current that moves the neuron across its threshold makes the spike times reproducible. Mainen and Sejnowski observed this in cortical neurons, whose spike times varied from trial to trial under a constant current and repeated under a fluctuating one [32]. This is the reason for biases below the threshold.

The test is repeated over a grid of couplings for three input gains, with three pairs of starts each (Figure 39). A stronger input makes more neurons fire, more spikes travel through the couplings, and the locking fails at smaller couplings. The largest coupling on the grid with a median differing fraction below 0.01 is 0.02 for $G = 1$ and 0.01 for $G = 4$. The test is finite, with 1000 samples of one input and three pairs of starts, so these values hold for this network and this input.

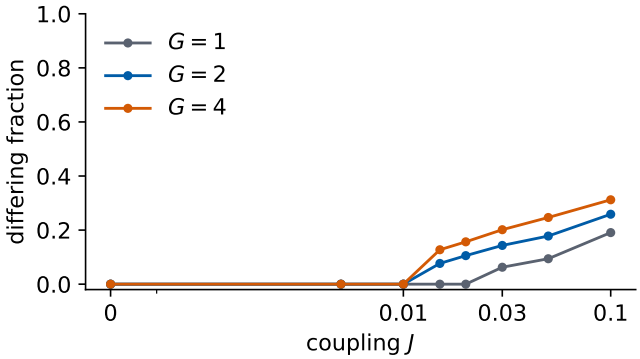


Figure 39: Locking of the spiking reservoir against the coupling J. The median, over three pairs of starts, of the fraction of neurons whose spikes differ in the last 100 of 1000 samples, for three input gains G.

10.8 Memory of the spiking reservoir

The question is how much of its input history a network of spiking neurons holds, compared with an echo state network of the same size. The memory capacity of equation (6.1), measured with the protocol of Figure 15 and the random input fed as z_k, answers it (Figure 40). In the figure, look at how fast each curve falls from 1. The spiking curves fall after one or two delays, the curves of the echo state networks after tens of delays. With 300 neurons the memory capacity lies between 2.1 and 3.0 over three settings, against 44.1 for an echo state network of 300 units. The present input is read well, $C(0) = 0.978$, and the previous one only partly, $C(1) = 0.659$. Two explanations are plausible and were not tested separately. A random value held for 10 ms produces at most a few spikes per neuron, a coarse and strongly nonlinear code of the input. And each trace is a sum of

decaying exponentials over past spikes, from which the contribution of one particular past sample is hard to separate when the time constants are close. The echo state network instead mixes its state through the recurrent matrix, which moves information about each past input into new directions of the state space. A random input is the hardest case for the spike code, since every sample is new, while a smooth record changes little from sample to sample.

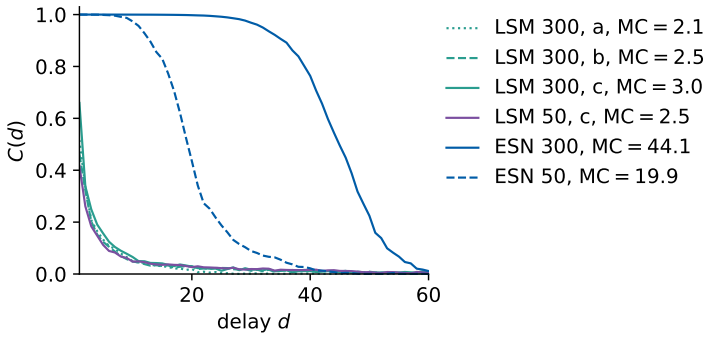


Figure 40: Recall capacity $C(d)$ against the delay d for three settings of the spiking reservoir with 300 neurons, one with 50 neurons, and echo state networks with 50 and 300 units, all with the same protocol.

10.9 Forecasts of the spiking reservoir

The question is whether so short a memory still allows forecasts of the five records. The coupling $J \in \{0, 0.01, 0.03\}$, the gain $G \in \{1, 2, 4\}$, the membrane time constant $\tau_m \in \{5, 20\}$ ms, the largest trace time $\tau_s \in \{100, 300, 1000\}$ ms and the ridge strength are chosen on the validation block, with the protocol of the echo state network and one random network for all records. Table 4 and Figure 41 show a clear split. On the smooth records the spiking reservoir comes close to the echo state network, $C_{\text{tot}} = 38.5$ against 39.2 on Lorenz, more than twice NVAR. A smooth record changes little within a sample, so the recent history is summarised well by traces with different time constants, and the short memory for random inputs does not matter there. On the laser it reaches only 20.5, and already its one-step capacity is 0.803. The laser pulses change within one or two samples, and a neuron emits at most a few spikes in a sample of 10 ms, so the spike code resolves the fast pulsations coarsely. The silent neurons are a likely further cause. Other encodings of the input were not tried.

test C_{tot}	Lorenz	MG 15	MG 25	MG 35	Santa Fe
AR	15.4	49.8	33.1	38.0	16.5
NVAR	15.4	50.0	45.5	48.6	34.7
ESN, 300 units	39.2	50.0	48.3	49.8	34.7
spiking, 300 neurons	38.5	49.7	45.9	47.2	20.5
selected J	0	0	0.03	0	0
one trace time τ_s	33.8	47.1	20.3	25.6	12.3

Table 4: Total capacity on the test block of the spiking reservoir next to AR, NVAR and the echo state network. The last two rows give the coupling selected on the validation block and the score with one common trace time for all neurons in place of the spread from 5 ms to τ_s.

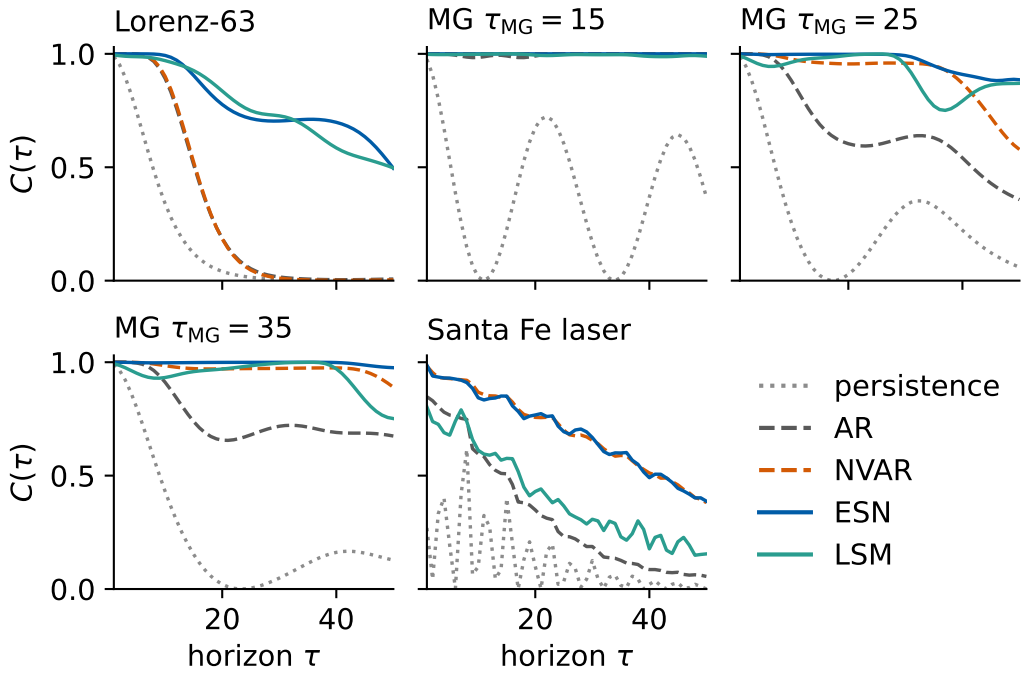


Figure 41: Capacity $C(\tau)$ on the test block against the horizon for persistence, AR, NVAR, the echo state network and the spiking reservoir, one panel per record.

10.10 Ingredients of the spiking reservoir

The validation scores show which ingredients matter. The coupling does not help. The uncoupled network, $J = 0$, gives the best validation score on four records. On MG 25 the selected $J = 0.03$ is ahead of $J = 0$ by 0.014, far less than the range of 0.6 between the test scores of five random networks on this record, a spread of test scores that bounds how finely the scores separate models, so there the two couplings are tied. On the laser the score falls from 23.8 at $J = 0$ to 19.8 at $J = 0.03$. The selected network is therefore a set of independent LIF neurons on four records, each a nonlinear filter of the input with its own bias, input weight and trace time. A likely explanation, consistent with the locking test but not tested directly, is that recurrent spikes break the locking to the input before they add useful features.

The spread of trace times is essential. With one common trace time for all neurons the score on MG 25 falls from 45.9 to 20.3 and on Lorenz from 38.5 to 33.8 (Table 4). A single exponential filter gives every feature the same memory time, and the readout cannot weight recent and older parts of the history differently. As an analogy, the spread of trace times plays the role of the range of relaxation rates of the eigenmodes of an echo state network, the numbers (3.3). Five random networks give test scores within 2.2 of each other on every record, and on Lorenz the five spiking networks, between 37.4 and 38.6, all stay below the five echo state networks, between 39.0 and 40.0. These conclusions hold for a network of 300 neurons, one encoding of the input and the grids tested.

10.11 Spikes and gradients

The reservoir never passes a derivative through a spike, and this is one of its attractions for spiking networks. A spike is a step function of the voltage, $S = \Theta(v - v_{\mathrm{th}})$, whose derivative vanishes everywhere except at the threshold. The spike count of a neuron is therefore a staircase function of every weight, and its exact gradient is zero almost everywhere, so gradient descent receives no information about which weight to change. Trained spiking networks keep the step in the forward computation and replace its derivative in the gradient computation by a smooth peak, a surrogate gradient [33]. A common choice is the derivative of a fast sigmoid [34],

$$\frac{\partial S}{\partial v} \;\to\; \frac{1}{v_{\mathrm{th}}}\,\frac{1}{\left(1+\gamma_{\mathrm{sg}}|\zeta|\right)^2}, \qquad \zeta = \frac{v - v_{\mathrm{th}}}{v_{\mathrm{th}}}, \tag{10.9}$$

where the steepness γ_{sg} sets the width of the peak. For the single neuron of Figure 34 with $\gamma_{sg} = 20$, the surrogate gradient of the spike count in a window of 500 ms follows the slope of the rate curve within 11% for currents above 0.2 nA (Figure 42). Below the rheobase the true spike count and its slope are zero, while the surrogate gradient is not, because it responds to a voltage close to the threshold whether or not the neuron fires. The surrogate is an approximation whose quality depends on γ_{sg} and on the network. A reservoir avoids the problem altogether, since only its linear readout is fitted.

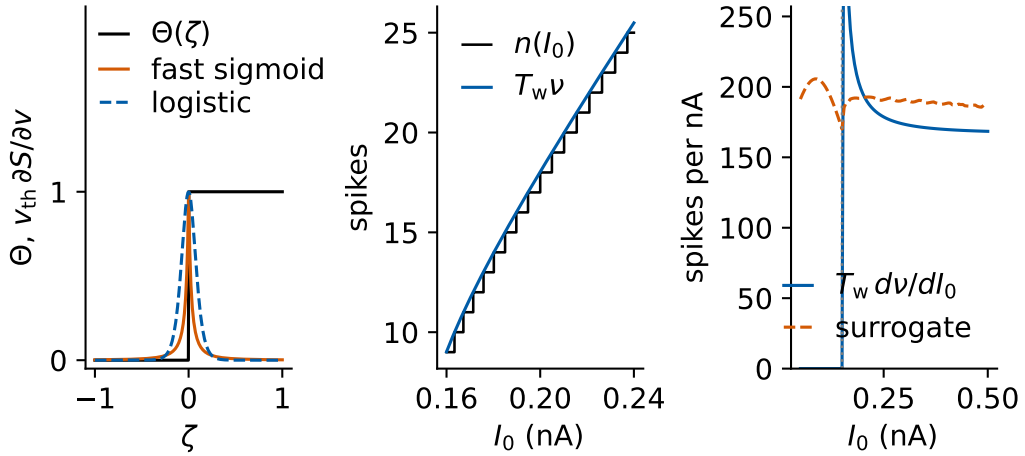


Figure 42: Spikes and gradients. Left, the step function Θ of the spike and two smooth surrogate derivatives. Middle, the spike count of a neuron in 500 ms against a constant input current, a staircase, with the smooth curve from the rate formula. Right, the surrogate gradient of the spike count against the slope of the smooth curve.

The spiking reservoir showed that a threshold can replace the hyperbolic tangent and that a bank of different single-neuron filters carries most of the forecasting power. Its state is still a set of real numbers, voltages and traces. A reservoir can be built from a system whose states are discrete, provided the real-valued record can be written into it.

11 Cellular automata

11.1 Discrete systems with local rules

The requirements for a reservoir ask for no neural network, and the most discrete driven system one can build is a cellular automaton. Its space is a row or a grid of cells, its time advances in steps, every cell holds one of a few states, and every cell is updated by the same local rule from the states of its neighbours. On digital hardware such a system needs nothing beyond the lookup of a small table or a few bit operations per cell.

Cellular automata were introduced by von Neumann as abstract machines that build copies of themselves [38]. The Game of Life of Conway, made known by Gardner [37], showed that three local rules on a square lattice produce structures that move, collide and persist, a small universe with objects of its own. A glider of five live cells returns to its shape after four steps, shifted by one cell diagonally, and travels across the lattice (Figure 43). Wolfram studied the one-dimensional automata systematically and sorted their behaviour into a few classes [35, 36]. In physics cellular automata are models in which a few local rules reproduce collective behaviour. Frisch, Hasslacher and Pomeau showed that particles that hop and collide on a lattice obey, on large scales, the Navier–Stokes equation of a fluid [39], and Nagel and Schreckenberg modelled freeway traffic with a cellular automaton of cars on a lattice road [40].

An automaton differs from the reservoirs met so far in a way that matters for the requirements. Its cells hold bits and do not leak. Nothing in the rule makes an old state fade by a fixed factor per step, as the leak rate does in equation (2.8), and a difference in one cell is either 0 or 1, so a difference

between two configurations cannot grow in amplitude, only in the number and extent of the differing cells, and it can also vanish. Whether such a system can forget its start, keep a fading record of its input and respond nonlinearly is therefore an open question, and the dynamics of the automaton without input is the place to start.

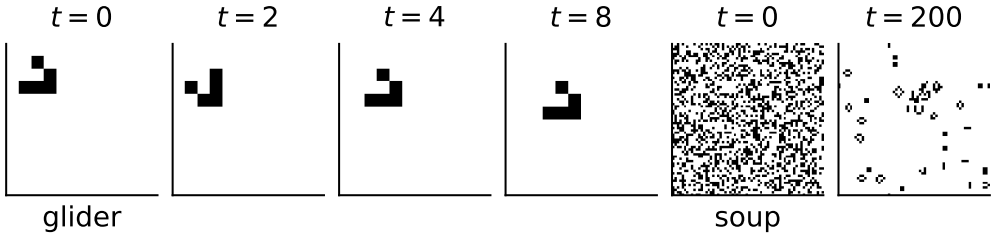


Figure 43: The Game of Life on a square lattice. A dead cell with three live neighbours among its eight becomes alive, a live cell with two or three live neighbours survives, and every other cell is dead at the next step. Left, a glider at the steps 0, 2, 4 and 8. Right, a random configuration (labelled soup) at step 0 and at step 200.

11.2 Definition of a cellular automaton

A cellular automaton is a dynamical system that is discrete in space, in time and in its states. A row of cells, each in state 0 or 1, is updated at every tick of a clock, and every cell looks only at itself and its two neighbours to decide its new state.

Cellular automaton

A cellular automaton consists of

- a lattice of L cells $j = 0, 1, \ldots, L-1$, here a ring, so that cell $L-1$ is the left neighbour of cell 0,
- $q = 2$ cell states, 0 and 1,
- a neighbourhood of radius $a = 1$, the three cells $j-1$, j and $j+1$,
- a local rule $f\colon \{0,1\}^3 \to \{0,1\}$ that returns the new state of a cell from the states of its neighbourhood.

The state of cell j after i iterations is $c_j^{(i)}$, and the configuration of the ring is $\mathbf{c}^{(i)} = (c_0^{(i)}, \ldots, c_{L-1}^{(i)})$. The global map F applies the rule to all cells at once,

$$c_j^{(i)} = f\big(c_{j-1}^{(i-1)}, c_j^{(i-1)}, c_{j+1}^{(i-1)}\big) \quad \text{for all } j, \qquad \mathbf{c}^{(i)} = F\big(\mathbf{c}^{(i-1)}\big), \tag{11.1}$$

and the evolution of the automaton is the iteration of F from an initial configuration $\mathbf{c}^{(0)}$.

Figure 44 draws one update. Every cell reads three cells of the previous configuration, its left neighbour, itself and its right neighbour, and all cells are updated together. A cell can therefore influence only its neighbours in one iteration, a maximal speed of signals of one cell per iteration. Automata with more states or wider neighbourhoods exist, and this chapter uses only these.

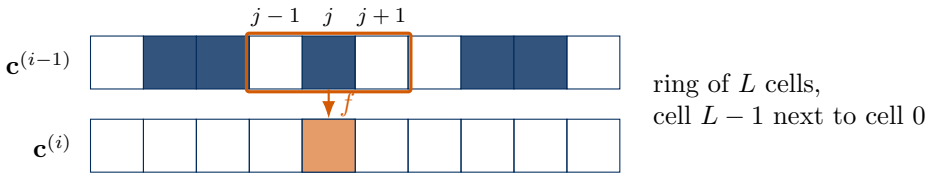


Figure 44: One update of a cellular automaton with $q = 2$ states (white 0, dark 1) and radius $a = 1$. The new state of cell j (orange) is the local rule f applied to the three framed cells of the previous configuration. All cells are updated at once by the same rule.

11.3 Elementary rules and their numbers

The simplest automata have $q = 2$ and $a = 1$, and Wolfram called them elementary [35]. A neighbourhood of three binary cells is one of $2^3 = 8$ patterns, and a rule assigns a new state to each of them, so there are $2^8 = 256$ elementary rules. Listing the patterns in the order $111, 110, 101, 100, 011, 010, 001, 000$ and reading the eight new states as a binary number gives the rule number. Rule 90 has the new states $0, 1, 0, 1, 1, 0, 1, 0$, and

$$01011010_2 = 64 + 16 + 8 + 2 = 90. \tag{11.2}$$

Inspection of its table shows that the new state is 1 when the two neighbours differ and 0 when they agree, whatever the centre, so rule 90 sets every cell to the exclusive or of its two neighbours, $c_j^{(i)} = c_{j-1}^{(i-1)} \oplus c_{j+1}^{(i-1)}$, which is addition modulo 2. Figure 45 shows its table next to that of rule 30, whose new state is $c_{j-1} \oplus (c_j \vee c_{j+1})$, with $\vee$ the logical or.

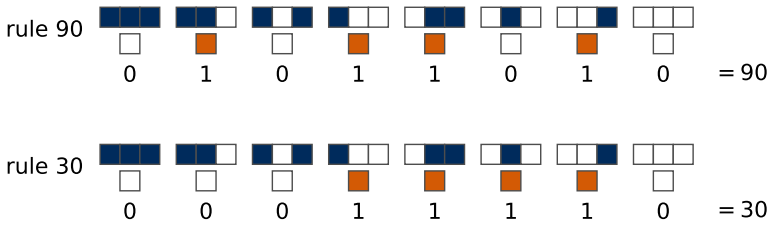


Figure 45: Rule tables of two elementary automata. For each of the eight neighbourhood patterns (top row of each table) the new state of the centre cell (bottom row), for rule 90 and rule 30. Read as binary numbers, the bottom rows give the rule numbers.

A single cell in state 1 on a ring of nine cells evolves under rule 90 as

$$\begin{array}{ll} i = 0 & 000010000 \\ i = 1 & 000101000 \\ i = 2 & 001000100 \\ i = 3 & 010101010 \\ i = 4 & 100000001 \end{array}$$

At $i = 2$ the centre is 0 because at $i = 1$ its two neighbours are both 1 and $1 \oplus 1 = 0$, and at $i = 4$ the cells around the centre cancel in pairs in the same way. Continued on a larger ring, the pattern grows into nested triangles.

Rule 90 is linear modulo 2. Its new state is the sum of the two neighbours modulo 2, so the global map is a matrix acting on the configuration modulo 2, and superposition holds, $F(\mathbf{c}+\mathbf{c}') = F(\mathbf{c})+F(\mathbf{c}')$ with the sum taken cell by cell modulo 2. On a ring of eight cells, for example,

$$\begin{array}{ll} \mathbf{c} = 11000000 & F(\mathbf{c}) = 11100001 \\ \mathbf{c}' = 00010110 & F(\mathbf{c}') = 00100111 \\ \mathbf{c} + \mathbf{c}' = 11010110 & F(\mathbf{c} + \mathbf{c}') = 11000110 \end{array}$$

and adding the two right-hand entries of the first rows modulo 2 gives the third. Rule 150, $c_j \leftarrow c_{j-1} \oplus c_j \oplus c_{j+1}$, is linear modulo 2 as well.

11.4 Three kinds of behaviour and four classes

Started from one cell in state 1 or from a random configuration, elementary rules show three kinds of long-time behaviour (Figure 46). Under rule 32 the single cell disappears at the first iteration, and the random configuration is empty from iteration 3 on, after which nothing changes. This behaviour is trivial. Under rule 108 the single cell stays where it is, and the random configuration settles into vertical stripes, some fixed and some blinking, so that the whole lattice repeats with period 2 from iteration 2 on. This behaviour is regular. Under rule 30 the single cell grows into an irregular triangle, and the random configuration does not repeat within the 60 iterations shown. This behaviour is irregular, its patterns do not repeat, and a local change spreads through the lattice.

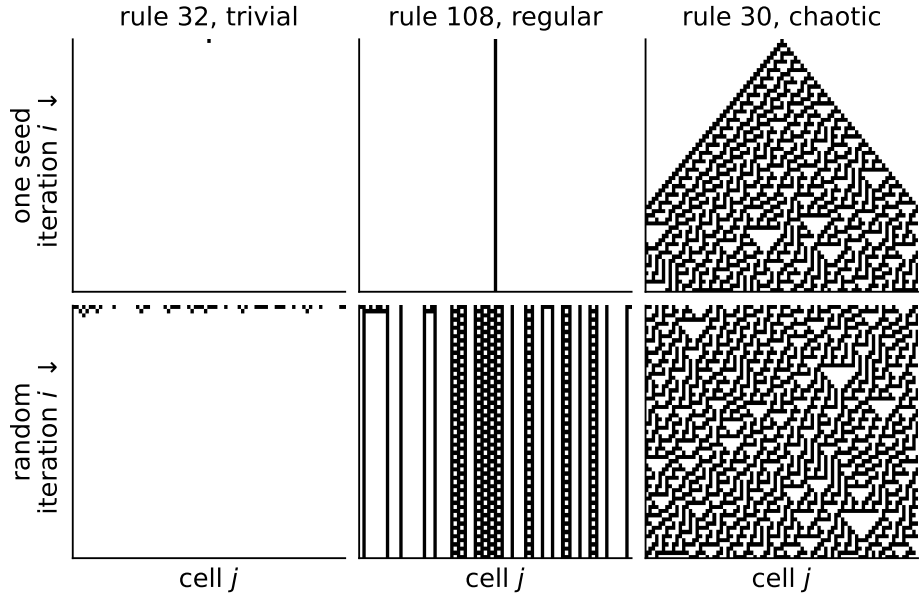


Figure 46: Three kinds of behaviour of elementary rules on a ring of 81 cells over 60 iterations, with the iteration i running downwards and black cells in state 1. Top, from a single cell in state 1. Bottom, from a random configuration. Left, rule 32, trivial. Middle, rule 108, regular. Right, rule 30, irregular.

Wolfram sorted the behaviour of automata started from random configurations into four classes [36]. Class I evolves to a homogeneous fixed configuration, like rule 32. Class II evolves to isolated stable or periodic structures, like rule 4, which keeps isolated cells in state 1 and erases everything else. Rule 108 in Figure 46 behaves in this way, with structures that repeat every two iterations. Class III evolves to aperiodic, random-looking patterns, like rule 30 and the linear rules 90 and 150. Class IV forms localised structures that travel and interact over long times, and rule 110 is the standard example.

Rule 32 is in class I. Its number is 00100000_2, so only the pattern 101 produces a 1. A cell becomes 1 only when it is a 0 between two 1s, so an alternating stretch $\ldots 1\,0\,1\,0 \ldots$ keeps its pattern, shifted by one cell, but loses a cell at each end per iteration. On a ring of 14 cells,

$$
\begin{array}{ll}
i = 0 & 0\,1\,0\,1\,0\,1\,0\,1\,1\,0\,0\,1\,0\,0 \\
i = 1 & 0\,0\,1\,0\,1\,0\,1\,0\,0\,0\,0\,0\,0\,0 \\
i = 2 & 0\,0\,0\,1\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0 \\
i = 3 & 0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0 \\
i = 4 & 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0
\end{array}
$$

the longest alternating stretch shrinks from both ends until nothing is left. On 1000 random rings of 101 cells the lattice is empty after a median of 3 and at most 8 iterations. The time to erase a random ring is set by its longest alternating stretch.

11.5 Traffic on a ring road

Cellular automata are also physical models. Rule 184 is the simplest model of traffic on a one-lane ring road. A cell in state 1 holds a car, and at every step a car moves one cell to the right if the cell ahead is empty and stays otherwise. Its table, $10111000_2 = 184$, says that a cell is occupied at the next step if its car is blocked ($c_j = c_{j+1} = 1$) or if a car arrives from the left ($c_{j-1} = 1$, $c_j = 0$). Cars are never created or destroyed, so the density η of cars is conserved. The flux Φ is the number of cars that move per cell and step.

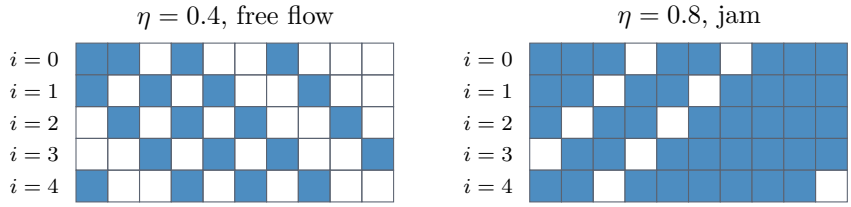


Figure 47: Rule 184 on a ring of ten cells, cars in blue, moving to the right. Left, four cars. After the first step every car has an empty cell ahead, and all four move at every step. Right, eight cars. Only the two cars directly behind the two empty cells move at each step, and the empty cells travel to the left.

The fundamental diagram $\Phi(\eta)$ follows from two observations, which Figure 47 illustrates. Below half filling every car eventually finds an empty cell ahead. A car that has an empty cell ahead moves, leaves an empty cell behind and finds the cell ahead still empty at the next step, unless a car stands there, so after the initial queues have dissolved every car moves at every step and $\Phi = \eta$. Above half filling the roles of cars and empty cells exchange. An empty cell with a car on its left is filled at the next step, and the empty place moves one cell to the left, so the empty cells behave like cars driving to the left, with density $1 - \eta$ below one half. By the first observation applied to them, every empty cell moves at every step, and each such move is the move of one car, so $\Phi = 1 - \eta$. Together,

$$\Phi(\eta) = \min(\eta,\, 1 - \eta), \tag{11.3}$$

with a cusp at $\eta = 1/2$, the transition between a free-flow phase and a jammed phase. In the jammed phase queues of standing cars travel backwards, against the direction of the cars. A simulation on rings of 1000 cells reproduces equation (11.3) to four decimals on a grid of densities (Figure 48).

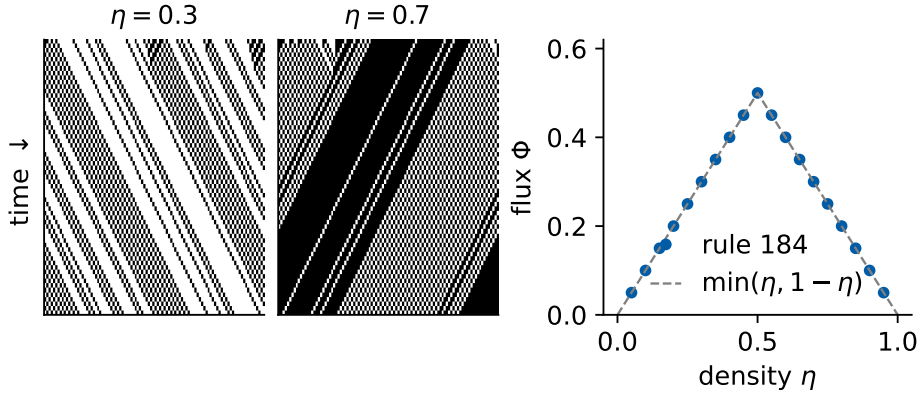


Figure 48: Rule 184 as a traffic model. Left, space-time diagrams at the densities $\eta = 0.3$ (free flow) and $\eta = 0.7$ (jams that travel backwards). Right, the measured flux Φ against the density, with the curve of equation (11.3).

11.6 Damage spreading

The echo state property compared two copies of a reservoir started in different states. Its counterpart for an automaton without input is damage spreading. One cell of a random configuration is flipped, and the set of cells in which the two copies differ, the damage, is followed in time (Figure 49).

Under rule 4 the damage stays on one cell. Under rule 90 superposition decides the outcome. The flipped copy is $\mathbf{c} + \mathbf{p}$, with $\mathbf{p}$ a configuration with a single 1, and $F^i(\mathbf{c} + \mathbf{p}) = F^i(\mathbf{c}) + F^i(\mathbf{p})$, so the damage is the evolution of a single cell, the nested triangles, whatever the background. Under rule 30 the damage fills an asymmetric cone whose right edge moves by 1.00 cell per iteration and whose left edge by 0.23. The formula $c_{j-1} \oplus (c_j \vee c_{j+1})$ explains the asymmetry. A difference in the left neighbour always changes the new state, through the exclusive or, so the damage moves right at the maximal speed. A difference in the right neighbour changes the new state only when $c_j = 0$, so the left edge advances only some of the time. After 128 iterations the damage of rule 30 covers a fraction 0.323 of a ring of 257 cells, and that of rule 110 a fraction 0.158.

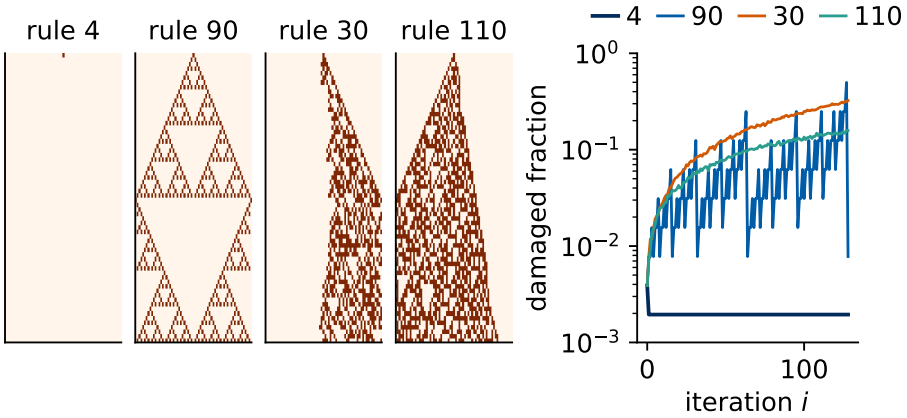


Figure 49: Damage spreading. Left, the cells in which two copies differ after one cell was flipped (orange), over 64 iterations, for rules 4, 90, 30 and 110. Right, the damaged fraction of a ring of 257 cells against the iteration, averaged over 20 random starts.

A difference between two configurations cannot grow in size, since a cell is either 0 or 1, and it grows instead in extent. The spreading rate measures this growth,

$$v_f := \frac{\text{width of the damaged region after } i \text{ iterations}}{2i + 1}, \tag{11.4}$$

where $2i + 1$ is the largest possible width, reached when the damage grows by one cell per iteration on each side. So $v_f = 1$ means spreading at the maximal speed in both directions and $v_f \approx 0$ a width that stays fixed. A damaged region that travels along the lattice without growing also has $v_f \approx 0$. As an analogy, v_f plays the role of the largest Lyapunov exponent λ_1 of equation (8.2), with $v_f \approx 0$ corresponding to $\lambda_1 \leq 0$ and $v_f > 0$ to $\lambda_1 > 0$. The analogy is loose, since λ_1 is an exponential rate of growth of a size and v_f a linear rate of growth of an extent.

12 A cellular automaton as a reservoir

The automata of Figures 46 and 49 ran without input. Their damage stayed in place, travelled or spread, and their configurations died out, repeated or stayed irregular. A reservoir is driven by a record, so the question is what happens to an input after it is written into the cells, whether the configuration then holds a readable and fading record of the recent inputs, and which rules make it do so.

12.1 The scheme

Yilmaz replaced the echo state network by a cellular automaton [42]. Each value of the record is first rounded to one of Q levels and written as a code word of a few binary digits. At every input step these digits are written into some cells of the automaton, the rule is iterated a fixed number of times N_{gen}, and the configurations of these iterations, the space-time pattern of the evolution, are

concatenated into the feature vector of a linear readout. The configuration left after the last iteration is the starting point of the next input step, so the automaton accumulates the history of the input, as the state of an echo state network does. Several random placements of the input on the lattice, R of them, are used side by side in the compartments, consecutive segments, of L_c cells each, of one ring of $L = RL_c$ cells, and contribute separate parts of the feature vector. The rule acts on the whole ring, so bits that move along the lattice pass from one compartment into the next. The letter R counts compartments here and has nothing to do with the resistance of the spiking neuron. Nothing inside is trained, the readout is fitted by ridge regression, and the automaton needs only bit or modulo operations instead of products of real matrices (Figure 50).

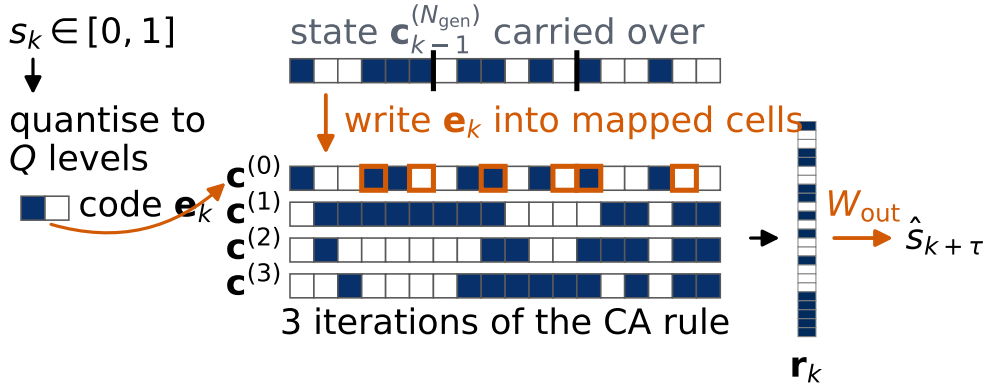


Figure 50: One step of a cellular automaton reservoir, for rule 90 with three compartments of six cells, an input coded in two bits and $N_{gen} = 3$ iterations. The binary code of the input s_k is written into fixed, randomly chosen cells (orange frames) of the configuration $\mathbf{c}_{k-1}^{(N_{gen})}$ left by the previous step, the rule is iterated, and all configurations form the feature vector $\mathbf{r}_k$ of the linear readout to the forecast $\hat{s}_{k+\tau}$.

12.2 A record of the input in a driven automaton

A small example shows how a driven automaton can hold a record of its past inputs. A ring of 12 cells starts empty, and at every input step k a bit b_k is written into the first cell, replacing its content, after which the rule is applied once. Figure 51 shows the configurations for the input $1, 0, 1, 1, 0, 0, 1, 0, 0, 0$ under three rules.

Under the rule that copies the left neighbour, rule 240, every bit moves one cell to the right per step. At step k cell j holds the bit written j steps earlier, so the row at $k = 7$, $1\,0\,0\,1\,1\,0\,1\,0\ldots$, is the input $b_7, b_6, \ldots, b_1$ read backwards. The configuration is a delay line like the shift reservoir of Figure 14, and a linear readout recovers b_{k-d} from cell d. After 12 steps a bit returns to the first cell and is overwritten by the new input, so the record holds the last 12 inputs and forgets everything older. This is fading memory of the simplest kind, a memory that ends abruptly at a fixed age. A rule under which a single changed cell drifts along the lattice without spreading is called a shift rule here, and rule 240 is the simplest one. The same overwriting gives the echo state property. Two copies of the ring started from different random configurations and driven by the same bits become identical once every cell has passed through the written cell, after at most 12 steps in 20 computed pairs.

Rule 180, another shift rule, also moves bits to the right, and it changes them on the way, since it computes $c_{j-1} \oplus c_j \oplus c_j c_{j+1}$, with $\oplus$ the sum modulo 2. The record is partly mixed, and a linear readout recovers past bits only partly. Rule 90 sends every bit in both directions at once, and copies of different inputs meet and add modulo 2. On this ring the sums erase all but the previous bit. The driven ring visits only 64 configurations, the previous bit b_{k-1} is a function of the configuration and the older bits are not, and even b_{k-1} enters only through sums modulo 2 that a linear readout cannot extract. Rules 32 and 4, finally, keep nothing of the past. Rule 32 erases every isolated bit, and rule 4 keeps a bit only in its own cell, where the next input overwrites it.

A computed memory test makes these statements quantitative. The ring is driven by 8000 random bits, the features are the configurations before and after the iteration, $\mathbf{c}_k^{(0)}$ and $\mathbf{c}_k^{(1)}$, and for every

delay d a least-squares readout fitted on the first half returns b_{k-d} on the second half, with the recall capacity $C(d)$ of equation (6.1). Table 5 gives the result. Rule 240 recalls the last 11 past bits without error and nothing beyond, rule 180 recalls them partly, with a memory capacity of 7.2 out of the 11 that the ring can hold, and rules 90, 32 and 4 recall nothing but the present bit. Rule 30, which spreads differences irregularly, recalls the previous bit with $C = 0.24$ and little else. A readable record of the recent input needs a rule that moves bits along the lattice without spreading and mixing them. The example is a toy, with one written cell, one iteration per step and a ring of 12 cells, and it is the picture against which the reservoir results can be read.

rule	$C(1)$	$C(2)$	$C(3)$	$C(5)$	$C(8)$	$C(11)$	$C(12)$	MC
240	1.00	1.00	1.00	1.00	1.00	1.00	0.00	11.0
180	0.60	0.59	0.88	0.60	0.86	0.20	0.00	7.2
30	0.24	0.08	0.06	0.01	0.00	0.00	0.00	0.4
90, 32, 4	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.0

Table 5: Recall capacity $C(d)$ of past bits for the driven ring of Figure 51, with a linear readout of $\mathbf{c}_k^{(0)}$ and $\mathbf{c}_k^{(1)}$, from 8000 random input bits, half for the fit and half for the test. MC is the sum of $C(d)$ over $d \geq 1$. All rules recall the present bit, $C(0) = 1$, since it is written into the features.

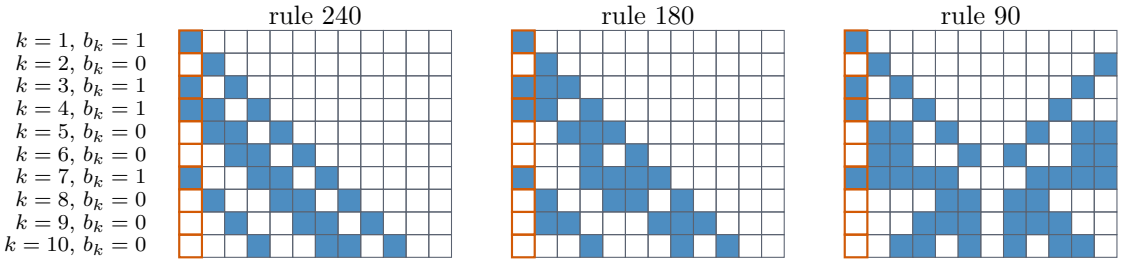


Figure 51: A driven automaton as a record of its input. A ring of 12 cells, empty at the start, receives the bit b_k in its first cell (orange frame) at every input step k, by replacement, and the rule is then applied once. Each row is the configuration $\mathbf{c}_k^{(0)}$ right after the writing, cells in state 1 in blue. Left, the shift rule 240, $c_j \leftarrow c_{j-1}$. Middle, the shift rule 180. Right, rule 90. The grids are computed from the rules.

12.3 Three fates of an injected bit

The twelve-cell ring of Figure 51 followed many bits under three rules. Figure 52 follows a single bit on a quiet ring of 81 cells over 40 iterations under nine rules, and it shows what can happen to an input once it is written. Above each panel the rule is written in its algebraic normal form, a sum modulo 2 of products of the three neighbourhood cells, which shows which cells the rule copies and which products it adds. Rule 180, $c_{j-1} \oplus (c_j \wedge \neg c_{j+1})$ with $\wedge$ the logical and and $\neg$ the negation, reads $c_{j-1} \oplus c_j \oplus c_j c_{j+1}$ in this form, since $\neg c_{j+1} = 1 \oplus c_{j+1}$.

Three things can happen to the bit. Under rule 180 it moves to the right by one cell per iteration on average, alternating between a single bit and a pair of neighbouring bits, rule 38 moves it to the left in the same alternating way, and rule 138 carries it to the left intact, one cell per iteration. Rule 27 also moves the bit along the lattice, on a background that blinks between all 0 and all 1, because it maps 000 to 1 and 111 to 0. A bit that travels intact remains a readable record of an earlier input. Under rules 90 and 60 the bit spreads into a triangle of nested triangles, and under rule 30 into an irregular triangle with 47 live cells after 40 iterations. A bit that spreads meets and mixes with the bits of later inputs. Under rule 204, $c_j \leftarrow c_j$, the bit stays where it was written, and the next input overwrites it, so nothing of the past survives.

Rule 210 shows that the behaviour of a single bit on a quiet ring is an incomplete description. From one cell it draws the same nested triangles as rule 90, because the 1s of that pattern are never

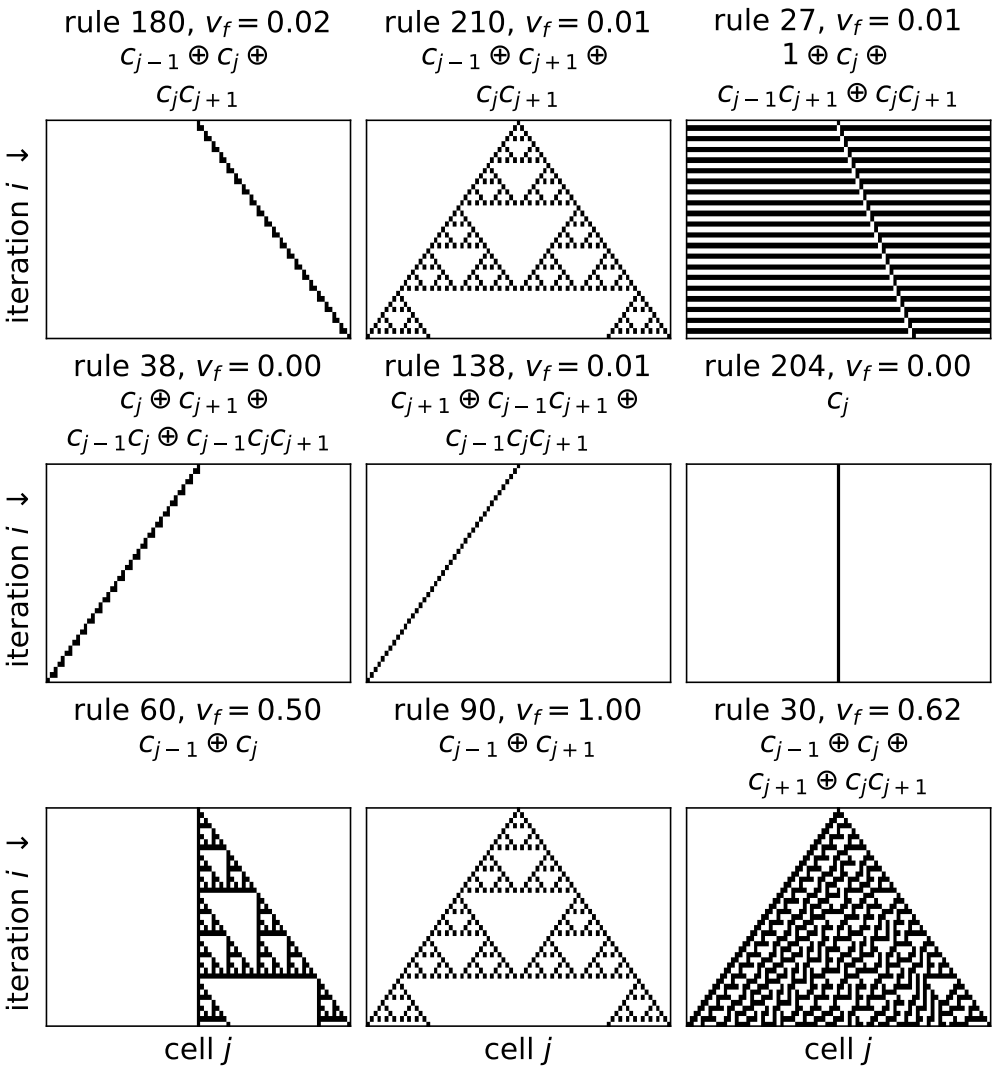


Figure 52: A single bit on a quiet ring of 81 cells over 40 iterations under nine elementary rules, iteration i downwards, black for state 1. Above each panel the rule number, the spreading rate v_f of equation (11.4) measured on a random background, and the algebraic normal form of the rule, with $\oplus$ the sum modulo 2 and products of cells written as $c_j c_{j+1}$.

neighbours and its product term $c_j c_{j+1}$ vanishes. On a random background, where the spreading rate is measured, the product term acts, and its damage spreads with $v_f = 0.01$ only. Rules 180 and 210 both copy the left neighbour and change it by a local exclusive or. A flipped cell on a random background then drifts to the right without spreading, by 0.99 and 1.00 cells per iteration, and the damaged region stays 2.9 and 2.0 cells wide on average after 100 iterations.

12.4 Memory of bits

The original test of the scheme is the five-bit memory task, which asks a system to store a short binary pattern over a long period of irrelevant input [45]. The input has four binary channels, and at every step one of them is 1 and the others are 0. During the first five steps channel 1 or channel 2 is on, which writes one bit of a pattern, so there are $2^5 = 32$ patterns. Then the third channel, the distractor, is on for $T_0 - 1$ steps. At step $T_0 + 5$ the fourth channel, the cue, is on once, and afterwards the distractor is on again until the end. With $T_0 = 200$ the pattern has to be kept over two hundred steps of distractor.

The system has four outputs, one per channel. Until the cue, output 3 must be 1 and the others 0. In the five steps after the cue, outputs 1 and 2 must repeat the pattern. For the pattern $1, 0, 1, 1, 0$, with the bit 1 written by channel 1 and the bit 0 by channel 2, the input channels on in the first five

steps are 1, 2, 1, 1, 2. Then channel 3 is on for 199 steps, channel 4 at step 205, and channel 3 again for the last five steps, in which outputs 1 and 2 must show 1, 0, 1, 1, 0. During the 199 distractor steps the input is the same at every step and carries no information, so the five bits have to be held by the configuration alone, while the rule is applied to it N_{gen} times per step and the distractor is written into the input cells at every step. In the picture of Figure 51 a bit that only travels reaches a written cell within at most L_c cells of travel and is overwritten, long before two hundred steps have passed, and this is what makes the task hard. One run feeds one of the 32 patterns, and it succeeds if every output at every step is within 0.5 of its target. One trial draws a reservoir, fits its readout on all 32 patterns and counts as successful only if all 32 runs succeed.

Yilmaz solved this task with elementary rules and reported the percentage of failed trials on a grid of R and N_{gen}. His paper leaves open how a new input is combined with the configuration and how many cells a compartment has. The runs shown here follow the implementation described by Nichele and Molund [43]. Since the input is already binary, no encoding is needed. Each compartment has 40 cells, and in each compartment four cells are chosen at random, once, one for each channel. At every input step the four bits overwrite these four cells of the configuration left by the previous step, the rule is iterated N_{gen} times, and the N_{gen} configurations after the writing form the feature vector, $RN_{\text{gen}} \cdot 40$ binary numbers. The configuration right after the writing, before the first iteration, is not part of these features. Each grid point has 10 trials, each with its own random choice of cells.

Both rules fail in every trial with a single compartment, and the failures fall as R and N_{gen} grow (Figure 53). At $R = N_{\text{gen}} = 4$ rule 90 fails in 70% of the trials and rule 150 in 100%. At $R = N_{\text{gen}} = 8$ rule 90 reaches 0% failed trials and rule 150 20%.

More compartments and more iterations plausibly increase the chance that copies of the injected bits spread into cells that the distractor does not overwrite and leave a trace that a linear readout can separate. How the readout decodes the pattern from the sums that rule 90 produces was not analysed. Training and testing use the same 32 patterns, so the readout has to separate 32 configurations, an easier task than decoding single bits from arbitrary inputs, as the recall test of Table 5 demanded.

At the one grid point that both studies share, $R = N_{\text{gen}} = 8$, Yilmaz reports 100% failed trials for rule 90, so only the trend can be compared, since the details that the paper leaves open change the outcome at a given R and N_{gen}.

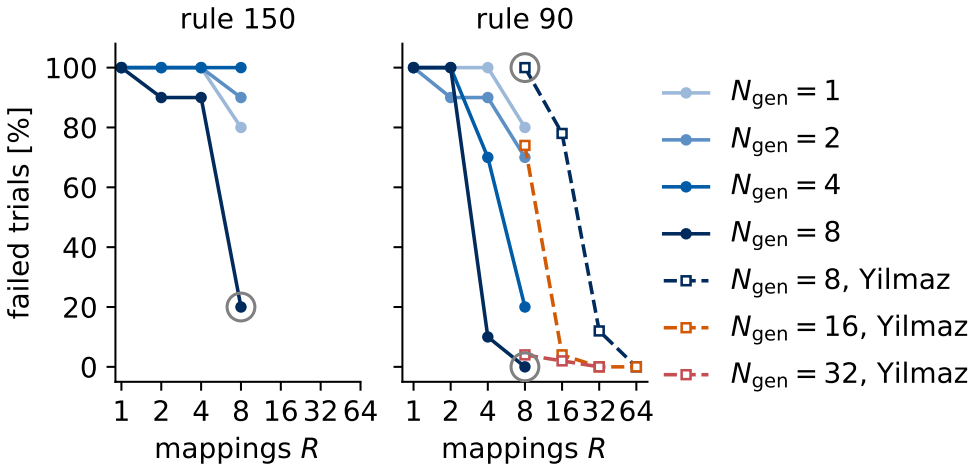


Figure 53: The five-bit memory task. Percentage of failed trials out of 10 against the number of compartments R (labelled mappings in the figure), one line per number of iterations N_{gen}. Left, rule 150. Right, rule 90, these runs (solid) and the table of Yilmaz (dashed), whose grid meets the grid of these runs only at $R = N_{\text{gen}} = 8$ (grey circle).

12.5 Writing a real number into cells

A measured record consists of real numbers, and an automaton holds discrete states. Kantic and coworkers split the step of the reservoir into stages [44], and the scheme used here follows them.

Encoding a record into a cellular automaton

1. The value s_k is quantised to one of Q levels, $\ell_k = \mathrm{clip}(\lfloor Q s_k \rfloor, 0, Q-1)$, where clipping to the range from 0 to $Q-1$ handles the test values that leave $[0,1]$.
2. The level is written as a code word $\mathbf{e}_k$ of B binary digits, with $Q = 2^B$.
3. The ring of $L = RL_c$ cells is divided into R compartments, consecutive segments of L_c cells, and the rule acts on the whole ring. In each compartment B cells are chosen at random, once, to receive the digits of the code word at every step.
4. The digits are written into the configuration $\mathbf{c}_{k-1}^{(N_{\mathrm{gen}})}$ left by the previous step, either by replacing the content of these cells or by adding to it modulo 2, while all other cells keep their content. The result is $\mathbf{c}_k^{(0)}$.
5. The rule is iterated N_{gen} times, $\mathbf{c}_k^{(i)} = F(\mathbf{c}_k^{(i-1)})$.
6. The feature vector is $\mathbf{r}_k = (\mathbf{c}_k^{(0)}, \ldots, \mathbf{c}_k^{(N_{\mathrm{gen}})})$, read as $(N_{\mathrm{gen}}+1)L$ real numbers, and the forecast is the readout of equation (2.1).

The code word can be the plain binary representation of the level or a Gray code, in which neighbouring levels differ in a single digit, so that a small change of the signal flips one cell. The binary Gray code of a level ℓ is $\ell \oplus \lfloor \ell/2 \rfloor$, the digit-wise exclusive or of ℓ with itself shifted by one place. In binary the step from level 3 to level 4 changes all three digits, $011 \to 100$, while in Gray code it changes one, $010 \to 110$. For $Q = 16$ levels a code word has $B = 4$ digits, and the value $s_k = 0.30$ gives $\ell_k = \lfloor 4.8 \rfloor = 4$, the binary word 0100 and the Gray word 0110.

Figure 54 follows two consecutive input steps through these stages on a small ring. Each sample becomes a level and a code word of three bits, the code word overwrites three cells chosen in each compartment, the same cells at every step, and the rule moves and combines the bits over three iterations, also called generations. The four rows of one step form its feature vector. The last row of step k is the configuration into which the code word of step $k+1$ is written, and only the framed cells change at that moment. The other cells carry what the earlier inputs left behind, which is the memory of the reservoir.

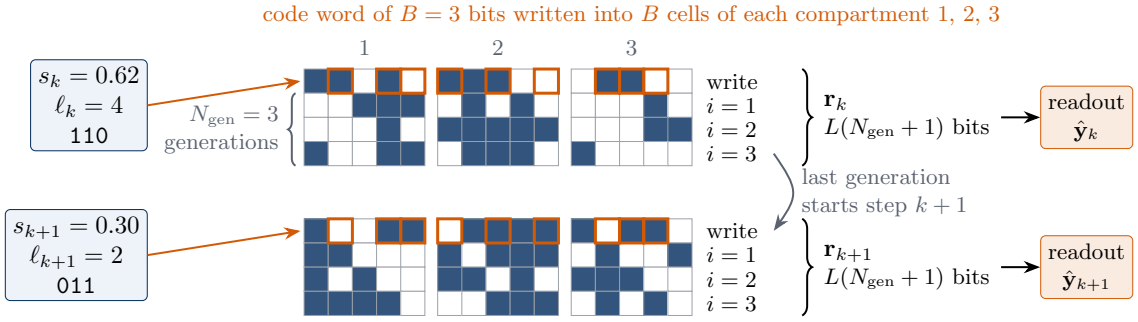


Figure 54: Two input steps of a cellular automaton reservoir with rule 180, $Q = 8$ levels, Gray code words of $B = 3$ bits, $R = 3$ compartments of $L_c = 5$ cells and $N_{\mathrm{gen}} = 3$ generations. Left, the sample, its level and its code word. Middle, the configuration after the code word is written (top row of each block, input cells framed in orange) and the generations $i = 1, 2, 3$ that follow, black for state 1. Right, the rows of one step form the feature vector $\mathbf{r}_k$ of the readout. The arrow marks that the last generation of step k is the start of step $k+1$. The configurations are computed for an illustrative random start.

Quantisation costs accuracy that no readout can restore. Within a level of width $1/Q$ the rounding error of a value spread evenly over the level is uniform on an interval of length $1/Q$, whose variance is $(1/Q)^2/12$, so its standard deviation is $1/(Q\sqrt{12})$, which is 0.072, 0.018 and 0.0045 for $Q = 4$, 16 and 64. On the laser record the measured root-mean-square error is 0.0690, 0.0180 and 0.0045 (Figure 55).

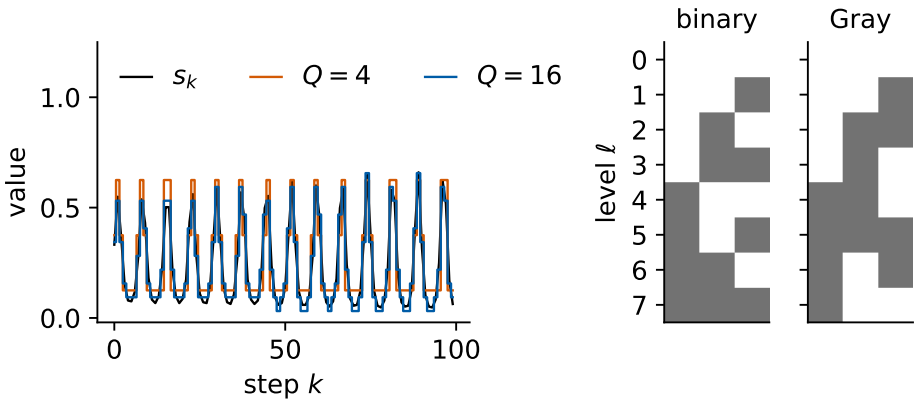


Figure 55: Encoding a real signal. Left, the laser record s_k and its quantised values $(\ell_k + 1/2)/Q$ for $Q = 4$ and $Q = 16$ levels against the step. Right, the code words of the levels 0 to 7 as rows of three bits (grey for 1), in binary and in Gray code.

12.6 A map of all elementary rules

The ring of Figure 51 and the bits of Figure 52 suggest that the rules that move a bit intact make the best reservoirs. A map of the forecast quality over all elementary rules puts this to a test. Many of the 256 rules are copies of each other. Reflecting a rule from left to right, $f'(c_{j-1}, c_j, c_{j+1}) = f(c_{j+1}, c_j, c_{j-1})$, or exchanging the roles of 0 and 1, $f'(c_{j-1}, c_j, c_{j+1}) = 1 - f(1 - c_{j-1}, 1 - c_j, 1 - c_{j+1})$, gives the same dynamics up to a mirror image or a complement of every configuration. The 256 rules fall into 88 such classes, each labelled by its smallest rule number. Rules 154, 166, 180 and 210 form one class, rule 210 being the complement of rule 180 and rule 166 its mirror image. Inside a reservoir the members of a class can still differ, because the written code word is not complemented with the rule and a mirrored lattice meets a different random placement.

Each entry of Figure 56 is the validation C_{tot} of one reservoir, with $Q = 16$ levels in Gray code, replacement, $R = 16$ compartments of $L_c = 16$ cells and one random placement of the input cells, for $N_{\mathrm{gen}} = 1, 2, 3, 4, 6, 8$, which gives 512 to 2304 features. The columns are the classes, ordered by the mean over the laser, MG 25 and all values of N_{gen}, and the strip on top colours each class by the spreading rate v_f of equation (11.4).

The first thing to look at is the left third of the map, which is dark on all three records while the strip above it is pale. All ten best classes have v_f of at most 0.032. The class of rule 27 is best by the mean, with 34.8, and the class of rule 154 (rules 154, 166, 180 and 210) gives the best single entries on the laser and MG 25, 39.1 on the laser and 41.3 on MG 25, both at $N_{\mathrm{gen}} = 8$. Near the right end of the map, next to the spreading rule 60, lie rules 0, 51 and 204, whose values do not change with N_{gen}. Under these rules nothing of an earlier input survives the next writing, and the readout sees only the present quantised value. The three records rank the rules alike, with a rank correlation, the Pearson correlation of the ranks of the values (Spearman), of +0.91 between the laser and each Mackey–Glass record.

The rows show what the number of generation steps does. Under a shift rule a bit written at one input step has moved N_{gen} cells when the next code word arrives, so N_{gen} sets how far apart successive inputs land in the lattice, and the class of rule 154 rises from 31.4 at $N_{\mathrm{gen}} = 1$ to 39.1 at $N_{\mathrm{gen}} = 8$ on the laser. Under a spreading rule every generation step scrambles the written code further. Rule 30 falls from 23.7 at $N_{\mathrm{gen}} = 1$ to 3.8 at $N_{\mathrm{gen}} = 8$ on the laser, and the mean of the 13 classes with $0.5 \le v_f \le 1$ falls from 20.7 to 8.4, while the mean of the 67 classes with $v_f < 0.05$ stays between 19.9 and 21.9.

The spreading rate and the forecast quality are linked in one direction only. Their correlation over the classes is −0.28 on the laser and −0.43 on MG 25, but the rank correlations are +0.20 and −0.05. The negative linear correlation comes from the few fast-spreading rules, none of which is among the 37 best classes, while among the slowly spreading rules v_f does not order the forecast quality, down to rule 204 with $v_f = 0.00$ near the bottom. Every good reservoir in this map spreads damage slowly,

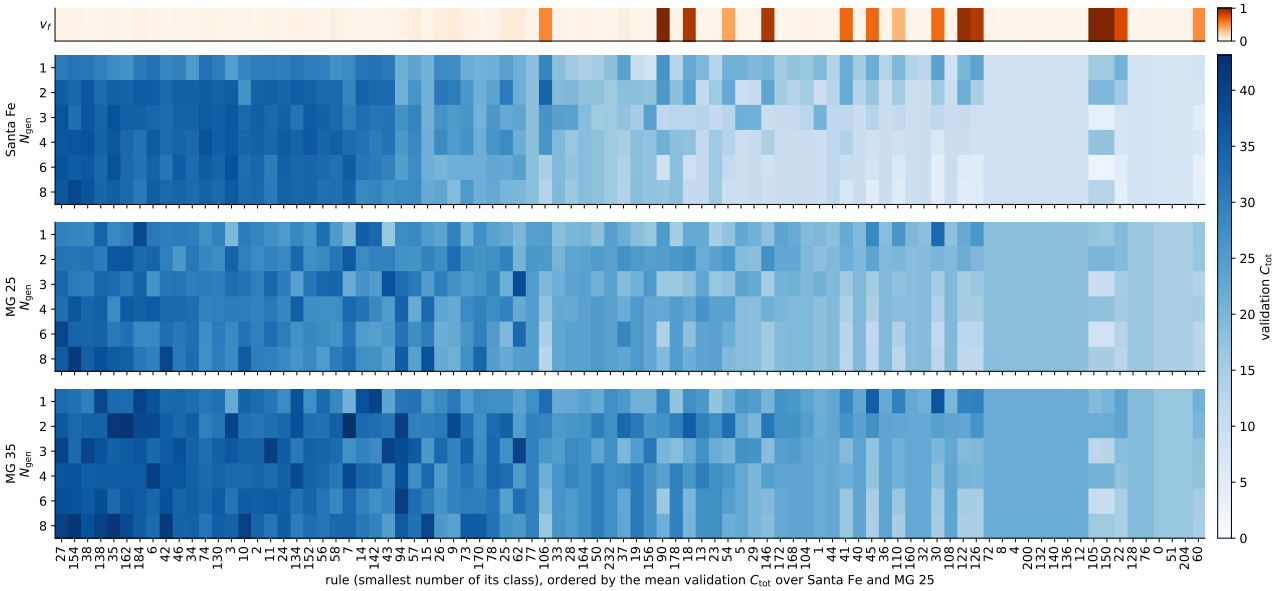


Figure 56: Forecast quality of all elementary rules. Validation C_{tot} of the reservoir for the 88 classes of elementary rules (columns, labelled by the smallest rule number of the class) and the numbers of generation steps N_{gen} (rows), for the laser (top), MG 25 and MG 35, on one colour scale. The columns are ordered by the mean over the laser, MG 25 and all N_{gen}, best on the left. The strip on top gives the spreading rate v_f of each class, from 0 (pale) to 1 (dark). One random placement per rule, $Q = 16$, $R = 16$, $L_c = 16$.

and many rules that spread damage slowly still make bad reservoirs, those that erase or freeze the input.

The map uses one random placement per rule. Repeated with five placements, the five best classes vary on the laser by a standard deviation of at most 0.70, while their means differ by up to 5.04, so there the class of rule 154 leads the class of rule 27 by more than the variation between placements, 39.2 against 37.3. On MG 25 the standard deviation reaches 1.94, the two classes have the same mean, and these runs do not resolve the order at the top. The map is a validation result for one quantisation, one lattice and one placement per rule, and the number of features grows with N_{gen}, so part of the gain of the shift rules at large N_{gen} may come from the larger readout.

12.7 The echo state property with input

Damage spreading has no input. In the reservoir the input overwrites only B cells of each compartment at every step, so the other cells need not forget the initial configuration, and the echo state property has to be tested with the input. Two copies of the forecasting reservoir start from different random configurations of the whole ring and receive the same encoded record, and the fraction of cells in which the configurations $\mathbf{c}_k^{(N_{\mathrm{gen}})}$ of the two copies differ is followed.

At the setting selected for the laser, $Q = 16$, $R = 16$ and $N_{\mathrm{gen}} = 4$, the shift rules lock in all 5 pairs of starts, after at most 26 steps for rule 180 and 25 for rule 210. Rules 90, 30 and 4 do not lock within the 4000 steps in any pair, and on the test origins the copies still differ in the fractions 0.383, 0.487 and 0.090 of the cells. Under rule 4 an isolated cell in state 1 survives as long as its neighbours are 0 and no cell changes from 0 to 1, so a difference on a cell away from the written cells can persist. Under rules 90 and 30 a difference plausibly spreads faster than the writing removes it. Under the shift rules every difference travels along the compartment until it reaches a written cell, where it is overwritten, as in the twelve-cell ring of Figure 51. Within these finite runs the echo state property holds for the shift rules and fails for rules 90, 30 and 4, and the rules that forecast well are the ones whose state becomes a function of the input history alone.

12.8 Forecasts of the automaton reservoir

The forecasting search first compares a few rules on the validation block, among them 32, 4, 90, 150, 30, 110, 180 and 210, at $Q = 16$, $R = 16$ and $N_{\mathrm{gen}} = 4$. It selects rule 180 for four records and rule 210 for MG 35, both members of the class with the best single entries on the laser and MG 25 in the map of Figure 56. A second search for each record then scans $Q \in \{4, 16, 64\}$, $N_{\mathrm{gen}} \in \{1, 2, 4, 8\}$ and $R \in \{4, 8, 16\}$ with this rule, with compartments of $L_c = 16$ cells, Gray code and replacement. Addition in place of replacement destroys the readability of the code, and with it the validation C_{tot} stays at most 22.5 on the four irregular records. The number of features is $(N_{\mathrm{gen}} + 1)RL_c$, from 128 at $R = 4$ and $N_{\mathrm{gen}} = 1$ to 2304 at $R = 16$ and $N_{\mathrm{gen}} = 8$. At every selected setting the two copies of the echo state test lock in all 5 pairs, after at most 24 to 28 steps for the settings with $Q = 16$. The setting selected for MG 15, with $Q = 4$ and $N_{\mathrm{gen}} = 2$, needs 169 steps, more than the washout, so there the first rows of the fit still depend on the start. The search thus lands on the shift rules that the map singled out, and at the selected settings these rules also pass the echo state test.

test C_{tot}	Lorenz	MG 15	MG 25	MG 35	Santa Fe
AR	15.4	49.8	33.1	38.0	16.5
NVAR	15.4	50.0	45.5	48.6	34.7
ESN, 300 features	39.2	50.0	48.3	49.8	34.7
automaton, 320 features	29.0	49.9	34.5	40.9	28.6
automaton, selected	40.5	50.0	42.8	44.8	33.3
rule	180	180	180	210	180
(Q, N_{gen}, R)	(16, 8, 16)	(4, 2, 16)	(16, 8, 16)	(16, 8, 16)	(16, 4, 16)
features	2304	768	2304	2304	1280

Table 6: Total capacity on the test block of the cellular automaton reservoir at a setting with 320 features, $R = 4$ compartments of 16 cells and $N_{\mathrm{gen}} = 4$ with the selected rule and Q chosen on the validation block, and at the setting selected on the validation block, next to AR, NVAR and the echo state network.

The question is whether a reservoir of bits, which sees the record only through a few quantisation levels, can compete with the echo state network. Table 6 and Figure 57 answer it. On Lorenz the automaton reaches 40.5, above the 39.2 of the echo state network, and on the laser 33.3 against 34.7, a difference within the spread of the echo state network over random reservoirs. On MG 25 and MG 35 it stays about five units below. The comparison is not at equal size. The selected automata use up to 7.7 times the 300 features of the echo state network, and at 320 features they fall behind it on every irregular record. A bit carries less than a real number, so the automaton needs more of them. At the setting selected for the laser the Gray code gives a validation C_{tot} of 38.0 against 35.1 for the binary code, in line with its property that a small change of the signal flips one cell.

The one-step forecasts of Figure 58 show where the automaton loses. Look at the scatter around the diagonal, which is visibly wider than for the echo state network, with $C = 0.938$ on the laser against 0.987. Two causes are likely. The input is quantised, so the automaton never sees changes of the signal smaller than one level, 1/16 of the range at the selected Q. And its state is a pattern of bits, which the next writing partly overwrites, while a unit of the echo state network holds a real number that responds smoothly to the input. A hypothesis that these data suggest and do not test is that the automaton does best where the forecast needs a long memory and coarse amplitude information, as for Lorenz, and falls behind where fine amplitude resolution over a short window matters. These conclusions hold for the rules, encodings and grids tested and one random placement per setting.

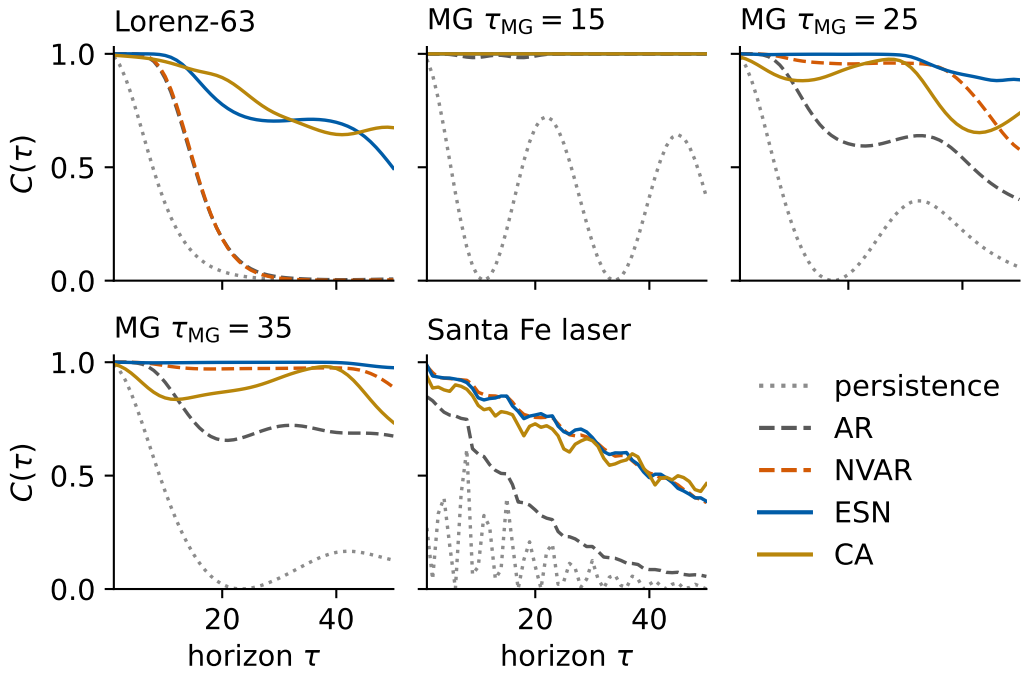


Figure 57: Capacity $C(\tau)$ on the test block against the horizon for persistence, AR, NVAR, the echo state network and the selected cellular automaton reservoir (CA), one panel per record.

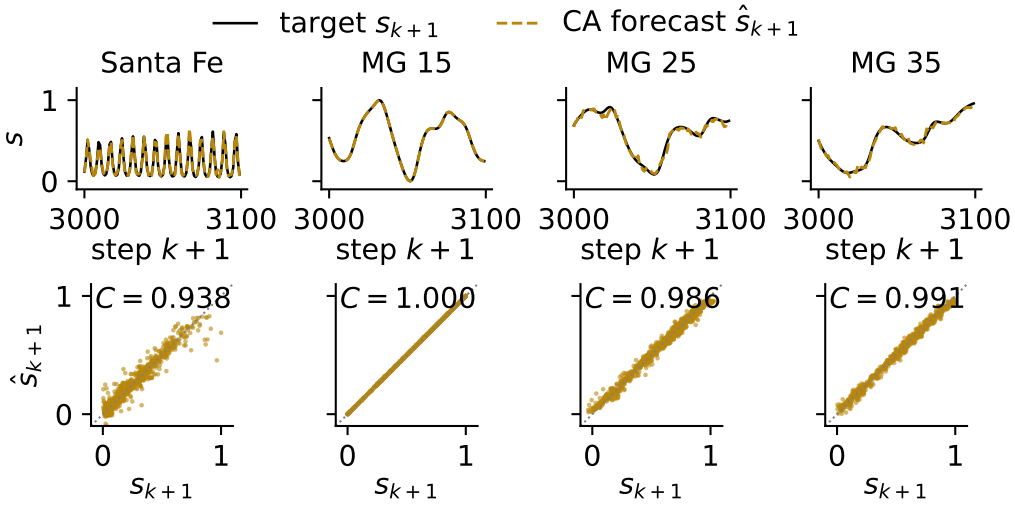


Figure 58: One-step forecasts of the cellular automaton reservoir on the test block. Top, the target s_{k+1} and the forecast for the first 100 test targets of the laser and the three Mackey–Glass records. Bottom, the forecast against the target, with the capacity C.

12.9 What the automaton adds

The rules with the best single entries on the laser and MG 25, the class of rule 154 (rules 154, 166, 180 and 210), copy a neighbour and add a local product, as the normal form $c_{j-1} \oplus c_j \oplus c_j c_{j+1}$ of rule 180 shows. The best class by the mean, that of rule 27, has the normal form $1 \oplus c_j \oplus c_j c_{j+1} \oplus c_{j-1} c_{j+1}$, with no linear term of a neighbour, and moves a bit by half a cell per iteration. Their forecasts may come from the transport of the written code bits alone, which turns the lattice into a delay line of bit-coded past values, or from the local products, which add nonlinear features. A control model without any automaton separates the two. Its feature vector at step k holds the Gray-code bits of the last n_{lag} samples $s_k, \ldots, s_{k-n_{\mathrm{lag}}+1}$, with $Q = 16$ levels and $B = 4$ bits, a shift register of code words with zero code words before the start of the record, and the readout is the ridge regression of the automaton reservoir. Three variants are compared, a register of $n_{\mathrm{lag}} = 64$ lags, a register with as many features as the selected automaton of the record, $n_{\mathrm{lag}} = 576$ lags on Lorenz and 320 on the

laser, and a register that adds to the bits of n_{lag} lags all 16 products of a bit of lag d with a bit of lag $d+1$, with n_{lag} chosen to match the size of the automaton. A linear readout of these products and bits can form the exclusive or and the product of any bit of one lag with any bit of the next, the kind of terms that rule 180 forms between neighbouring cells.

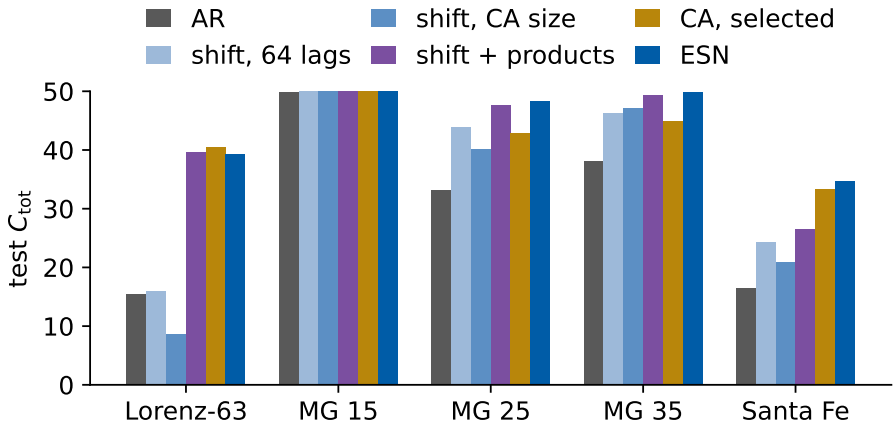


Figure 59: The shift-register control. Test C_{tot} of AR, of three shift registers of Gray-coded past samples with a linear readout (64 lags, as many features as the selected automaton, and with products of adjacent lags), of the selected cellular automaton reservoir and of the echo state network, for the five records.

Figure 59 answers the question in two parts. On MG 25 and MG 35 the plain register of 64 lags matches or beats the automaton, 43.9 and 46.3 against 42.8 and 44.8. For these records the intact transport of quantised past values accounts for the forecast, and the automaton adds nothing over a delay line. On Lorenz and on the laser the plain register falls far behind, with 16.0 on Lorenz against 40.5 and 24.3 on the laser against 33.3. The products of adjacent lags raise the register to 39.5 on Lorenz, close to the automaton, and to 26.5 on the laser, where the automaton stays ahead by 6.8. On these two records nonlinear products of neighbouring code bits are needed, and on the laser the products that rule 180 forms between cells brought next to each other by the shift did better than this register design. The long register with as many features as the automaton does worse than the short one on Lorenz and the laser, plausibly because a readout of so many binary features overfits the 2387 training origins. The comparison uses one quantisation, one test block per record and one of many possible register designs.

12.10 Langton's parameter and the edge of chaos

The 256 elementary rules need an order. Langton proposed a single parameter [41]. One state is called quiescent, here 0, and the rule must map the all-quiescent neighbourhood to it. Langton's parameter λ_L is the fraction of entries of the rule table that map to a state other than the quiescent one. For a rule with $000 \to 0$, one of the 128 even rule numbers, λ_L is the number of 1 digits of the rule number in binary divided by 8, so rule 32 has $\lambda_L = 1/8$ and rules 90, 30, 150, 180 and 210 have $\lambda_L = 1/2$. For $\lambda_L = 0$ every configuration dies. Langton observed that, on average, as λ_L grows the dynamics passes from fixed and periodic patterns to complex and then to chaotic ones, with the complex behaviour in an intermediate range of λ_L, which he called the edge of chaos. The idea that computation works best there has been carried over to reservoirs as a hypothesis.

The test uses the 128 elementary rules with $000 \to 0$, whose values of λ_L are $0, 1/8, \ldots, 1$. For every rule the spreading rate v_f of equation (11.4) is measured after 100 iterations, and a reservoir built from the rule with $Q = 16$, $R = 8$ and $N_{\mathrm{gen}} = 4,640$ features, is scored on the laser and on MG 25. On average the spreading rate grows with λ_L, with a large spread, since λ_L alone does not fix the dynamics of a rule. The shift rule 240 of Figure 51 is an example, with $\lambda_L = 1/2$ like rule 90 and no spreading at all. If the edge of chaos were the best place for a reservoir, the forecast quality would peak where spreading sets in. The left panel of Figure 60 shows no such peak. Over the 128 rules the correlation between the spreading rate and the validation C_{tot} is -0.24 on the laser and -0.32 on

MG 25, and the best rules are rule 166 on the laser, the mirror image of rule 180, and rule 180 on MG 25, both shift rules with $\lambda_L = 0.500$ and spreading rates of at most 0.02. For forecasting with a linear readout these data do not support the hypothesis that the best reservoirs sit at the edge of chaos. They favour rules that carry information along the lattice without scrambling it, the behaviour of the shift rules in Figure 51. The test covers two records, one size of the reservoir and one random placement per rule. The map of Figure 56, which includes the rules with $000 \to 1$, gives the same answer, with rank correlations close to zero between the spreading rate and the forecast quality.

The memory task of bits orders the rules differently (Figure 60). There spreading is needed. In Figure 51 a bit that only travels reaches a written cell within at most L_c cells of travel and is overwritten, far sooner than two hundred steps, so the five bits survive the distraction only if copies of them spread to many cells. The six rules that Yilmaz found useless, 32, 160, 4, 108, 218 and 250, have spreading rates of at most 0.01 and fail in every trial of these runs. Spreading is not sufficient, since several fast-spreading rules, 22, 30, 126 and 182, also fail in every trial at this size, rule 90, which is linear modulo 2, does best with no failed trial, and rule 150 ties with rule 54 at 20% failed trials. The two tasks ask for different properties of the reservoir, long storage of a few bits in one case and a readable record of the recent real-valued input in the other, and a single parameter of the rule does not capture both.

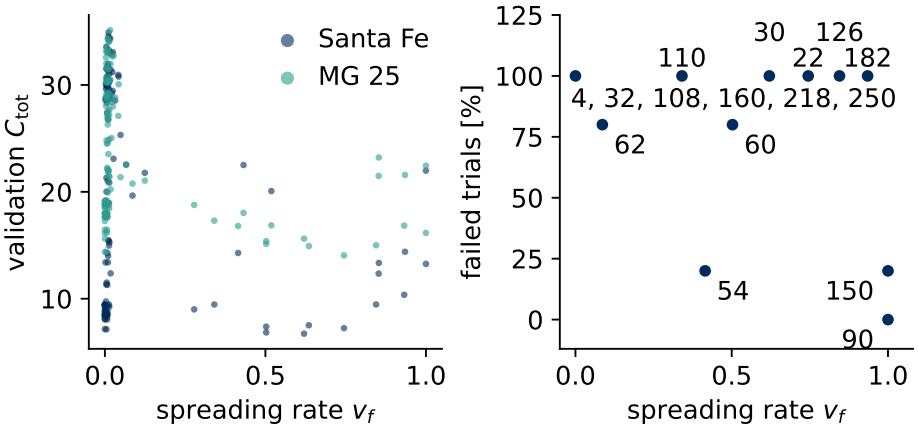


Figure 60: Damage spreading and reservoir quality. Left, the validation C_{tot} of the reservoirs built from the 128 elementary rules with a quiescent state against the spreading rate v_f, for the laser and MG 25. Right, the percentage of failed trials of the five-bit task at $R = N_{\mathrm{gen}} = 8$, with 5 trials per rule, against v_f for the sixteen rules named by Yilmaz.

The three reservoirs of this chapter, random hyperbolic tangents, spiking neurons and a shifting pattern of bits, share no physics. What they share is the behaviour that the tests measured, a state that forgets its start and its distant past, keeps a readable trace of its recent input and transforms it nonlinearly. Where these tests passed, a linear readout fitted by one linear solve forecast the records. Where they failed, as for the automaton rules 90, 30 and 4, the scores were low, and for the spiking network the coupling $J = 0.03$, at which the two copies no longer lock, lowered the validation scores on four records, most clearly on the laser. The closed loops at $\rho = 1.2$ were not put to these tests, and they failed by the climate criterion.

References

1. A. S. Weigend, N. A. Gershenfeld (eds.), Time Series Prediction, Forecasting the Future and Understanding the Past (Addison-Wesley, 1994, reissued by Routledge, 2018), doi:10.4324/9780429492648

2. E. N. Lorenz, Deterministic nonperiodic flow, J. Atmos. Sci. **20**, 130 (1963), doi:10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2

3. M. C. Mackey, L. Glass, Oscillation and chaos in physiological control systems, Science **197**, 287 (1977), doi:10.1126/science.267326

4. J. L. Elman, Finding structure in time, Cogn. Sci. **14**, 179 (1990), doi:10.1207/s15516709cog1402_1
5. Y. Bengio, P. Simard, P. Frasconi, Learning long-term dependencies with gradient descent is difficult, IEEE Trans. Neural Netw. **5**, 157 (1994), doi:10.1109/72.279181
6. R. Pascanu, T. Mikolov, Y. Bengio, On the difficulty of training recurrent neural networks, Proc. 30th Int. Conf. Mach. Learn., PMLR **28**, 1310 (2013), arXiv:1211.5063
7. A. F. Atiya, A. G. Parlos, New results on recurrent network training, IEEE Trans. Neural Netw. **11**, 697 (2000), doi:10.1109/72.846741
8. D. J. Gauthier, E. Bollt, A. Griffith, W. A. S. Barbosa, Next generation reservoir computing, Nat. Commun. **12**, 5564 (2021), doi:10.1038/s41467-021-25801-2
9. B. O. Koopman, Hamiltonian systems and transformation in Hilbert space, Proc. Natl. Acad. Sci. USA **17**, 315 (1931), doi:10.1073/pnas.17.5.315
10. E. Bollt, On explaining the surprising success of reservoir computing forecaster of chaos, Chaos **31**, 013108 (2021), doi:10.1063/5.0024890
11. A. E. Hoerl, R. W. Kennard, Ridge regression, Biased estimation for nonorthogonal problems, Technometrics **12**, 55 (1970), doi:10.1080/00401706.1970.10488634
12. H. Jaeger, The echo state approach to analysing and training recurrent neural networks, GMD Report 148 (2001), https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf
13. M. Lukoševičius, H. Jaeger, Reservoir computing approaches to recurrent neural network training, Comput. Sci. Rev. **3**, 127 (2009), doi:10.1016/j.cosrev.2009.03.005
14. W. Maass, T. Natschläger, H. Markram, Real-time computing without stable states, Neural Comput. **14**, 2531 (2002), doi:10.1162/089976602760407955
15. I. B. Yildiz, H. Jaeger, S. J. Kiebel, Re-visiting the echo state property, Neural Netw. **35**, 1 (2012), doi:10.1016/j.neunet.2012.07.005
16. S. Boyd, L. O. Chua, Fading memory and the problem of approximating nonlinear operators with Volterra series, IEEE Trans. Circuits Syst. **32**, 1150 (1985), doi:10.1109/TCS.1985.1085649
17. J. Dambre, D. Verstraeten, B. Schrauwen, S. Massar, Information processing capacity of dynamical systems, Sci. Rep. **2**, 514 (2012), doi:10.1038/srep00514
18. H. Jaeger, H. Haas, Harnessing nonlinearity, Predicting chaotic systems and saving energy in wireless communication, Science **304**, 78 (2004), doi:10.1126/science.1091277
19. J. Pathak, Z. Lu, B. R. Hunt, M. Girvan, E. Ott, Using machine learning to replicate chaotic attractors and calculate Lyapunov exponents from data, Chaos **27**, 121102 (2017), doi:10.1063/1.5010300
20. Z. Lu, B. R. Hunt, E. Ott, Attractor reconstruction by machine learning, Chaos **28**, 061104 (2018), doi:10.1063/1.5039508
21. G. Benettin, L. Galgani, A. Giorgilli, J.-M. Strelcyn, Lyapunov characteristic exponents for smooth dynamical systems and for Hamiltonian systems, a method for computing all of them, Part 1, Theory, Meccanica **15**, 9 (1980), doi:10.1007/BF02128236, and Part 2, Numerical application, Meccanica **15**, 21 (1980), doi:10.1007/BF02128237
22. G. Tanaka et al., Recent advances in physical reservoir computing, A review, Neural Netw. **115**, 100 (2019), doi:10.1016/j.neunet.2019.03.005
23. L. Grigoryeva, J.-P. Ortega, Echo state networks are universal, Neural Netw. **108**, 495 (2018), doi:10.1016/j.neunet.2018.08.025
24. C. Fernando, S. Sojakka, Pattern recognition in a bucket, in Advances in Artificial Life, Lect. Notes Comput. Sci. **2801**, 588 (2003), doi:10.1007/978-3-540-39432-7_63
25. H. Hauser, A. J. Ijspeert, R. M. Füchslin, R. Pfeifer, W. Maass, Towards a theoretical foundation for morphological computation with compliant bodies, Biol. Cybern. **105**, 355 (2011), doi:10.1007/s00422-012-0471-0
26. L. Appeltant et al., Information processing using a single dynamical node as complex system, Nat. Commun. **2**, 468 (2011), doi:10.1038/ncomms1476
27. C. Du, F. Cai, M. A. Zidan, W. Ma, S. H. Lee, W. D. Lu, Reservoir computing using dynamic memristors for temporal information processing, Nat. Commun. **8**, 2204 (2017), doi:10.1038/s41467-017-02337-y

28. M. Davies et al., Loihi, A neuromorphic manycore processor with on-chip learning, IEEE Micro **38**, 82 (2018), doi:10.1109/MM.2018.112130359

29. G. Gallego et al., Event-based vision, A survey, IEEE Trans. Pattern Anal. Mach. Intell. **44**, 154 (2022), doi:10.1109/TPAMI.2020.3008413

30. L. F. Abbott, Lapicque's introduction of the integrate-and-fire model neuron (1907), Brain Res. Bull. **50**, 303 (1999), doi:10.1016/S0361-9230(99)00161-6

31. N. Brunel, Dynamics of sparsely connected networks of excitatory and inhibitory spiking neurons, J. Comput. Neurosci. **8**, 183 (2000), doi:10.1023/A:1008925309027

32. Z. F. Mainen, T. J. Sejnowski, Reliability of spike timing in neocortical neurons, Science **268**, 1503 (1995), doi:10.1126/science.7770778

33. E. O. Neftci, H. Mostafa, F. Zenke, Surrogate gradient learning in spiking neural networks, IEEE Signal Process. Mag. **36**, 51 (2019), doi:10.1109/MSP.2019.2931595

34. F. Zenke, S. Ganguli, SuperSpike, Supervised learning in multilayer spiking neural networks, Neural Comput. **30**, 1514 (2018), doi:10.1162/neco_a_01086

35. S. Wolfram, Statistical mechanics of cellular automata, Rev. Mod. Phys. **55**, 601 (1983), doi:10.1103/RevModPhys.55.601

36. S. Wolfram, Universality and complexity in cellular automata, Physica D **10**, 1 (1984), doi:10.1016/0167-2789(84)90245-8

37. M. Gardner, The fantastic combinations of John Conway's new solitaire game life, Sci. Am. **223** (4), 120 (1970), doi:10.1038/scientificamerican1070-120

38. A. W. Burks, Von Neumann's self-reproducing automata, Technical report, Logic of Computers Group, University of Michigan (1969), doi:10.21236/AD0688840

39. U. Frisch, B. Hasslacher, Y. Pomeau, Lattice-gas automata for the Navier–Stokes equation, Phys. Rev. Lett. **56**, 1505 (1986), doi:10.1103/PhysRevLett.56.1505

40. K. Nagel, M. Schreckenberg, A cellular automaton model for freeway traffic, J. Phys. I France **2**, 2221 (1992), doi:10.1051/jp1:1992277

41. C. G. Langton, Computation at the edge of chaos, Phase transitions and emergent computation, Physica D **42**, 12 (1990), doi:10.1016/0167-2789(90)90064-V

42. O. Yilmaz, Reservoir computing using cellular automata, arXiv:1410.0162

43. S. Nichele, A. Molund, Deep reservoir computing using cellular automata, arXiv:1703.02806

44. J. Kantic, F. C. Legl, W. Stechele, J. Hermann, ReLiCADA, Reservoir computing using linear cellular automata design algorithm, Complex Intell. Syst. **10**, 3593 (2024), doi:10.1007/s40747-023-01330-x

45. H. Jaeger, Long short-term memory in echo state networks, Details of a simulation study, Jacobs University Technical Report 27 (2012), https://www.ai.rug.nl/minds/uploads/2478_Jaeger12.pdf